DESIGNING A PRIVACY-PRESERVING, OFFLINE MACHINE LEARNING SYSTEM FOR MALICIOUS URL DETECTION USING URL FEATURES

BY

CONNIE RODRIGUEZ

AN ADVANCED DESIGN PROJECT SUBMITTED TO THE FACULTY OF SAINT MARTIN'S UNIVERSITY IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF MASTER OF SCIENCE IN COMPUTER SCIENCE

SAINT MARTIN'S UNIVERSITY

MAY 7, 2026

CSC 598 CERTIFICATE OF APPROVAL

DESIGNING A PRIVACY-PRESERVING, OFFLINE MACHINE LEARNING SYSTEM FOR MALICIOUS URL DETECTION USING URL FEATURES

BY

CONNIE RODRIGUEZ

_____

Date of Acceptance

Special Committee directing the

Master's degree for Candidate

_____

Dr. Dvorak, Professor

Project Adviser

_____

Professor: _____

Committee Member

_____

Professor: _____

Ex-Officio

Abstract

This research presents the design, implementation, and evaluation of a privacy-preserving, offline machine learning system for malicious URL detection using URL-based features. Unlike many current malicious URL detection approaches that rely on external data sources such as Domain Name System (DNS) queries, WHOIS records, or webpage content analysis, this system operates solely on features derived from the URL string. This constraint enables deployment in air-gapped (i.e., isolated from external networks), privacy-sensitive, and resource-constrained environments, while supporting reproducibility and eliminating dependencies on external services. This design directly targets a gap in existing literature, where offline, privacy-preserving detection frameworks remain underexplored relative to their practical deployment need in isolated and constrained environments.

The system is implemented as a research-oriented prototype using a modular, multi-stage pipeline. The pipeline performs dataset construction, preprocessing, feature engineering (deriving lexical and structural attributes from URLs), model training, and evaluation using two real-world threat intelligence sources: PhishTank, a community verified repository of phishing URLs, and URLhaus, a curated feed of malware distribution URLs, balanced against legitimate domains from the Tranco Top Sites list. A total of 24 lexical and structural URL features are engineered and applied consistently across datasets to ensure methodological integrity and prevent data leakage. Three lightweight machine learning models, Logistic Regression, Decision Tree, and Random Forest, are trained and evaluated using stratified sampling and five-fold cross-validation.

A central contribution of this work is the evaluation of cross-dataset generalization, where models trained on one threat intelligence source are tested on another to assess their ability to maintain performance across different data sources and real-world variability. Additionally, the system introduces a single-responsibility pipeline architecture, enhancing transparency, reproducibility, and extensibility for academic and research applications.

This study contributes to cybersecurity research by analyzing the trade-offs between detection accuracy, generalization capability, computational efficiency, and privacy preservation in a fully offline detection framework. The evaluation addresses the viability of URL-only feature-based detection while examining the limitations and challenges associated with operating under strict data and environmental constraints. This work establishes a reproducible framework for future research and offers insights relevant to the potential deployment of lightweight malicious URL detection systems in constrained environments.

List of Figures and Tables

List of Figures

List of Tables

Table of Contents

CHAPTER 1: INTRODUCTION

1.1 Introduction

Phishing remains one of the most persistent and damaging forms of cybercrime, responsible for credential theft, financial fraud, identity compromise, and the initial access phase of countless other attacks [1]. By disguising malicious links as legitimate ones, attackers exploit human trust rather than software vulnerabilities, making phishing both technically simple and socially effective. Global cybersecurity reports consistently rank phishing as the leading initial attack vector, with millions of malicious URLs generated daily and increasingly sophisticated techniques appearing across the web, mobile, and email platforms [1], [2].

Despite rising user awareness, phishing remains effective because attackers rapidly adapt URL structures, domain variations, and obfuscation techniques. Traditional defensive mechanisms, such as blacklists, signature-based filters, or heuristic rules, struggle to keep pace with these evolving threats. Blacklists can only block previously identified malicious domains; heuristics can be easily bypassed through minor variations in URL length, token placement, or character composition [3]. As a result, organizations and individuals remain vulnerable to emerging zero-day phishing URLs that do not resemble known malicious patterns.

This challenge motivates the need for adaptive, data-driven detection systems that can generalize beyond known examples. Machine learning (ML) offers this adaptability. Instead of relying on fixed rules, ML models learn statistical patterns of malicious versus legitimate URLs and can classify newly observed URLs in near real-time. URL-based detection, where the classifier evaluates only the URL text rather than webpage content, offers distinct advantages. It is fast, lightweight, does not require page rendering, and is suitable for deployment in browsers, email gateways, mobile devices, and resource-constrained environments [3].

However, a critical and underexplored constraint in practical deployment scenarios is the need to operate in a fully offline, privacy-preserving environment. Many existing malicious URL detection systems rely on supplementary inputs such as Domain Name System (DNS) queries, WHOIS registration data, webpage content inspection, or cloud-based threat intelligence feeds during classification [3]. These dependencies introduce limitations in terms of data exposure risk, reliance on network connectivity, reduced reproducibility, and limited deployability in restricted or air-gapped environments. For security tools deployed on endpoint devices, within organizational intranets, or in environments where URL resolution itself poses a risk, these external dependencies are not acceptable constraints.

This project addresses that gap by implementing a privacy-preserving, offline machine learning pipeline for malicious URL detection. The system operates exclusively on 24

lexical and structural features derived from the URL string itself, with no DNS queries, WHOIS lookups, Hypertext Transfer Protocol (HTTP) requests, or external Application Programming Interface (API) calls permitted at any stage of the pipeline. This design enables deployment in air-gapped, privacy-sensitive, and resource-constrained environments while supporting full reproducibility and eliminating runtime dependencies on external services.

The system is evaluated using two real-world threat intelligence sources: PhishTank [4], a community-verified repository of phishing URLs, and URLhaus [5], a curated feed of malware distribution URLs, balanced against legitimate domains from the Tranco Top Sites list [6]. Three lightweight machine learning classifiers, Logistic Regression, Random Forest, and Decision Tree, are trained and evaluated using stratified sampling, which preserves the proportion of malicious and legitimate URLs across training and testing sets, and five-fold cross-validation, where the data is repeatedly split into five parts to provide a more reliable estimate of model performance. A central component of evaluation is cross-dataset generalization testing, where models are trained on one threat intelligence source and evaluated on the other in both directions, assessing the system's ability to maintain detection performance across distinct real-world URL distributions.

The study seeks to answer five core research questions:

1. Can 24 URL-string-only features, derived exclusively from the URL string, without reliance on DNS queries, WHOIS lookups, or external API calls, support strong phishing detection performance across two independently sourced real-world threat intelligence feeds in a fully offline, privacy-preserving environment?
2. Which URL-string features demonstrate the highest predictive stability across the datasets from different threat intelligence sources?
3. Which lightweight machine learning algorithm provides the best trade-off between classification performance and deployment feasibility for real-world phishing detection?
4. How well does a model trained on PhishTank URLs generalize when evaluated against URLhaus data, and vice versa? What performance gap, if any, exists between the two directions, and what might it reveal about the structural differences between phishing and malware delivery URLs?
5. What measurable improvements over the Iteration 1 baseline can be demonstrated through iterative feature selection and optimization across subsequent experimental rounds?

By addressing these questions, this project makes a focused and original contribution to cybersecurity research. It presents the design, implementation, and rigorous evaluation of a fully offline, privacy-preserving malicious URL detection system that operates solely

on URL-string-derived features, without external lookups, network queries, or runtime dependencies. While existing research frequently demonstrates strong detection performance by incorporating supplementary external signals, this work deliberately excludes them to establish what is achievable within the strict constraints of air-gapped, privacy-sensitive, and resource-constrained deployment environments. This design space remains underexplored in the literature relative to its practical relevance, and this project directly addresses that gap. The resulting framework provides a reproducible, transparent, and extensible baseline for future researchers and practitioners seeking to deploy lightweight malicious URL detection in environments where system independence and data privacy are non-negotiable requirements.

1.2 Background

The field of phishing detection has evolved substantially over the past few decades. Early systems focused on manually curated blacklists and heuristic rules. While computationally inexpensive, these methods are reactive and limited. Blacklists cannot identify newly generated malicious URLs, and heuristics often produce a high false positive rate or fail entirely when attackers insert deceptive tokens, alter domain structures, or use randomized characters.

The emergence of machine learning significantly advanced the field by enabling dynamic classification based on learned patterns in URL characteristics [7]. Traditional ML approaches typically rely on handcrafted lexical and structural features, such as URL length, the number of dots or hyphens, presence of digits, or whether the URL contains suspicious tokens. Algorithms such as Logistic Regression, Support Vector Machines, Random Forests, and Decision Trees demonstrated strong baseline performance [3]. These models offer interpretability and low computational overhead, making them suitable candidates for deployment in operational security tools, though they may struggle against sophisticated obfuscation and rapidly evolving phishing techniques.

More recently, deep learning approaches, including character-level convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Transformer-based models, have been applied to URL classification, automatically learning representations without manual feature engineering [8]. These models have demonstrated strong performance by capturing complex lexical patterns and contextual relationships within URL strings. However, their deployment in real-time or computationally constrained environments is often limited by computational requirements, memory usage, inference latency, and reduced interpretability [8]. For environments where explainability, low overhead, and offline operation are required, lightweight interpretable classifiers remain the more appropriate choice [9].

A growing body of research highlights a key challenge that affects both lightweight and deep learning approaches equally generalization across datasets. Models trained on one dataset often exhibit significant performance degradation when tested on another dataset collected from a different time period, geographic region, source platform, or threat intelligence feed. This phenomenon, broadly referred to as dataset shift or domain shift, reveals inherent differences in URL distribution, phishing techniques, and labeling practices across sources [10]. As phishing campaigns evolve rapidly, and threat intelligence feeds capture different segments of the malicious URL ecosystem, a model's performance on a single dataset is not a reliable indicator of real-world deployment performance.

This challenge is particularly relevant when comparing phishing-specific feeds such as PhishTank, which captures credential-harvesting URLs, typically characterized by long obfuscated paths and deceptive domain structures, against malware delivery feeds such as URLhaus, which tracks botnet infrastructure URLs often characterized by IP-based hosts, higher proportions of numeric characters, and structural patterns distinct from traditional phishing [4], [5]. Models trained on one type of threat may not generalize well to the other, and quantifying this asymmetry is essential for understanding the practical limitations of URL-only detection [10], [11].

An additional constraint that has received comparatively little attention in the literature is the requirement to operate without any external data sources at inference time. Many published phishing detection systems that demonstrate strong performance incorporate external supplementary signals. While these supplementary signals improve accuracy, they introduce dependencies that are incompatible with privacy-sensitive, air-gapped, or resource-constrained deployment environments. The design space of fully offline, URL-string-only detection systems capable of maintaining strong performance without these signals remains underexplored relative to its practical deployment relevance [9], [11].

This project situates itself within this evolving research landscape by focusing on lightweight, interpretable ML classifiers operating under a strict URL-string-only feature constraint, evaluated against two independently sourced real-world threat intelligence datasets. Rather than pursuing deep learning accuracy, the project emphasizes methods that are computationally efficient, fully offline, and suitable for operational deployment [9]. The cross-dataset generalization framework directly addresses the dataset shift problem by training on one threat source and evaluating on the other in both directions, providing quantitative evidence of how well URL-structural patterns transfer across fundamentally different categories of malicious URLs, and establishing a reproducible foundation for evaluating offline phishing detection under realistic operational constraints.

CHAPTER 2: PROJECT DESCRIPTION

2.1 Project Description

This project designs, implements, and evaluates a modular, eight-script machine learning pipeline for malicious URL detection. The system operates under three strict operational constraints:

- All classification decisions are made solely from features derived from the URL string.
- No network communication occurs at any stage of the pipeline.
- All processing is performed locally within a Python environment on a single host machine.

The system is constructed as a research-oriented prototype intended to establish a reproducible and transparent baseline for evaluating the capabilities of lightweight machine learning classifiers under fully offline conditions. Three classifiers, Logistic Regression, Random Forest, and Decision Tree, are trained and evaluated across two independently sourced real-world threat-intelligence datasets using stratified, five-fold cross-validation, and bidirectional cross-dataset generalization testing. This produces a comprehensive empirical evaluation of detection performance, feature importance, and generalization behavior under these constraints.

The system is implemented following the single-responsibility principle, where each of the eight Python scripts performs exactly one well-defined transformation and has exactly one reason to change. No script combines multiple responsibilities, and no script communicates with another through direct function calls, shared memory, or database connections. All inter-stage data transfer occurs exclusively through structured CSV files stored in the local data/ directory, and all trained model artifacts are serialized using pickle and stored in the outputs/ directory. This file-based integration architecture ensures that every intermediate data state is fully persistent, human-readable, and independently inspectable without requiring the full pipeline to be re-executed. Any individual stage can be modified, re-run, or validated in isolation provided its input files are present, making the system highly maintainable and extensible for future research.

The pipeline progresses through four logical phases:

1. Phase 1: data ingestion handles format-specific parsing or raw source files from the PhishTank and URLhaus, normalizing them into a standardized two-column CSV schema containing a URL string and a binary label.
2. Phase 2: dataset construction merges each malicious source independently with legitimate domains form the Tranco Top Sites list, enforcing a strict 50/50 class balance capped at 10,000 samples per class and producing two independent 20,000-row balanced datasets.

3. Phase 3: data preparation and feature engineering cleans, deduplicates, and validates both datasets before deriving all 24 URL-string features across both datasets in a single execution pass, producing two feature-rich CSVs containing 26 columns each: the original URL string, the binary class label, and 24 engineered numeric features.
4. Phase 4: modeling and evaluation trains and evaluates three lightweight classifiers on each dataset independently using an 80/20 stratified train-test split and five-fold cross-validation, followed by bidirectional cross-dataset generalization testing where each dataset serves as both the training source and the test target.

The system utilizes three publicly available data sources selected for their relevance to malicious URL detection and documented real-world operational relevance and availability:

- PhishTank [4], originally developed by OpenDNS, provides a community-verified repository of confirmed phishing URLs representing credential-harvesting threats.
- URLhaus [5] provides a curated threat intelligence feed of malware distribution URLs associated with botnet infrastructure.
- Tranco Top Sites list [6] provides a research-oriented ranking of legitimate domains designed to reduce bias and manipulation present in other popular lists.

Each dataset is balanced to a strict 50/50 class distribution ensuring that evaluation metrics reflect genuine model discrimination rather than majority-class bias.

In addition to the training and evaluation pipeline, the system includes a standalone inference component that loads the previously trained model artifacts and classifies user-supplied URLs without requiring pipeline re-execution. The inference process applies the same 24-feature extraction methodology used during training, evaluates the URL across all three classifiers, and returns individual model predictions with confidence scores, along with a majority-vote result.

To support usability, a lightweight graphical user interface (GUI) provides an alternative to command-line interaction, enabling non-technical users to access the same interface functionality. Both interaction modes operate fully offline and produce deterministic outputs for a given URL when using the same trained model artifacts.

2.1.1 Pipeline Architecture

The system is implemented as eight independent Python scripts, each with a single clearly defined responsibility. All inter-stage communication occurs through CSV files in the data/ directory and model artifacts in the outputs/ directory. No script shares in-memory state or directly invokes another script. The following table describes each script, its responsibility, its inputs and outputs:

Table 1 *Eight-Script Pipeline Architecture: Scripts, Responsibilities, Inputs, and Outputs*

Script	Responsibility	Input	Output
convert_phishtank.py	Parses the PhishTank XML export and converts it into a standardized two-column CSV (URL, label=1).	Online_valid.xml	phishtank_urls.csv
convert_urlhaus.py	Parses the URLhaus text feed, skips comment lines (#), and produces a standardized two-column CSV (URL, label=1).	Urlhaus_abuse_ch.txt	urlhaus_urls.csv
build_dataset.py	Merges each malicious source with Tranco legitimate URLs, enforces a 50/50 class balance, shuffles the data, and generates two balanced datasets.	Phishtank_urls.csv urlhaus_urls.csv tranco_top_sites.csv	phishtank_tranco_dataset.csv, urlhaus_tranco_dataset.csv
prepare_data.py	Cleans, validates, deduplicates, normalizes labels across both datasets in one run.	Phishtank_tranco_dataset.csv urlhaus_tranco_dataset.csv	cleaned_phishtank_tranco_dataset.csv, cleaned_urlhaus_tranco_dataset.csv
extract_features.py	Engineers all 24 URL string features across both datasets in one run.	Cleaned_phishtank_tranco_dataset.csv cleaned_urlhaus_tranco_dataset.csv	features_phishtank_tranco_dataset.csv, features_urlhaus_tranco_dataset.csv
train_models.py	Trains the 3 classifiers on each dataset; 80/20 stratified split; 5-	Features_phishtank_tranco_dataset.csv	Dashboards, model_results.csv, training_results_date_time.txt,

	fold cross-validation; saves output files.	features_urlhaus_tranco_dataset.csv	model_lr.pkl, model_rf.pkl, model_dt.pkl, scaler.pkl
cross_dataset_eval.py	Trains on PhishTank → test on URLhaus (Direction 1); train on URLhaus → test on PhishTank (Direction 2). Save dashboards and log.	Features_phishtank_tranco_dataset.csv features_urlhaus_tranco_dataset.csv	Cross-eval dashboards, cross_eval_results.csv, cross_eval_results_date_time.txt
predict.py	Loads saved .pkl model files; engineers 24 URL string features from a user-supplied URL; runs through all three classifiers; returns per-model classification and confidence scores with majority-vote summary. No model retraining occurs.	Outputs/models_name.pkl user-supplied URL string	Console output: Per model classification (malicious/legitimate) confidence score, and majority-vote summary

2.1.2 Project Objectives

The project is guided by five research objectives that collectively address the effectiveness, generalization, and iterative improvement of the offline phishing detection system:

1. Evaluate the effectiveness of 24 URL-string-only features for malicious URL detection across two independently sourced real-world threat intelligence datasets, operating without any external data lookups or network dependencies.
2. Assess cross-dataset generalization by training models on one threat intelligence source and evaluating on the other in both directions, quantifying the performance gap and identifying what any asymmetry reveals about the structural differences between phishing and malware delivery URLs.

3. Compare three lightweight machine learning classifiers, Logistic Regression, Random Forest, and Decision Tree, across accuracy, precision, recall, and F1-score, to determine which model provides the best trade-off between classification performance and deployment feasibility.
4. Identify which URL-string features demonstrate the highest predictive stability across both datasets through Random Forest feature-importance analysis, informing iterative feature selection in subsequent experimental rounds.
5. Demonstrate measurable improvements over the Iteration 1 baseline through iterative feature selection and optimization, documenting the impact of each experimental round on within-dataset and cross-dataset performance metrics.

2.2 Analysis

The analysis of this project centers on evaluating the feasibility, strengths, and limitations of a fully offline, privacy-preserving malicious URL detection system that relies solely on URL-string-derived features and lightweight machine learning classifiers. This section synthesizes the motivation behind the chosen models, the rationale for dataset selection, design constraints that shaped the system, and the challenges anticipated in evaluating cross-dataset generalization under strict offline operational requirements.

2.2.1 Rationale for Lightweight Lexical Features

The decision to restrict feature engineering (the extraction of structured attributes from raw URLs) to URL-string-derived lexical and structural features only was driven by both methodological and operational considerations. From a methodological standpoint, URL-string-only features guarantee an identical 24-column feature schema across both datasets, which is a prerequisite for valid cross-dataset comparison. Any feature requiring external resolution, such as DNS queries, or HTTP content retrieval, would introduce inconsistencies across datasets collected at different times and from different sources, undermining the integrity of the generalization evaluation.

From an operational standpoint, URL-string-only features support deployment in environments where external lookups are not feasible or acceptable, including air-gapped networks, privacy-sensitive organizational intranets, and resource-constrained endpoints devices. Features in this category were selected based on four criteria: low computational interpretability for security analysis [3], [11]. While deep learning models can extract richer representations automatically, they require substantial computational resources and sacrifice transparency. The 24-feaure lexical set provides a practical and defensible balance between predictive strength and deployment feasibility in constrained environments.

2.2.2 Model Selection Analysis

Three lightweight supervised classifiers were selected to provide complementary trade-offs between interpretability, model complexity, and predictive capability.

- Logistic Regression serves as a linear baseline, offering transparent coefficient-based decision logic and fast inference suitable for real-time filtering.
- Random Forest captures nonlinear feature interactions through ensemble (i.e., combining predictions from multiple models) averaging across multiple decision trees, often outperforming linear models on tabular URL data while remaining computationally efficient and supporting feature importance analysis.
- Decision Tree provides fully auditable rule-based classification, enabling direct inspection of the decision thresholds applied to URL-string features, though it is more susceptible to overfitting without depth constraints [3], [12].

This selection supports not only performance comparison but also analysis of how model structure and complexity influence cross-dataset generalization. A key question the model selection is designed to answer is whether simpler, more constrained models generalize reliably across threat sources than more complex ones, or whether ensemble methods maintain their performance advantage under dataset shift.

2.2.3 Dataset Selection Strategy

The two primary datasets were constructed from independently sources real-world threat intelligence feeds specifically chosen to create meaningful and realistic generalization tests.

- PhishTank [4] captures community-verified credential-harvesting phishing URLs, representing the traditional phishing threat surface.
- URLhaus [5] captures malware delivery and botnet infrastructure URLs actively used by security operations centers, representing a structurally distinct category of malicious URL.

These two sources differ in their collection methodology, threat type, URL structural patterns, and feature distributions, creating natural dataset shift that is essential for a rigorous cross-dataset generalization evaluation.

Legitimate URLs are sourced from the Tranco Top Sites list [6], a research-grade domain ranking hardened against manipulation through multisource averaging. Tranco was selected over alternatives such as the Alexa Top Sites list because its methodology reduces the risk of attacker-controlled or SEO-inflated domains across both datasets. Each dataset is balanced to a strict 50/50 class distribution of 10,000 malicious and 10,000 legitimate URLs, ensuring that evaluation metrics reflect genuine model discrimination rather than majority-class bias.

2.2.4 Expected Challenges

Several known challenges in phishing URL detection shaped the analytical framework of this project:

Dataset shift represents the primary challenge. Models trained on PhishTank's credential-harvesting URL patterns may not transfer cleanly to URLhaus's malware delivery URL patterns and vice versa, due to structural differences in how these threat categories manifest in URL strings. Quantifying this asymmetry is a central objective of the cross-dataset evaluation.

Feature generalizability is a related concern. Not all 24 URL-string features may contribute equally across both datasets. Features that are highly predictive for one threat source may carry less discriminative weight for the other, motivating the iterative feature selection analysis in Iteration 2.

Overfitting risk is present particularly for the Decision Tree classifier, which can memorize dataset-specific structural patterns rather than learning generalizable threat indicators. This risk is mitigated through depth constraints (max_depth=10) and minimum sample split requirements applied during training.

Class balance enforcement addresses the known problem of inconsistent benign-to-malicious ratios in publicly available phishing datasets. By enforcing a strict 50/50 balance across both datasets, this project ensures that accuracy metrics are not artificially inflated by majority-class dominance and that precision, recall, and F1-score reflect true classification capability.

Context limitation inherent to URL-string-only features remain an acknowledged constraint. Lexical features capture structural patterns but cannot represent semantic meaning, campaign context, or temporal threat evolution. This limitation defines the boundary of what the system is designed to evaluate and is consistent with the project's offline, privacy-preserving design requirements.

2.2.5 Analysis of Project Scope

The scope of this project was deliberately narrowed to lightweight machine learning classifiers operating under URL-string-only feature restrictions for three reasons:

- Operational feasibility: requires that the system remain deployable in resource-constrained environments without external infrastructure dependencies (fast inference, low resource usage).
- Methodological integrity: requires that both datasets share an identical schema, which is only achievable when all features are derived from the URL string alone.

- Research contribution: requires that the project occupies a distinct and defensible position in the literature by rigorously evaluating the offline, privacy-preserving design space that existing work has not systematically addressed.

This scope intentionally excludes deep learning architectures, external feature enrichment, and cloud-based classification services. These exclusions are not limitations of research, but rather deliberate design decisions aligned with the project's core research contribution. The goal is not to achieve the highest possible accuracy under any conditions, but to establish what is reliably achievable within strict offline and privacy-preserving constraints, providing a reproducible and extensible baseline for future research in the design space.

2.2.6 Transition to Advanced Design

The analytical considerations outlined in this chapter directly inform the experimental pipeline described in Chapter 3. The rationale for feature selection shapes the 24-feature engineering approach in Section 3.4. The model selection analysis motivates the training configurations and hyperparameter choices in Section 3.5. The dataset selection strategy defines the cross-dataset generalization procedures in Section 3.7. The expected challenges frame the iterative experimental methodology described in Sections 3.6.1, 3.6.2, and 3.6.3. Together, these analytical foundations ensure that the advanced design is not an arbitrary implementation but a methodologically justified response to the research questions and operational constraints defined in Chapters 1 and 2.

2.2.7 Iterative Experimental Methodology

This project employs an iterative experimental methodology in which each round of evaluation builds directly upon the findings of the previous round. Iteration 1 establishes the baseline by training and evaluating all three classifiers on both datasets using the full 24-feature schema, producing within-dataset and cross-dataset performance metrics that serve as the reference point for all subsequent comparisons. Iteration 2 applies feature selection based on Random Forest importance scores averaged across both datasets, resulting in a reduced feature set of 13 features. Features that do not contribute consistently to classification performance across threat sources are removed, and all three models are re-evaluated using this reduced feature set in both evaluation directions.

Iteration 3 applies grid search hyperparameter optimization to the reduced 13-feature set established in Iteration 2, evaluating whether systematic tuning produces further measurable improvement in cross-dataset generalization. This iterative structure ensures that design decisions are data-driven rather than arbitrary, with each modification justified by quantitative evidence from the preceding round.

The iterative approach is particularly appropriate for cross-dataset generalization research because features that are most predictive within a single dataset are not necessarily the features most stable across datasets. By separating the baseline evaluation from the feature selection round, this project isolates the specific impact of feature reduction on generalization performance, providing clearer and more defensible analysis than a single monolithic experimental run would allow. All iteration results are documented with full metric comparisons in Chapter 4, enabling transparent evaluation of whether each experimental round produced measurable improvement over its predecessor.

CHAPTER 3: SYSTEM DESIGN

3.1 System Design Principles

The design of this system centers on constructing a complete, reproducible, and privacy-preserving machine learning pipeline for malicious URL detection that operates exclusively on features derived from the URL string. The design is governed by four core principles that shaped every architectural and implementation decision throughout the system.

1. Reproducibility: All randomly determined operations use a fixed random seed (random_state=42), file paths are constructed using platform-independent methods, and re-execution on identical input files produces functionally equivalent output.
2. Single-responsibility principle: ensures that each of the eight pipeline scripts performs exactly one well-defined transformation, has exactly one reason to change, and communicates with adjacent stages through structured CSV files rather than direct function calls or shared memory.
3. Fail-fast input validation: ensures that all inputs are validated at the start of each script execution, with missing files, schema mismatches, encoding errors, and invalid label values detected and reported before any computational work begins.
4. Offline execution enforcement: ensures that no script initiates any network connection, DNS query, WHOIS lookup, or external API call at any stage of the pipeline, from raw data ingestion through inference.

The system is designed to support interpretability, efficiency, and modularity. Each pipeline stage is implemented as a standalone module in Python that can be independently tested, modified, and validated without affecting other stages, following established architectural principles of separation of concerns [13] and single responsibility [14]. This supports future extensibility such as integrating richer feature engineering, applying domain adaptation techniques, or incorporating additional datasets, without requiring architectural redesign. A script-based pipeline architecture was chosen over notebook-based approaches because notebook execution order dependencies have been shown to significantly compromise reproducibility in machine learning research [15]. All pipeline scripts, trained model artifacts, evaluation results, and visual dashboards are provided in the appendices as reproducible experimental artifacts.

3.2 System Architecture

The system follows a layered pipeline architecture aligned with the Extract, Transform, Load (ETL) pattern as adapted from machine learning workflows [16]. The extract phase corresponds to the two converter scripts that ingest raw source data from PhishTank and URLhaus. The transform phase spans dataset construction, data cleaning, and

feature engineering. The load phase corresponds to model training, cross-dataset evaluation, and output generation. Each layer consumes only the output of the preceding layer, and no layer re-reads raw inputs or bypasses an intermediate stage.

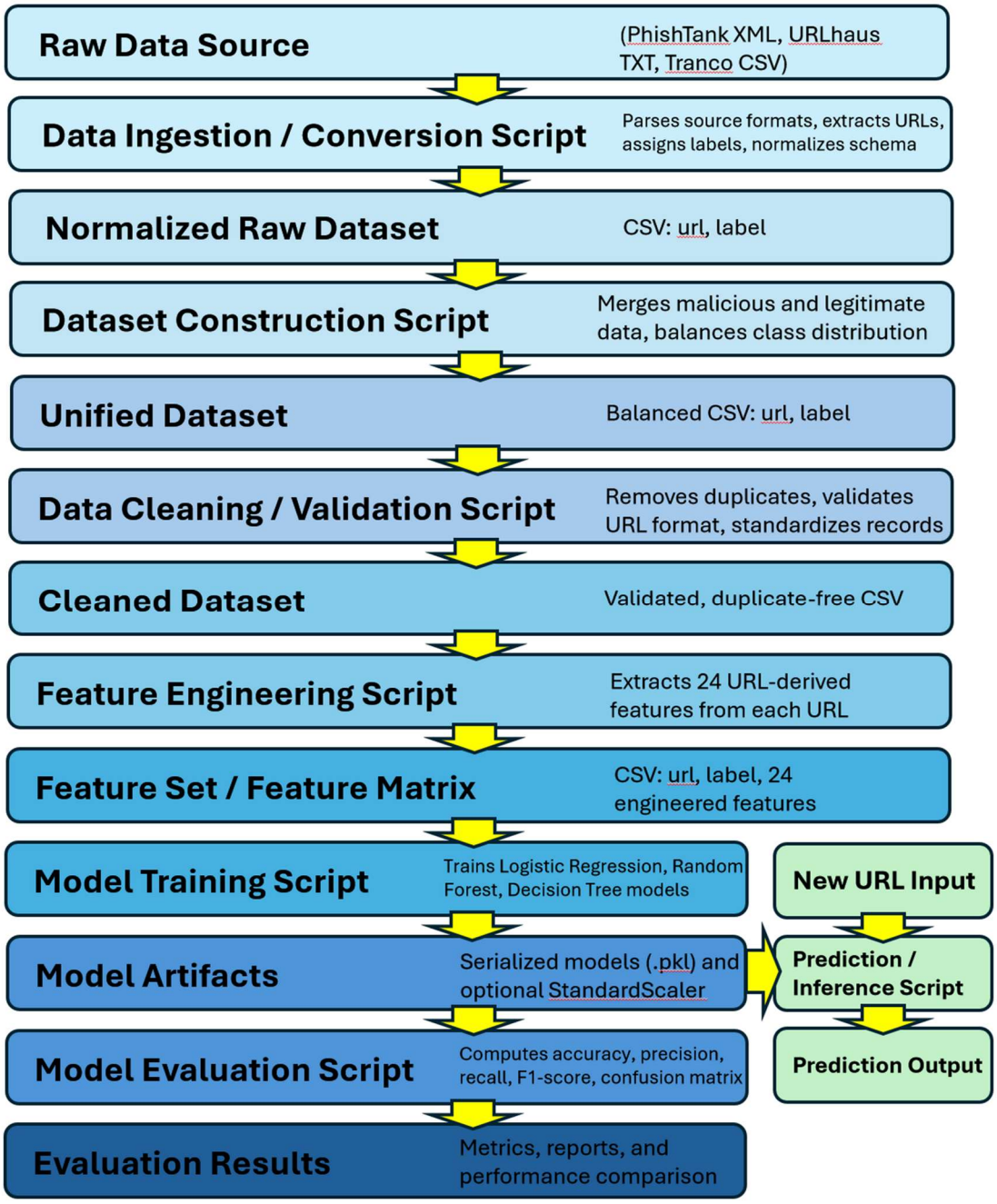


Figure 1 End-to-End Pipeline Diagram- Malicious URL Detection System

Overview of the complete eight-stage processing pipeline from the raw source data ingestion through model evaluation and inference, illustrating the sequential data flow through each transformation stage and the parallel inference path accessible after training is complete.

The pipeline is organized into four logical processing phases followed by a standalone inference path:

Phase 1- Data Ingestion convert_phishtank.py parses the PhishTank XML export file, extracts all verified malicious URL entries, and produces a standardized two-column CSV containing the URL string and a binary malicious label. convert_urlhaus.py parses the URLhaus plain-text feed, skips all comment lines prefixed with the hash character, and produces an equivalent two-column CSV. These two scripts isolate all format-specific parsing logic, ensuring that upstream format changes affect only a single script and do not propagate through the pipeline.

Phase 2- Dataset Construction build_dataset.py independently merges each malicious source with legitimate domains from the Tranco Top Sites list, enforcing a strict 50/50 class balance capped at 10,000 samples per class. The combined 20,000 row datasets are shuffled using a fixed random seed before saving. Both datasets are constructed in a single script execution, producing two parallel datasets that flow identically through all downstream stages.

Phase 3- Data Preparation and Feature Engineering prepare_data.py cleans, deduplicates, validates, and normalizes labels across both datasets in a single execution pass. extract_features.py derives all 24 URL-string features across both cleaned datasets in a single execution pass, producing two feature datasets containing 26 columns each: the original URL string, the binary class label, and 24 engineered numeric features. The identical feature schema applied to both datasets is a methodological requirement for valid cross-dataset comparison, ensuring that no feature is derived from information unavailable at inference time and that no data leakage occurs between the training and evaluation stages.

Phase 4- Modeling and Evaluation train_models.py trains three classifiers, Logistic Regression, Random Forest, and Decision Tree, on each feature dataset independently using an 80/20 stratified train-test split and five-fold cross-validation on the training set. For Logistic Regression, a StandardScaler is fitted on training data and applied to the test set using transform only, preventing data leakage. Random Forest and Decision Tree operate on unscaled features. Each trained model is serialized using pickle and saved to the outputs/ directory for use by the inference component. cross_dataset_eval.py trains each model on the full feature dataset from one source and evaluates on the full feature dataset from the other source in both directions,

applying the same preprocessing isolation to prevent leakage across the evaluation boundary.

Inference Path predict.py operates independently of the training pipeline. It loads the serialized model artifacts from the outputs/ directory, applies the identical 24-feature engineering logic used during training to a single user-supplied URL string, runs the URL through all three classifiers, and returns a per-model classification with confidence score and a majority-vote summary. No model retraining occurs during inference, and no network communication is initiated at any point. The inference path can be accessed through either the command-line interface or a graphical user interface implemented in phishing_gui.py, both of which utilize the same underlying feature extraction and classification logic.

The complete end-to-end pipeline flow is illustrated in Figure 1. Supporting UML diagrams including use case, activity, sequence, class state, and entity-relationship models providing detailed view of component relationships, data flow, and logical deployment structure are provided in Appendix III.

3.3 Data Preparation and Preprocessing

Data preparation and preprocessing are implemented in prepare_data.py and operates across both datasets in a single execution path. The preprocessing stage serves a single clearly defined purpose: to ensure that both datasets meet the structural, quality, and consistency requirements necessary for valid feature extraction and model training downstream. All transformations produce new output files rather than modifying raw source data, preserving the original inputs for reproducibility and reuse.

3.3.1 Encoding and File Handling

Both datasets are ingested using a two-stage encoding strategy. The system first attempts to read each file using UTF-8 encoding, which is the standard encoding for modern CSV files. If a UnicodeDecodeError is encountered, the system automatically retries using Latin-1 encoding with malformed lines skipped and logs a warning message indicating that the fallback encoding was used. This approach ensures that files containing non-standard characters or legacy encodings do not cause pipeline failures while maintaining transparency about any encoding inconsistencies in the source data.

3.3.2 Schema Validation

Before any cleaning operations are performed, prepare_data.py validates that each input dataset contains the required columns. The system searches for the URL column using a predefined candidate list including url, urls, link, domain, and address, and standardizes the column name to URL regardless of the original naming convention. The label column is identified similarly from a candidate list and validated for presence

before processing begins. If either required column is absent, the script exits immediately with a descriptive error message identifying the missing column and instructing the user to inspect the source file. This fail-fast validation prevents silent data corruption from propagating through downstream pipeline stages.

3.3.3 Duplicate Removal

Duplicate URL entries are identified and removed from both datasets. Retaining duplicate URLs creates a risk of data leakage where identical examples appear in both the training and test splits of a stratified split, artificially inflating evaluation metrics by allowing the model to memorize specific examples rather than learning generalizable patterns. The number of duplicates removed from each dataset is logged to provide transparency about the cleaning process.

3.3.4 Invalid Entry Removal

Rows containing null, empty, or whitespace-only URL values are removed, as these cannot be parsed by the feature extraction stage and would produce invalid feature vectors. Rows with missing, null, or unrecognizable label values are similarly removed, as incomplete labels cannot be used in supervised learning. The count of removed rows is logged for each category to maintain a clear audit trail of data quality decisions.

3.3.5 Label Normalization

Label values are normalized to a constraint binary integer format across both datasets. The system applies a predefined mapping that converts string label variants including phishing, malicious, bad, and spam to 1 for malicious, and legitimate, benign, good, and safe to 0 for legitimate. If the label column is already numeric, it is coerced to integer type directly. This normalization ensures consistent label interpretation across both datasets and all three classifiers regardless of the original labeling convention used by the source.

3.3.6 Cleaned Dataset Output

After all preprocessing operations are complete, the cleaned datasets are saved as:

- cleaned_phishtank_tranco_dataset.csv
- cleaned_urlhaus_tranco_dataset.csv

These standardized files serve as the sole input to the feature engineering stage described in Section 3.4. Each cleaned dataset retains only the URL and label columns, with all rows verified to contain a valid non-empty URL string and a binary label of 0 or 1.

3.4 Feature Engineering Approach

Feature engineering is implemented in extract_features.py and operates across both cleaned datasets in a single execution pass. The objective is to derive a fixed set of 24

numeric features from each URL string that are computationally inexpensive, consistently computable from the raw URL string alone without any external lookups, and sufficiently discriminative to support classification across two structurally distinct threat intelligence sources. The identical 24-feature schema is applied to both datasets, which is a methodological requirement for valid cross-dataset comparison. Because all features are derivable from the URL string itself, the schema remains consistent regardless of when or where a URL was collected, eliminating the risk of data leakage or lookup inconsistencies that would undermine the generalization evaluation.

Features are organized into four groups based on the type of URL characteristics they capture.

3.4.1 Group A- Length Measurements

Four features capture the character length of distinct URL components:

- url_length measures the total length of the full URL string.
- hostname_length measures the length of the hostname component.
- path_length measures the length of the path component.
- query_length measures the length of the query string.

These features are computed using Python's urllib.parse module, specifically the urlparse function, which decomposes the URL into its constituent components. URLs with no explicit path or query string receive a value of zero for the corresponding feature. Length-based features are widely used in malicious URL detection because attackers frequently embed additional directory levels, random strings, or misleading tokens to obscure the true destination, producing URLs that are substantially longer than legitimate counterparts [3].

3.4.2 Group B- Character Count Features

Ten features count the occurrence of specific characteristics within the full URL string:

- num_dots (.)
- num_hyphens (-)
- num_underscores (_)
- num_slashes (/)
- num_qmarks (?)
- num_equals (=)
- num_ampersands (&)
- num_at (@)
- num_percent (%)
- num_digits (0,1,...9)

These character frequency metrics capture structural irregularities commonly associated with malicious URLs. Excessive dots may indicate deep subdomain structures used to mimic legitimate domains, excessive hyphens may indicate deceptive domain construction, high digit counts may signal algorithmically generated domains or IP-based addresses, and percent signs indicate URL-encoded characters sometimes used to obfuscate malicious path components [3], [11].

3.4.3 Group C- Structural and Boolean Features

Six features capture structural properties and binary characteristics of the URL:

- has_https is a binary indicator of whether the URL scheme is HTTPS
- has_ip is a binary indicator of whether the hostname is a raw Ipv4 address rather than a domain name
- has_port is a binary indicator of whether the URL explicitly specifies a non-standard port number
- has_www is a binary indicator of whether the hostname begins with the www prefix
- has_at_in_url is a binary indicator of whether the at symbol appears anywhere in the URL string
- subdomain_depth is an integer count of subdomain levels present above the registered domain computed by splitting the hostname on period characters and subtracting two for the registered domain and top-level domain

The has_ip feature is particularly relevant for URLhaus data, where malware delivery infrastructure frequently uses raw IP addresses, which can help bypass domain-based detection and attribution mechanisms [12]. The has_at_in_url feature captures a known phishing technique where the at symbol causes browsers to treat everything preceding it as credentials, masking the true destination domain [3], [11]. The has_https feature, while not a strong standalone discriminator given the widespread adoption of HTTPS by both legitimate and malicious sites, contributes incremental signs when combined with other structural features.

3.4.4 Group D- Entropy and Ratio Features

Four features capture statistical properties of the URL string:

- url_entropy computes the Shannon entropy of the full URL string
- hostname_entropy computes the Shannon entropy of the hostname components only
- digit_ratio computes the proportion of digit characters relative to the total URL length
- special_char_ratio computes the proportion of selected special characters relative to the total URL length

Shannon entropy is computed using the formula:

$$H(X) = -\sum_{x} p(x) log_2 p(x)$$

where p(x) represents the relative frequency of each character in the string [17]. Higher entropy values indicate greater character randomness, which is associated with algorithmically generated or obfuscated malicious URLs. Isolating hostname entropy from full URL entropy allows the feature set to distinguish between obfuscation in the domain component versus the path or query string, which may reflect different attack strategies. The digit_ratio and special_char_ratio features complement entropy by capturing specific structural obfuscation patterns that aggregate entropy measures alone may not isolate [3]. For URLs with a length of zero, ratio features are assigned a value of zero using pandas fillna(0) to avoid division errors.

3.4.5 Feature Dataset Output

After feature extraction is complete, both feature datasets are saved to the data/ directory as:

- features_phishtank_tranco_dataset.csv
- features_urlhaus_tranco_dataset.csv

Each file contains 26 columns: the original URL string, the binary label, and the 24 engineered numeric features. These feature datasets serve as the sole input to the modeling and evaluation stage described in Section 3.5 through 3.7. The identical column schema across both files is validated at the start of cross_dataset_eval.py to confirm schema consistency before any cross-dataset evaluation begins.

3.4.6 Design Rationale and Limitations

The 24-feature schema was designed to balance predictive strength against the strict operational constraints of the system. All features are computationally inexpensive requiring only string parsing and arithmetic operations with no external dependencies. All features are interpretable, allowing security analysts to understand and audit the basis for any classification decision. All features are consistently computable from any URL string regardless of its source, collection time, or network reachability, which is essential for a system designed to operate in fully offline environments [9], [11].

The primary limitation of this feature set is that lexical and structural features cannot capture semantic meaning, campaign context, or temporal threat evolution. A URL constructed too closely to mimic a legitimate domain structure while remaining malicious may not be reliably detected by structural features alone. Additionally, as attackers adapt their URL construction techniques in response to detection systems, the discriminative power of specific features may degrade over time. These limitations

define the boundary of what the current system is designed to evaluate and are consistent with the project's offline, privacy-preserving design requirements. Addressing these limitations through richer feature representations or domain adaptation techniques represents a natural direction for future research.

3.5 Model Design and Rationale

The model selection strategy focused on identifying lightweight, interpretable, and computationally efficient classifiers suitable for URL-based malicious detection. Prior research demonstrates that traditional machine learning models, particularly Logistic Regression, Random Forests, and Decision Trees, perform competitively when applied to lexical URL features and are commonly used as baselines for phishing detection tasks [3], [12]. These three models were therefore selected to represent a spectrum of complexity, interpretability, and nonlinear modeling capability. This enables comparative analysis of how model structure influences both within-dataset performance and cross-dataset generalization under the URL-string-only feature constraint.

3.5.1 Logistic Regression

Logistic Regression (LR) was included as a linear baseline because it provides fast inference and transparent decision logic, both of which are desirable properties in cyber-defense classification tasks. Its strengths include low computational overhead and inherently probabilistic outputs, which support risk-based filtering rather than rigid binary decisions. Analysts can directly audit coefficient weights to understand how individual URL-string features influence phishing likelihood. [3]

In this system, model behavior is controlled through hyperparameters, which are user-defined settings that govern the training process, rather than parameters learned directly from the data. Logistic Regression is trained with a maximum of 1,000 iterations to ensure convergence, a balanced class weight setting to handle any residual class distribution effects, and feature scaling applied exclusively to the training data using StandardScaler fitted on training examples only, preventing data leakage into the test or cross-dataset evaluation sets.

LR has important limitations that affect detection reliability when URL feature relationships are nonlinear or when data distributions shift [10]:

- It cannot inherently capture nonlinear feature interactions without manual feature engineering or expansion [11].
- Coefficient weights may reflect dataset-specific artifacts rather than generalizable or transferable threat indicators [11].
- It exhibits high sensitivity to the training data distribution, which can destabilize inference when encountering previously unseen or structurally unfamiliar URL patterns [3], [10].

- Performance evaluated within a single dataset may not generalize to out-of-distribution scenarios, leading to unreliable decision boundaries [10].
- It is less effective when URLs are heavily obfuscated, randomized, or exhibit adversarial structural patterns [18].

Together, these limitations demonstrate that while LR is valuable for establishing baseline separability, its performance optimism within a single dataset can be misleading. Its sensitivity to the training data distribution further limits generalization, as decision boundaries may not remain stable when applied to previously unseen or structurally different URL patterns [3], [10]. Without additional nonlinear feature representation or domain-shift mitigation, the model may fail to maintain decision reliability at deployment time [9]. Despite these limitations, LR is frequently included in phishing detection because it offers transparency and fast inference, making it a suitable baseline against which more complex models can be compared.

3.5.2 Random Forest

Random Forest (RF) was selected as a nonlinear ensemble baseline because it supports strong pattern recognition without sacrificing computational efficiency or interpretability. Unlike a single Decision Tree, RF reduces boundary instability by averaging predictions across many randomized trees constructed through bootstrap sampling. This makes it a relevant candidate for cyber-defense URL classification tasks, where phishing signals often emerge from combinations of lexical patterns rather than any single indicator [3], [10].

Its strengths include efficient parallelizable training and inference, making it suitable for latency-sensitive threat filtering environments [9], as well as native support for nonlinear feature interactions without requiring external tokenization or embedding layers [3]. The model is also less sensitive to noise due to feature randomness and bootstrap aggregation, and it provides interpretable feature importance rankings that allow analysts to audit which URL-string characteristics most strongly influence classification decisions [3]. Outputs remain deterministic when a fixed random state is applied, ensuring reproducibility across experimental runs [19].

In this system, Random Forest is configured with 200 estimators (i.e., individual decision trees within the ensemble), a balanced class weight setting, and parallel processing enabled across all available CPU cores to ensure efficient and consistent model training. These settings allow the model to be trained repeatedly on the 20,000-row datasets used in this study within reasonable time and resource limits, without affecting model behavior or evaluation outcomes.

RF has important limitations that must be considered when evaluating real-world deployment reliability:

- Inference boundaries may still reflect corpus-specific statistical patterns, particularly when benign domain distributions differ across sources.
- High performance within a single dataset may result from distribution matching rather than true generalizable threat detection capability.
- Feature importance scores provide structural insight but do not capture semantic or campaign-level threat context.
- Model behavior remains dependent on the training data distribution, requiring evaluation under dataset shift to assess inference stability.

Together, these limitations demonstrate that while RF improves upon linear models in capturing nonlinear feature relationships, its performance may still be overly optimistic when evaluated within a single dataset. Without explicit validation under distribution shift, the model may not maintain consistent detection reliability across diverse URL corpora [10]. Despite these limitations, RF is widely used in phishing detection research because it offers a strong balance between predictive performance, interpretability, and computational efficiency, making it an effective baseline for studying cross-dataset generalization without introducing the complexity or latency of deep learning approaches.

3.5.3 Decision Tree

Decision Trees (DT) were included in this study as a fully transparent rule-based baseline to examine direct feature separability using the engineered URL-string features. Unlike Logistic Regression or Random Forest, a DT produces a single hierarchical set of conditions that analysts can visually trace from root to leaf, making it particularly valuable for understanding precisely why a classification decision was reached, a core requirement in explainable cybersecurity research [9], [20].

Its strengths include direct rule auditability, enabling clear visibility into feature thresholds that influence phishing classification, as well as fast and predictable inference suitable for real-time detection systems where low latency is critical [9]. DT also does not require feature scaling, reducing preprocessing complexity while preserving lexical pattern representation. Additionally, when depth-limited, the model remains lightweight and interpretable, supporting reproducible baseline analysis in constrained environments [20].

In this system, the Decision Tree is trained on a maximum depth of 10 to prevent overfitting, a minimum samples split of 20 to ensure that nodes are not divided on insufficient data, and a balanced class weight setting consistent with the other classifiers. This depth constraint is a deliberate design decision that mitigates a known limitation of unconstrained Decision Trees: the tendency to memorize dataset-specific structural patterns rather than learning generalizable threat indicators [11].

DT has important limitations that affect generalization and deployment reliability:

- Without depth constraints, inference rules may become highly distribution-specific, memorizing training data structures rather than capturing dataset-agnostic phishing signals [11].
- Even when constrained, rule structures remain sensitive to the training corpus, leading to degraded performance when evaluated on lexically unfamiliar or structurally divergent URL datasets.
- High performance within a single dataset may reflect rule optimization to that dataset rather than durable detection capability under domain shift.
- The model does not capture semantic or campaign-level threat context beyond the engineered URL-string features.
- Rule stability may degrade when applied to datasets with different structural characteristics, requiring further validation under cross-dataset conditions.

Together, these limitations demonstrate that while DT provides maximum interpretability and clear rule-based insight into feature separability, its performance may be overly optimistic when evaluated within a single dataset. Even with depth constraints, the model remains sensitive to training data structure, and its decision rules may not generalize reliably across distinct URL corpora. Despite these limitations, DT serves an important role in this study by exposing how phishing detection rules emerge from lexical features and by highlighting the extent to which such rules degrade under distribution shift, making it a valuable baseline for comparison with more generalizable ensemble methods.

3.6 Experimental Setup

The experimental design was constructed to ensure methodological rigor, reproducibility, and comparability across both datasets and both evaluation directions. All implementation was conducted in Python using the scikit-learn machine learning library, which is widely recognized for its reliability, standardization, and reproducibility in academic and industrial research contexts [19]. All randomly determined operations throughout the experimental setup use a fixed random seed of 42, including dataset shuffling, train-test split assignments, cross-validation fold generation, and all three classifier initializations, ensuring that pipeline re-execution on identical input files produces functionally equivalent results.

3.6.1 Baseline Experimental Setup (Iteration 1)

Iteration 1 establishes the baseline performance of all three classifiers using the full 24-feature schema on both datasets. The experimental procedure is defined as follows:

Dataset Partitioning

The dataset partitioning strategy establishes a controlled and representative split between training and testing data, ensuring reliable performance evaluation. Stratified sampling is used to preserve class balance across both sets.

- Each dataset is split using an 80/20 stratified train-test split.
- This results in:
    - 16,000 training samples
    - 4,000 test samples
- Stratification preserves the 50/50 class distribution (malicious vs. legitimate) across both sets.
- This prevents imbalanced splits that could artificially inflate or deflate evaluation metrics, ensuring representative class proportions in both training and testing phases.

Cross-Validation Procedure

Cross-validation is applied to provide a more reliable estimate of model generalization performance beyond a single train-test split. A stratified approach ensures consistent class representation across folds (subsets of the data).

- Five-fold stratified cross-validation is performed on the training set for each model and dataset.
- The 16,000-row training set is divided into five equal folds:
    - Train on 4 folds
    - Evaluate on 1 fold
    - Rotate across all 5 folds
- Performance is reported as:
    - Mean F1-score
    - Standard deviation across folds
- This process is repeated independently for each model on each dataset, resulting in:
    - 30 total cross-validation evaluations (3 models x 2 datasets x 5 folds), summarized as 6 mean F1-scores, each with a corresponding standard deviation.
- This provides a more complete estimate of generalization performance than a single train-test split alone.

Feature Scaling

Feature scaling is selectively applied based on model requirements to prevent data leakage while maintaining appropriate preprocessing for each algorithm.

- Feature scaling is applied only to Logistic Regression using StandardScaler:

    - Fitted exclusively on training data
    - Applied to test data using transform, which applies the same scaling parameters learned from the training data without recalculating them
- This prevents data leakage, ensuring no test-set information influences scaling parameters.
- Random Forest and Decision Tree models operate on unscaled features, as tree-based methods:
    - Use threshold-based splits
    - Are invariant to feature magnitude
    - Do not require normalization

Model Hyperparameters

The following hyperparameters are used to ensure consistent and reproducible training across all models in Iteration 1.

- Logistic Regression
    - max_iter = 1000
    - random_state = 42
    - class_weight = ‘balanced’
- Random Forest
    - n_estimators = 200
    - random_state = 42
    - n_jobs = -1
    - class_weight = ‘balanced’
- Decision Tree
    - max_depth = 10
    - min_samples_split = 20
    - random_state = 42
    - class_weight = ‘balanced’
- the class_weight = ‘balanced’ setting is applied consistently across all models to:
    - Adjust class weights inversely proportional to class frequencies
    - Mitigate residual class distribution effects
    - Ensure stable training behavior despite enforced dataset balance

This hyperparameter configuration defines the baseline model setup and is held constant in subsequent iterations to isolate the impact of feature selection before being varied in Iteration 3.

Summary of Iteration 1

Iteration 1 establishes a controlled baseline for evaluating all three models using a consistent feature set, balanced datasets, and standardized training procedures.

Stratified splitting and cross-validation ensure that performance estimates are stable and representative, while careful preprocessing prevents data leakage. This baseline configuration provides a reliable point of comparison for assessing model behavior and serves as the foundation for subsequent cross-dataset generalization analysis.

3.6.2 Feature Selection Methodology (Iteration 2)

Iteration 2 applies a data-driven feature selection procedure to identify and remove URL-string features that do not contribute consistently to classification performance across both threat intelligence sources. This step is motivated by a key observation from Iteration 1: Random Forest feature importance distribution differs between the PhishTank and URLhaus datasets, indicating that some features are highly predictive for one source but less informative for the other. Features that are source-specific rather than broadly predictive may reduce cross-dataset generalization by introducing noise into the model's decision boundary when applied to unfamiliar URL distributions.

Feature Importance Analysis

Feature importance analysis is used to quantify how much each feature contributes to model prediction across both datasets.

- A Random Forest classifier is trained independently on each dataset using the full 24-feature schema.
- Feature importance scores are extracted from each model.
- These scores reflect the relative contribution of each feature to classification decisions within that dataset.

Cross-Dataset Importance Aggregation

To ensure that feature selection reflects generalizable patterns rather than dataset-specific behavior, importance scores are combined across both datasets.

- Feature importance scores from the two datasets are averaged to produce a mean importance score for each feature.
- This averaging ensures that features are evaluated based on their contribution across both threat sources, rather than being biased toward a single dataset.

Feature Selection Thresholding

A threshold-based filtering approach is applied to identify features that contribute minimally to classification performance.

- A threshold of 0.01 (1%) is used as the cutoff for feature inclusion.
- Features with a mean importance score below this threshold are removed from the feature set.

- This results in a reduced feature subset that retains only features with consistent predictive contribution across both datasets.
- Applying this threshold to the averaged feature importance scores retained 13 features from the original 24-feature set.

Model Retraining and Evaluation

All models are retrained and evaluated using the reduced feature set under the same controlled conditions as Iteration 1 to ensure a fair comparison.

The same experimental setup from Iteration 1 is preserved, including the baseline hyperparameter configuration, which is held constant in this iteration to isolate the impact of feature selection. This configuration is subsequently varied in Iteration 3 during hyperparameter optimization.

- Logistic Regression, Random Forest, and Decision Tree are retrained using the reduced feature set.
- The same experimental setup from Iteration 1 is preserved including:
    - 80/20 stratified train-test split
    - Five-fold stratified cross-validation
    - Features scaling applied only to Logistic Regression
    - Fixed hyperparameters and random state
- Both within-dataset and cross-dataset evaluations are performed.

Performance Comparison

The impact of feature selection is assessed by comparing results from Iteration 2 against the Iteration 1 baseline.

- Models are evaluated using the same performance metrics:
    - Accuracy
    - Precision
    - Recall
    - F1-score
- Performance changes are analyzed to determine whether feature reduction improves:
    - Within-dataset performance
    - Feature predictive stability across both threat sources
    - Cross-dataset generalization

Summary of Iteration 2

Iteration 2 refines the feature space by removing features that do not contribute consistently across both datasets, reducing the influence of dataset-specific noise on model training. By applying a unified importance threshold and retraining all models

under identical conditions, this iteration isolates the impact of feature selection on model performance. This approach enables a direct assessment of whether a reduced, more generalizable feature set improves cross-dataset reliability while maintaining or enhancing classification effectiveness.

3.6.3 Hyperparameter Tuning Methodology (Iteration 3)

Iteration 3 applies systematic grid search hyperparameter optimization to identify the best-performing configuration for each model on the reduced 13-feature set established by Iteration 2. This step is motivated by the observation that the baseline hyperparameter configurations used in Iteration 1 and 2 were selected for comparability rather than performance optimization. While feature selection in Iteration 2 improved cross-dataset generalization, particularly for Logistic Regression in the URLhaus to PhishTank direction, the question remained whether targeted hyperparameter tuning could produce further measurable improvement without sacrificing within-dataset performance stability. By holding the reduced feature set constant and varying only hyperparameter configurations, Iteration 3 isolates the contribution of model optimization as a distinct experimental variable.

Grid Search Procedure

Grid Search cross-validation is used to exhaustively evaluate all specified hyperparameter combinations and identify the configuration that produces the highest cross-validation F1-score for each model.

- GridSearchCV from scikit-learn is applied to each model independently on each dataset
- 5-fold stratified cross-validation is used as the evaluation strategy within each grid search, consistent with the cross-validation methodology applied in Iteration 1 and 2
- F1-score is used as the optimization metric, consistent with the project's emphasis on balancing precision and recall for phishing detection
- refit=True is specified so that the best estimator is automatically retrained on the full training set after grid search completes, making it ready for direct evaluation
- n_jobs=-1 is specified to utilize all available CPU cores for parallel grid search, reducing computation time across the large number of parameter combinations evaluated

Parameter Grids Searched and Rationale

The following parameter grids are defined for each model based on the parameters most likely to influence cross-dataset generalization performance:

- **Logistic Regression: Regularization Strength ©**

    - Values Tested: 0.01, 0.1, 1.0, 10.0, 100.0
    - C controls the inverse of regularization strength, governing the trade-off between fitting the training data closely and learning patterns that generalize to unseen data. Smaller C values apply stronger regularization, constraining the model to simpler decision boundaries that may transfer more reliably across datasets with different URL distributions. Larger C values reduce this penalty, allowing the model to fit training-specific patterns more precisely. C was selected as the primary tuning parameter for Logistic Regression because the URLhaus to PhishTank cross-dataset direction showed the largest recall gap in Iteration 1 and 2, and regularization strength directly influences how tightly a linear classifier fits its training distribution. A model trained with insufficient regularization on URLhaus data may learn patterns too specific to malware delivery URL structures to transfer effectively to credential-harvesting patterns present in PhishTank test data.
- **Random Forest: Number of Estimators and Maximum Depth**
    - n_estimators values tested: 100, 200, 300
    - max_depth values tested: None (unlimited), 10, 20, 30
    - Number of estimators controls how many individual decision trees are constructed and averaged to produce each prediction. More trees generally improve prediction stability and reduce variance by averaging out individual tree errors but increase training time proportionally. Maximum depth controls how many levels of splits each tree is permitted to make before being forced to classify a sample. Shallower trees capture only the most broadly applicable patterns and are less likely to memorize dataset-specific URL structures, while deeper trees can capture more complex feature interactions but risk overfitting to training-specific patterns. Both parameters were selected for tuning because their interaction determines whether the Random Forest learns generalizable URL structural patterns or dataset-specific decision boundaries that do not transfer well across threat categories.
- **Decision Tree: Maximum Depth and Minimum Samples Split**
    - max_depth values tested: 5, 10, 15, 20
    - min_samples_split values tested: 10, 20, 30, 50
    - Maximum depth controls the complexity of the decision rules the tree is permitted to learn. Minimum samples split specifies the minimum number of training examples a node must contain before it is allowed to split into further branches. Higher values prevent the tree from constructing highly specific rules based on small subsets of training examples, reducing the risk of overfitting to dataset-specific URL patterns that may not appear in

the test distribution. Both parameters were selected because the baseline depth=10 and min_samples_split=20 configuration was potentially limiting the model's ability to capture the full complexity of PhishTank URL decision boundaries, and because shallower configurations might improve generalization for the structurally simpler URLhaus patterns.

Preprocessing Isolation

Feature scaling and preprocessing isolation are maintained under the same controlled conditions as Iteration 1 and 2 to ensure a fair comparison.

- StandardScaler is fitted exclusively on the training data for each grid search run and applied to test data using transform only, without refitting
- For cross-dataset evaluation, the scaler is fitted on the training dataset and applied to the test dataset, ensuring no information from the test distribution influences scaling parameters
- Grid search is performed on the training dataset before evaluation on the test dataset, ensuring that hyperparameter selection does not incorporate any information from the test distribution
- Random Forest and Decision Tree models operate on unscaled features within the grid search consistent with Iterations 1 and 2

Model Retraining and Evaluation

After grid search identifies the best configuration for each model, all models are retrained using their optimal parameters and evaluated under the same conditions as previous iterations to ensure direct comparability.

- Logistic Regression, Random Forest, and Decision Tree are retrained using their best hyperparameter configurations identified by grid search
- The same experimental setup from Iteration 1 and 2 is preserved including:
    - 80/20 stratified train-test split
    - Fixed random_state=42 for all operations
    - Features scaling applied only to Logistic Regression
    - Reduced 13-feature set from Iteration 2
- Both within-dataset and cross-dataset evaluations are performed in both directions

Performance Comparison

The impact of hyperparameter tuning is assessed by comparing Iteration 3 results against both the Iteration 1 baseline and the Iteration 2 feature selection results, producing a three-way comparison across the full experimental record.

- Models are evaluated using the same performance metrics:

    - Accuracy
    - Precision
    - Recall
    - F1-score
- Performance changes are analyzed to determine whether the hyperparameter optimization improves:
    - Cross-dataset generalization, particularly in the URLhaus to PhishTank direction
    - Within-dataset performance stability
    - Overall model strength across both threat intelligence sources
- Best hyperparameter configurations are recorded for each model and dataset context and saved to best_params.csv in the outputs/ directory for full transparency and reproducibility

Summary of Iteration 3

Iteration 3 completes the three-iteration experimental methodology by applying systematic grid search hyperparameter optimization to the reduced 13-feature set established in Iteration 2. By holding the feature set constant and varying only hyperparameter configurations, this iteration isolates the contribution of model tuning as a distinct experimental variable and enables a clean three-way comparison across all iterations. The grid search procedure uses 5-fold stratified cross-validation scored by F1-score, consistent with the evaluation methodology applied throughout the project, and preserves all preprocessing isolation controls to ensure valid and comparable results. The findings from this iteration directly address research question five regarding measurable improvements through iterative optimization and contribute to the model selection decision documented in Section 4.6.

3.6.4 Model Training

This section describes the training configuration, hyperparameter strategy, training procedure, and reproducibility controls that are applied consistently across all three experimental iterations. The training methodology is designed around the principle that each iteration changes exactly one experimental variable while holding all other conditions constant, enabling clean attribution of performance differences to specific design decisions.

Training Configuration

All models are trained using a consistent set of configurations to maintain comparability across experiments and iterations:

- Models are implemented using the scikit-learn machine learning library, which provides standardized and reproducible implementations of all three classifiers

- Identical feature sets are used within each iteration:
    - Iteration 1: full 24-feature set
    - Iteration 2: reduced 13-feature set
    - Iteration 3: reduced 13-feature set with tuned hyperparameters
- Preprocessing steps are applied consistently across all datasets and evaluation directions:
    - Feature scaling via StandardScaler is applied to Logistic Regression training data only
    - The scaler is fitted on training data and applied via transform to test data, preventing data leakage
    - Random Forest and Decision Tree operate on unscaled features as tree-based methods use threshold-based splits that are invariant to feature magnitude
- All models are trained independently on each dataset to support both within-dataset and cross-dataset evaluation
- The class_weight=balanced setting is applied consistently across all three models and all three iterations to adjust class weights inversely proportional to class frequencies and ensure stable training behavior

Hyperparameter Strategy

Hyperparameter configurations vary across iterations to reflect the progressive optimization methodology of this project:

- **Iteration 1- Baseline configurations**:
    - Logistic Regression: max_iter=1000, random_state=42, class_weight=balanced
    - Random Forest: n_estimators=200, random_state=42, n_jobs=-1, class_weight=balanced
    - Decision Tree: max_depth=10, min_samples_split=20, random_state=42, class_weight=balanced
    - Baseline configurations are used without tuning to ensure that initial performance differences are attributable to model characteristics, feature representation, and dataset properties rather than optimization artifacts
- **Iteration 2- Baseline configurations retained**:
    - The same baseline hyperparameter configurations from Iteration 1 are retained and unchanged
    - Feature selection is the sole experimental variable, ensuring that any performance change between Iteration 1 and Iteration 2 is attributable specifically to removal of low-importance features
- **Iteration 3- Grid search hyperparameter tuning**:

    - GridSearchCV with 5-fold stratified cross-validation scored by F1-score is used to identify the best-performing hyperparameter configuration for each model on the reduced feature set.
    - Hyperparameter tuning is treated as the sole experimental variable, with the reduced 13-feature set held constant from Iteration 2.

This progressive approach ensures that each iteration changes exactly one experimental variable, enabling clear attribution of performance differences to specific design decisions across the three-iteration experimental framework.

Training Procedure

Models are trained and evaluated using the following procedures across all three iterations:

- Within-dataset evaluation:
    - Models are trained on the training portion of the data (80%, 16,000 rows per dataset)
    - Five-fold stratified cross-validation is performed on the training set to estimate generalization performance.
    - Models are evaluated on the held-out test set (20%, 4,000 rows per dataset)
- Cross-dataset evaluation:
    - Models are trained on the full dataset from one threat intelligence source are applied directly to the full dataset from the other source (20,000 rows each direction)
    - No retraining, resampling, or feature modification occurs between training and evaluation
    - For Iteration 3 cross-dataset evaluation, grid search is performed on the training dataset before evaluation on the test data, ensuring that hyperparameter selection does not incorporate any information from the test distribution

Reproducibility and Consistency

Reproducibility is a core requirement of the training process to ensure that results can be independently verified.

- A fixed random seed of random_state=42 is applied to all randomly determined operations throughout the pipeline, including:
    - Dataset shuffling and train-test split assignment
    - Cross-validation fold generation
    - Model initialization for all three classifiers across all three iterations

- All models are trained using identical experimental conditions within each iteration including feature sets, preprocessing steps, and evaluation methodology.
- Reproducibility was confirmed by running both the Iteration 2 and Iteration 3 evaluations twice each under identical input conditions, with both runs producing identical results across all metrics, feature importance scores, and best hyperparameter configurations

This guarantees that observed performance differences reflect true model behavior rather than randomness or implementation variation.

Summary of Model Training Approach

This section establishes the consistent training framework applied across all three experimental iterations, ensuring that performance differences observed in Chapter 4 are attributed to specific design decisions rather than variations in training conditions. This combination of a fixed random seed, strict preprocessing isolation, and confirmed reproducibility through independent re-execution collectively satisfies the methodological rigor required for a verifiable experimental record.

3.6.5 Evaluation Metrics

Four evaluation metrics are used to assess model performance consistently across all three iterations and both evaluation directions. These metrics capture different aspects of classification performance and provide a comprehensive understanding of model behavior across both within-dataset and cross-dataset evaluation scenarios.

Primary Evaluation Metrics

The following four metrics are computed for every model, dataset, and evaluation direction throughout the experimental record:

- **Accuracy** measures the overall proportion of correct classifications across both classes, calculated as the number of correct predictions divided by the total number of predictions. While accuracy provides a general indication of model performance, it is considered alongside the other metrics to ensure a complete picture of classification behavior, particularly in cases where class-specific performance differences are meaningful.
- **Precision** measures the proportion of URLs predicted as malicious that are genuinely malicious, calculated as true positives divided by the sum of true positives and false positives. High precision indicates that a model produces few false alarms, meaning legitimate URLs are rarely misclassified as malicious. This metric is important in malicious URL detection contexts where unnecessary

blocking of legitimate URLs can disrupt user workflows or reduce trust in the detection system.

- **Recall** measures the proportion of genuinely malicious URLs that are correctly identified, calculated as true positives divided by the sum of true positives and false negatives. High recall indicates that the model successfully detects the majority of malicious URLs presented to it. In malicious URL detection, recall is weighted as the most operationally critical metric because a false negative, classifying a malicious URL as legitimate, allows a threat to reach an end user undetected, posing a direct security risk.
- **F1-Score** is the harmonic mean of precision and recall, providing a single balanced metric that accounts for both false positives and false negatives simultaneously. The F1-score is particularly informative when both precision and recall carry meaningful consequences, as is the case in malicious URL detection. F1-score is used as the primary optimization metric in Iteration 3 grid search cross-validation, consistent with its role as the primary summary metric throughout the evaluation.

Operational Importance of Recall

The asymmetry between the consequences of false negatives and false positives in malicious URL detection directly shapes the interpretation of results throughout this project. A false positive, a legitimate URL incorrectly classified as malicious, produces an unnecessary warning or block but does not expose the user to harm and can be reviewed or overridden. A false negative, a malicious URL incorrectly classified as legitimate, allows a malicious URL to reach an end user undetected, potentially resulting in credential theft, financial loss, or malware infection. This asymmetry is well established in cybersecurity classification literature and directly informs the emphasis placed on recall when interpreting cross-dataset generalization in Chapter 4 [9].

This operational weighting of recall is reflected in several design decisions throughout the experimental methodology:

- Recall improvement is the primary metric used to assess the value of each iteration in the URLhaus to PhishTank cross-dataset direction, where the largest performance gap was observed
- F1-score is used as the grid search optimization metric in Iteration 3 rather than accuracy, ensuring that hyperparameter selection balances recall against precision rather than optimizing for overall correctness alone
- The confusion matrix is recorded for every model and evaluation direction to provide full transparency into the specific counts of false negatives and false positives, enabling precise quantification of missed threats across all experimental conditions

Confusion Matrix Analysis

The full confusion matrix is recorded for every model, dataset, and evaluation direction across all three iterations. The confusion matrix provides four counts that together characterize the complete classification behavior of each model:

- **True Negatives (TN):** legitimate URLs correctly classified as legitimate
- **False Positives (FP):** legitimate URLs incorrectly classified as malicious
- **False Negatives (FN):** malicious URLs incorrectly classified as legitimate
- **True Positives (TP):** malicious URLs correctly classified as malicious

Reporting the full confusion matrix rather than aggregate metrics alone enables precise interpretation of model behavior that summary statistics may obscure. For example, two models with identical F1-scores may have very different false negative counts depending on how precision and recall are balanced which has direct implications for the real-world threat detection capability of each configuration. In the cross-dataset evaluation directions, confusion matrix counts are particularly informative because they allow the exact number of missed malicious URLs per 10,000 evaluated to be quantified and compared across iterations, providing a concrete and operationally meaningful measure of improvement beyond percentage point changes in recall.

Summary of Evaluation Metrics

The four metrics, accuracy, precision, recall, and F1-score, together with the full confusion matrix provide a complete and operationally grounded evaluation framework that is applied consistently across all three iterations and both evaluation directions. Recall is emphasized as the most operationally critical metric given the asymmetric consequences of false negatives in malicious URL detection, and F1-score serves as the primary optimization target in grid search to ensure balanced model tuning. This evaluation framework directly supports the interpretation of results in Chapter 4 and provides the quantitative foundation for the final model selection decision documented in Section 4.6.

3.7 Cross-Dataset Generalization Procedures

Cross-dataset generalization evaluation is a central component of this project's experimental design, directly addressing Research Question 4 regarding the transferability of learned URL-structural patterns across fundamentally different threat intelligence sources. While within-dataset evaluation measures how well a model performs when training and the test data are drawn from the same distribution, cross-dataset evaluation measures how well decision boundaries learned from one threat source transfer to a completely different source with distinct URL characteristics, collection methodology, and threat category. This distinction is critical for assessing real-world deployment viability, where models will inevitably encounter URLs that differ

structurally from their training data [10], [11]. Cross-dataset evaluation is performed across all three iterations using the appropriate feature set and hyperparameter configuration for each iteration, enabling a complete three-way comparison of generalization performance as the experimental methodology progresses.

3.7.1 Evaluation Design

The cross-dataset evaluation framework simulates realistic deployment conditions by requiring models to operate on entirely unseen data distributions:

- Two bidirectional evaluation directions are executed using the full feature datasets produced by extract_features.py:
    - Direction 1: Train on the complete PhishTank and Tranco dataset (20,000 rows) and evaluate on the complete URLhaus and Tranco dataset (20,000 rows)
    - Direction 2: Train on the complete URLhaus and Tranco dataset (20,000 rows) and evaluate on the complete PhishTank and Tranco dataset (20,000 rows)
- No data from the evaluation dataset is observed during training in either direction
- No retraining, resampling, or feature modification occurs between training and evaluation
- Unlike within-dataset evaluation, the full datasets are used for both training and testing rather than applying an 80/20 split which:
    - Maximizes the available training data patterns and evaluation population size
    - Provides a stronger test of generalization under distribution shift
- The bidirectional design ensures that generalization is evaluated symmetrically:
    - Structural differences between phishing URLs (PhishTank) and malware delivery URLs (URLhaus) may produce asymmetric generalization difficulty
    - Evaluating both directions allows identification of whether generalization occurs, whether performance differs depending on training source, and what these differences reveal about underlying URL structure and threat behavior [4], [5]
- Cross-dataset evaluation is performed in each iteration using the configuration appropriate to that iteration:
    - Iteration 1: full 24-feature set with baseline hyperparameters
    - Iteration 2: reduced 13-feature set with baseline hyperparameters
    - Iteration 3: reduced 13-feature set with grid search optimized hyperparameters, where grid search is performed on the training dataset before evaluation on the test dataset

3.7.2 Preprocessing Isolation

Preprocessing is strictly isolated between training and evaluation datasets to prevent data leakage and preserve the validity of generalization results:

- All preprocessing transformations are fitted on the training dataset
- These transformations are applied to the evaluation dataset using transform only, without recalculating parameters
- Specifically for Logistic Regression:
    - StandardScaler is fitted on the training dataset only
    - The same scaling parameters are applied to the evaluation dataset using transform
- This ensures that:
    - No information from the evaluation dataset influences training
    - Generalization performance reflects true out-of-distribution behavior
- Fitting preprocessing steps on both datasets simultaneously would introduce test data statistics into the training process, producing artificially inflated performance that would not reflect real-world deployment conditions
- For Iteration 3 cross-dataset evaluation, the StandardScaler and grid search are both fitted exclusively on the training dataset, ensuring that neither scaling parameters nor hyperparameter selection incorporate any information from the test distribution

3.7.3 Schema Consistency Enforcement

To ensure valid cross-dataset comparison, feature schema consistently is enforced prior to evaluation:

- The cross_dataset_eval.py script validates that both datasets share identical feature schema before any evaluation begins
- If a mismatch is detected:
    - Execution halts immediately
    - A descriptive error identifies the inconsistent columns
    - The user is instructed to regenerate features using extract_features.py
- This guarantees that:
    - The same feature schema is used for both training and evaluation in every iteration
    - Model inputs remain consistent across datasets regardless of whether the full 24-feaure set or reduced 13-feature set is in use
- Because all features are derived solely from URL string parsing without external data sources, schema consistency is inherently maintained when both datasets are processed through the same feature extraction pipeline

3.7.4 Performance Metrics and Reporting

Model performance is evaluated using the same metrics defined in Section 3.6.5 to ensure consistency between within-dataset and cross-dataset comparisons:

- The following metrics are recorded for each model and evaluation direction across all three iterations:
    - Accuracy
    - Precision
    - Recall
    - F1-score
- Full confusion matrix values are also captured:
    - True Positives (TP)
    - True Negatives (TN)
    - False Positives (FP)
    - False Negatives (FN)
- Results are:
    - Saved to cross_eval_results.csv in the outputs/ directory for Iteration 1
    - Saved to feature_selection_results.csv for Iteration 2
    - Saved to hyperparameter_tuning_results.csv for Iteration 3
    - Logged in timestamped plain-text files for full reproducibility across all iterations
- Visual dashboards are generated for each evaluation direction and each iteration, including:
    - Metric comparisons across all models
    - Confusion matrix heatmaps
    - Top ten feature importance from Random Forest model
    - Delta recall charts in Iterations 2 and 3 showing improvement relative to previous iterations
- Feature importance rankings for each direction are compared across iterations to identify features that demonstrate consistent predictive value across both datasets, directly informing the feature selection process in Iteration 2 and the stability analysis in Section 4.5

Summary of Cross-Dataset Generalization Procedures

The cross-dataset evaluation framework provides a rigorous and deployment-relevant assessment of model generalization across distinct real-world URL distributions across all three experimental iterations. By enforcing strict dataset separation, consistent preprocessing isolation, schema validation, and bidirectional evaluation in every iteration, the methodology isolates the effects of distribution shift on model performance and ensures that performance comparisons across iterations reflect genuine change in

model behavior rather than evaluation condition variation. The resulting three-way comparison documented in Chapter 4 enables identification of both the extent and asymmetry of generalization across threat sources, the impact of feature selection on cross-dataset recall, and the contribution of hyperparameter tuning to further generalization improvement, collectively forming the empirical foundation for the final model selection decision in Section 4.6.

3.8 Design Justification

The design decisions governing this experimental pipeline were not made arbitrarily but reflect deliberate methodological choices aligned with the project's core research contribution and operational constraints. This section articulates the justification for the most consequential design decisions: the URL-string-only feature constraint, dataset selection, pipeline architecture, model selection, and the iterative evaluation framework.

3.8.1 URL-String-Only Feature Constraint

The decision to restrict all feature engineering to information derivable from the raw URL string is the foundational design constraint from which all other decisions follow:

- This constraint was selected for three primary reasons:
    - Consistency across datasets:
        - Ensures that the same feature schema can be applied regardless of when or where URLs were collected
        - Avoids reliance on external resolution such as DNS lookups, which could introduce inconsistencies between datasets collected at different times and from different sources
    - Deployment flexibility:
        - Enables operation in offline, air-gapped, and privacy-sensitive environments
        - Eliminates dependencies on external services or network connectivity at any stage of the pipeline
    - Efficiency and determinism:
        - All feature extraction is based on string parsing and arithmetic operations
        - Ensures near-instantaneous inference with no network latency
- Collectively these properties support key software quality attributes including reliability, portability, and reproducibility, which are essential for research-grade systems intended for constrained or security-sensitive environments [9], [11], [21]
- This constraint also defines the boundary of what the system is designed to evaluate and is consistent with the project's original contribution of establishing a rigorous baseline for fully offline, privacy-preserving malicious URL detection

3.8.2 Dataset Selection Justification

The selection of PhishTank and URLhaus is based on their representation of structurally distinct categories of malicious URL behavior:

- **PhishTank:**
  - Represents credential-harvesting phishing URLs
  - Characterized by deceptive domain structures, obfuscated paths, and human-targeted deception designed to mislead users into believing a site is legitimate
- **URLhaus:**
  - Represents malware delivery and botnet infrastructure URLs
  - Characterized by IP-based hosts, numeric and encoded patterns, and infrastructure-oriented structures designed to facilitate automated malware distribution
- This structural divergence creates a meaningful and challenging generalization test, reflecting real-world threat diversity more accurately than using two datasets from the same threat category would [4], [5]
- The divergence also produces the directional asymmetry in cross-dataset generalization observed across all three iterations, where PhishTank-trained models generalize more readily to URLhaus than the reverse, a finding that would not be observable without structurally distinct threat sources
- **Legitimate dataset- Tranco:**
  - Selected over alternatives such as Alexa Top Sites list because:
    - Rankings are averaged across multiple sources over a rolling time window
    - Reduces the risk of manipulated, SEO-inflated, or transiently popular domains appearing as legitimate examples
  - This makes Tranco a more stable and research-appropriate source for the legitimate class [6]

3.8.3 Pipeline Architecture Justification

The pipeline is designed using a single-responsibility, script-based architecture to maximize reproducibility, modularity, and auditability:

- **Why not monolithic scripts?**
  - Reduce the ability to independently verify individual stages
  - Make debugging and validation more difficult
- **Why not notebook-based workflows?**
  - Depend on execution order rather than explicit data flow contracts
  - Reduce reproducibility and transparency [15]
- **Advantages of the chosen architecture:**
  - Each stage has a single clearly defined responsibility

    - Intermediate outputs are persistent, observable, and independently reproducible
    - Any stage can be re-executed in isolation without requiring prior stages to be rerun, provided its input files are present
- The pipeline follows an ETL-aligned structure where data flows unidirectionally and no stage bypasses or re-reads raw inputs, ensuring:
    - Clear transformation traceability from raw source data through to trained model artifacts
    - Strong separation of concerns between data ingestion, preparation, feature engineering, and modeling
    - Full auditability of all intermediate artifacts [16], [22]

3.8.4 Model Selection Justification

The three selected classifiers represent a deliberate analytical spectrum rather than an arbitrary selection:

- **Logistic Regression:**
    - Establishes the linear separability baseline of the feature space
    - Provides transparent and interpretable coefficient weights
    - Demonstrates the greatest improvement across the three iterations in the URLhaus to PhishTank direction, validating its inclusion as a meaningful experimental subject rather than simply a baseline reference
- **Random Forest:**
    - Captures nonlinear feature interactions without requiring external tokenization or embedding layers
    - Provides feature importance rankings that directly support the feature selection analysis in Iteration 2 and the stability analysis in Section 4.5
    - Achieves the highest overall performance metrics across both datasets and both evaluation directions
- **Decision Tree:**
    - Provides a fully auditable rule-based model where every classification decision can be traced through explicit feature thresholds
    - Selected as the primary final model recommendation due to its combination of competitive cross-dataset performance, full interpretability, and coverage across all three iterations
- Together these models enable systematic analysis of:
    - Model complexity versus performance trade-offs
    - Interpretability versus predictive power
    - Generalization behavior under distribution shift across structurally distinct threat sources

- This structured selection supports evaluation of how classifier design impacts both within-dataset performance and cross-dataset generalization [3], [12].

3.8.5 Iterative Evaluation Justification

The experimental design is structured as three progressive iterations to ensure that all design decisions are data-driven rather than predetermined:

- **Iteration 1:**
  - Establishes baseline performance using the full 24-feature set and baseline hyperparameters
  - Identifies which features contribute differently across datasets through Random Forest importance analysis
  - Reveals the cross-dataset generalization asymmetry between the two evaluation directions that motivates subsequent iterations
- **Iteration 2:**
  - Applies feature selection based on importance patterns observed in Iteration 1
  - Evaluates whether removing dataset-specific low-importance features improves cross-dataset generalization while preserving within-dataset performance
  - Feature selection is the sole experimental variable, isolating its contribution cleanly from other factors
- **Iteration 3:**
  - Applies grid search hyperparameter optimization to the reduced feature set established in Iteration 2
  - Evaluates whether systematic tuning produces further measurable improvement in cross-dataset generalization
  - Hyperparameter configuration is the sole experimental variable, isolating its contribution from the feature selection changes made in Iteration 2
- This three-iteration structure ensures that:
  - Each design decision is empirically justified by the findings of the preceding iteration
  - The contribution of each experimental change can be attributed cleanly and independently
  - The methodology remains reproducible, defensible, and aligned with established principles of iterative machine learning system design [10], [16]

Summary of Design Justification

The design of this experimental pipeline reflects a cohesive set of methodological choices centered on reproducibility, generalization, and deployment realism. By

constraining features to URL-string-only inputs, selecting structurally diverse datasets, implementing a modular pipeline architecture, and evaluating models across an interpretable complexity spectrum, the system is positioned to rigorously assess cross-dataset generalization under strict offline and privacy-preserving constraints. The three-iteration evaluation framework ensures that feature selection and hyperparameter tuning decisions are both grounded in empirical evidence and independently verifiable, resulting in a strongly defensible experimental design aligned with real-world cybersecurity deployment requirements.

3.9 Summary

Chapter 3 presented the complete advanced design of the privacy-preserving, offline malicious URL detection system, including the architectural principles, pipeline structure, data preparation procedures, feature engineering methodology, model design rationale, experimental setup across three iterative evaluation rounds, cross-dataset generalization procedures, and justification for each major design decision.

All components of the system are governed by a single foundational constraint: classification must be performed using only URL-string-derived features, without reliance on external lookups, network dependencies, or runtime service calls. This constraint directly shaped the feature schema, pipeline architecture, dataset selection strategy, and iterative experimental methodology. These design choices are intentionally interdependent, with each component reinforcing the others to produce a system that is reproducible, transparent, and suitable for deployment in constrained security-sensitive environments [13] [9], [11].

The effectiveness of this design is evaluated through three progressive experimental iterations: Iteration 1 establishes the baseline performance of all three classifiers using the full 24-feature schema, Iteration 2 applies data-driven feature selection to identify and remove low-importance features, and Iteration 3 applies grid search hyperparameter optimization to the reduced feature set. Each iteration changes exactly one experimental variable while holding all other conditions constant, ensuring that the performance differences observed are attributable to specific and defensible design decisions. Cross-dataset generalization testing evaluates both directions of transfer between the PhishTank and URLhaus threat intelligence sources, providing quantitative evidence of model behavior under realistic distribution shift conditions. The empirical results of all three iterations and both evaluation directions are presented in Chapter 4.

CHAPTER 4: RESULTS AND FINDINGS

This chapter presents the empirical evaluation of the malicious URL detection system, with the objective of assessing performance, generalization, and reproducibility across three iterative experimental rounds. All results reported in this chapter were produced by executing the pipeline described in Chapter 3 on a single local machine using Python 3.11.9 within an isolated virtual environment. All randomly determined operations were controlled using a fixed random_state of 42 to ensure consistency across runs. Reproducibility was further validated by executing the Iteration 2 and Iteration 3 evaluations twice under identical input conditions, with both runs producing identical results across all evaluation metrics, thereby confirming the deterministic behavior of the pipeline.

The chapter is structured to progressively present baseline performance, generalization behavior, and model refinement outcomes. Section 4.1 and 4.2 establish the Iteration 1 baseline through within-dataset and cross-dataset evaluation using the full 24-feature schema. Sections 4.3 and 4.4 assess the impact of feature selection and hyperparameter tuning, including a comparative analysis across iterations. Section 4.5 examines feature stability, followed by Section 4.6, which defines the final system configuration and ensemble-based model integration. 4.7 and 4.8 synthesize results across all iterations and summarize the key findings. Collectively, these sections provide a comprehensive empirical evaluation that directly addresses the five research questions defined in Chapter 1.

4.1 Within-Dataset Performance (Iteration 1 Baseline)

Iteration 1 establishes the baseline performance of all three classifiers on both datasets using the full 24-feature schema and baseline hyperparameters. Each dataset was partitioned using an 80/20 stratified train-test split producing 16,000 training rows and 4,000 test rows per dataset. Five-fold stratified cross-validation was applied to the training set for each model and each dataset. Results are reported on the held-out test set. These scores reflect each model's performance on data it had never been exposed to during training, providing a fair and unbiased measure of classification performance on unseen test data.

4.1.1 PhishTank and Tranco Dataset Results

All three classifiers achieved a strong within-dataset performance on the PhishTank and Tranco dataset, with F1-scores ranging from 0.9972 to 0.9980 on the 4,000-row test set.

Table 2 *Within-Dataset Performance Results: PhishTank and Tranco Dataset (Iteration 1, Full 24 Features) Note. Data sourced from model_results.csv in the outputs/ directory*

Model	Accuracy	Precision	Recall	F1-score	Test Rows
Logistic Regression	0.9972	0.9990	0.9955	0.9972	4,000
Random Forest	0.9975	0.9980	0.9970	0.9975	4,000

Decision Tree (depth=10)	0.9980	0.9985	0.9975	0.9980	4,000

The Decision Tree achieved the highest F1-score of 0.9980 on this dataset, marginally outperforming Random Forest at 0.9975 and Logistic Regression at 0.9972. All three models achieved precision above 0.9980, indicating that false positive rates were low, with four or fewer legitimate URLs per 2,000 incorrectly classified as malicious across all models. Recall ranged from 0.9955 to 0.9975, with Logistic Regression producing the most false negatives at nine missed malicious URLs out of 2,000, compared to five for Decision Tree and six for Random Forest.

The five-fold cross-validation F1-scores on the training set were 0.9964 ± 0.0011 for Logistic Regression, 0.9967 ± 0.0009 for Random Forest, and 0.9962 ± 0.0010 for Decision Tree, indicating consistent performance across folds with no evidence of significant overfitting.

The top five features by Random Forest importance for PhishTank training were path_length (0.3144), num_slashes (0.2776), url_entropy (0.1005), url_length (0.0684), and num_dots (0.0550). Together these five features account for approximately 81.6% of the total importance weight, indicating that URL path structure and entropy-based obfuscation are the primary features contributing to classification decisions in the PhishTank dataset. The complete top ten feature importance rankings are shown in Table 3.

Table 3 *Top 10 Feature Importances: PhishTank Training (Iteration 1) Note. Data sourced from model_results.csv in the outputs/ directory.*

Feature	Importance Score
path_length	0.3144
num_slashes	0.2776
url_entropy	0.1005
url_length	0.0684
num_dots	0.0550
subdomain_depth	0.0433
hostname_length	0.0323
num_digits	0.0290
hostname_entropy	0.0221
digit_ratio	0.0216

4.1.2 URLhaus and Tranco Dataset Results

All three classifiers achieved near-perfect within-dataset performance on the URLhaus and Tranco dataset, with F1-scores ranging from 0.9997 to 1.0000 on the 4,000-row test set.

Table 4 *Within-Dataset Performance Results: URLhaus and Tranco Dataset (Iteration 1, Full 24 Features) Note. Data sourced from model_results.csv in the outputs/ directory.*

Model	Accuracy	Precision	Recall	F1-score	Test Rows
Logistic Regression	0.9998	1.0000	0.9995	0.9997	4,000
Random Forest	0.9998	1.0000	0.9995	0.9997	4,000
Decision Tree (depth=10)	1.0000	1.0000	1.0000	1.0000	4,000

The Decision Tree achieved a perfect F1-score of 1.000 with zero false positives and zero false negatives on the URLhaus test set. Logistic Regression and Random Forest both achieved 0.9997 F1 with identical confusion matrices showing zero false positives and one false negative each. The five-fold cross-validation F1-scores were 0.9997 ± 0.0001 for Logistic Regression, 0.9999 ± 0.0001 for Random Forest, and 0.9999 ± 0.0001 for Decision Tree, indicating a highly stable and consistent training data patterns with minimal variance across folds.

The URLhaus feature importance distribution differs notably from PhishTank. The top five features were path_length (0.2369), num_slashes (0.2284), num_dots (0.1140), url_length (0.0981), and subdomain_depth (0.0786). Notably, url_entropy drops to outside the top five for URLhaus, scoring only 0.0078 compared to 0.1005 for PhishTank, while num_dots and subdomain_depth gain substantially more weight. This shift in importance distribution reflects the structural differences between credential-harvesting phishing URLs and malware delivery URLs discussed in Chapter 1. The complete top ten feature importance ranking for URLhaus are shown in Table 5.

Table 5 *Top 10 Feature Importances: URLhaus Training (Iteration 1) Note. Data sourced from model_results.csv in the outputs/ directory.*

Feature	Importance Score
path_length	0.2369
num_slashes	0.2284
num_dots	0.1140
url_length	0.0981
subdomain_depth	0.0786
num_digits	0.0706
digit_ratio	0.0395

has_https	0.0381
special_char_ratio	0.0291
has_ip	0.0250

4.1.3 Within-Dataset Performance Summary

The Iteration 1 baseline results confirm that the 24 URL-string-only features provide strong discriminative signal for malicious URL detection within matched training and test distributions. All three classifiers achieved F1-scores above 0.997 on both datasets without relying on external lookups, network queries, or supplementary data sources, providing evidence relevant to research question one regarding the viability of offline URL-string-only detection. The URLhaus dataset produced consistently higher within-dataset performance than PhishTank across all three models, which is consistent with its lower feature importance variance and the more consistent structural patterns observed in malware delivery URLs compared to the more diverse credential-harvesting patterns in PhishTank. Feature importance rankings for both datasets are shown in Tables 3 and 5. The dashboard visualizations for the within-dataset Iteration 1 evaluation are shown in Figure 2 (PhishTank and Tranco) and Figure 3 (URLhaus and Tranco).

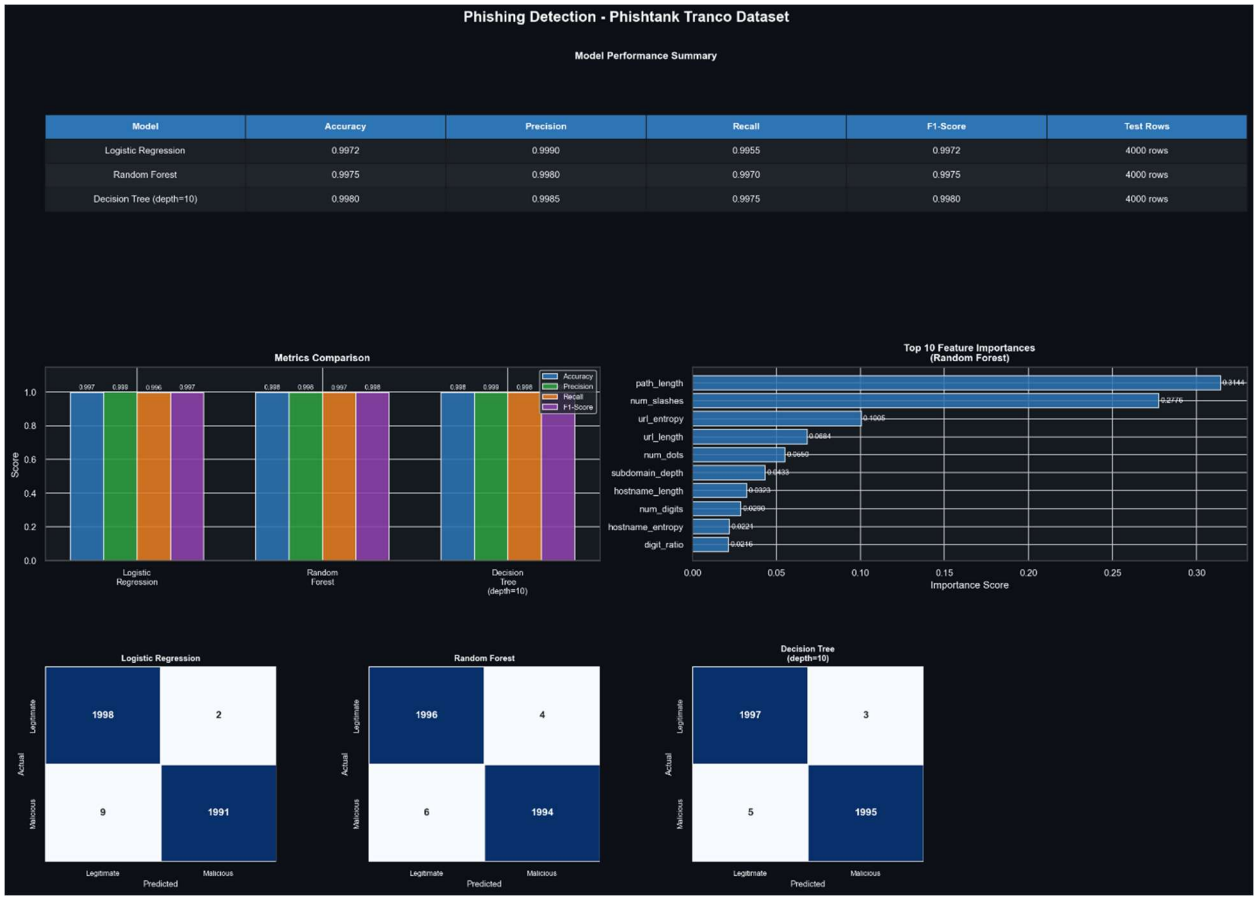


Figure 2 Within-Dataset Evaluation Dashboard: PhishTank and Tranco Dataset
(Iteration 1, Full 24 Features)
Comparison of accuracy, precision, recall, and F1-score across Logistic Regression, Random Forest, and Decision Tree on the 4,000-row held-out test set, including confusion matrix summaries and per-class classification reports. Generated by train_models.py and saved as dashboard_features_phishtank_tranco_dataset.png in the outputs/ directory.

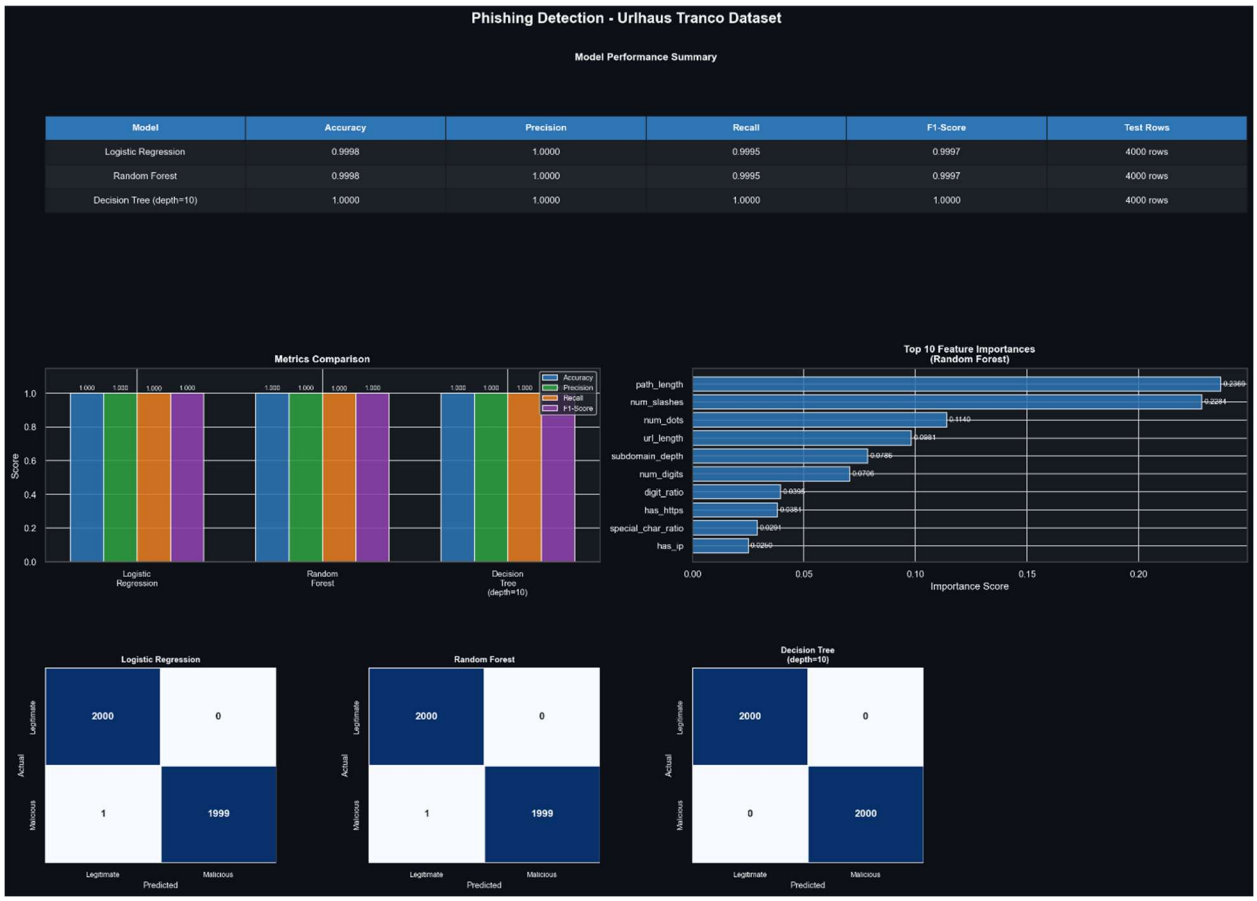


Figure 3 Within-Dataset Evaluation Dashboard: URLhaus and Tranco Dataset
(Iteration 1, Full 24 Features)
Comparison of accuracy, precision, recall, and F1-score across Logistic Regression, Random Forest, and Decision Tree on the 4,000-row held-out test set, including confusion matrix summaries and per-class classification reports. Generated by train_models.py and saved as dashboard_features_urlhaus_tranco_dataset.png in the outputs/ directory.

4.2 Cross-Dataset Generalization

This section extends the baseline by evaluating cross-dataset generalization. Cross-dataset generalization evaluation trains each model on the complete dataset from one

threat intelligence source and evaluates it on the complete dataset from the other source without any retraining, resampling, or feature modification. Both datasets contain 20,000 rows each for this evaluation. The full dataset, rather than the 80/20 split, is used to maximize the amount of training data available and test population in each direction. Two directions are evaluated as described in Section 3.7.

4.2.1 Direction 1: Train PhishTank, Test on URLhaus

All three classifiers demonstrated strong generalization from PhishTank training data to the URLhaus test set, with F1-scores ranging from 0.9991 to 1.0000.

Table 6 *Cross-Dataset Generalization Results: Train on PhishTank, Test on URLhaus (Iteration 1, Full 24 Features) Note. Data sourced from cross_eval_results.csv in the outputs/ directory.*

Model	Accuracy	Precision	Recall	F1-score	Test Rows
Logistic Regression	0.9991	0.9989	0.9993	0.9991	20,000
Random Forest	1.0000	0.9999	1.0000	1.0000	20,000
Decision Tree (depth=10)	0.9996	0.9991	1.0000	0.9996	20,000

Random Forest achieved perfect 1.0000 F1 with only one false positive and zero false negatives across the 20,000 test examples. Decision Tree achieved 0.9996 F1 with zero false negatives and nine false positives. Logistic Regression achieved 0.9991 F1 with eleven false positives and seven false negatives. The near-perfect performance in this direction suggests that the URL path structure and entropy patterns learned from PhishTank training data transfer effectively to the URLhaus threat distribution, likely because the most stable features of path_length and num_slashes are strong predictors in both threat categories.

4.2.2 Direction 2: Train on URLhaus, Test on PhishTank

The URLhaus to PhishTank direction produced notably lower performance than the reverse direction, with F1-scores ranging from 0.9730 to 0.9904, revealing a meaningful asymmetry in cross-dataset generalization capability.

Table 7 *Cross-Dataset Generalization Results: Train on URLhaus, Test on PhishTank (Iteration 1, Full 24 Features) Note. Data sourced from cross_eval_results.csv in the outputs/ directory.*

Model	Accuracy	Precision	Recall	F1-score	Test Rows
Logistic Regression	0.9738	1.0000	0.9475	0.9730	20,000

Random Forest	0.9831	1.0000	0.9662	0.9828	20,000
Decision Tree (depth=10)	0.9905	1.0000	0.9810	0.9904	20,000

All three models achieved perfect precision of 1.0000 in this direction, meaning zero false positives, no legitimate URLs were misclassified as malicious. However, recall ranged from 0.9475 to 0.9810, indicating that between 190 and 525 malicious PhishTank URLs per 10,000 were incorrectly classified as legitimate by models trained exclusively on URLhaus data. The recall gap is most pronounced for Logistic Regression at 5.25% below perfect recall, moderate for Random Forest at 3.38%, and smallest for Decision Tree at 1.90%.

This asymmetry is meaningful research finding. Models trained on URLhaus malware delivery URL patterns encounter difficulty fully identifying the credential-harvesting obfuscation patterns characteristic of PhishTank URLs, particularly the entropy-based path randomization that scored 0.1005 importance for PhishTank training but only 0.0078 for URLhaus training. Because URLhaus-trained models assign low weight to url_entropy and hostname_entropy, they lack the learned sensitivity to these signals when applied to PhishTank test data. The cross-dataset recall gap does not necessarily indicate model failure but instead reflects a genuine underlying structural difference between two distinct categories of malicious URL behavior, as discussed in Section 4.5.

The dashboard visualizations for the cross-dataset Iteration 1 evaluation are shown in Figure 4 (PhishTank to URLhaus) and Figure 5 (URLhaus to PhishTank).

Figure 4 Cross-Dataset Evaluation Dashboard: Train on PhishTank, Test on URLhaus
(Iteration 1, Full 24 Features)
Metric comparisons across all three classifiers evaluated on the full 20,000-row URLhaus dataset after training on PhishTank data, including confusion matrix heatmaps and feature importance rankings. Generated by cross_dataset_eval.py and saved as dashboard_cross_eval_PhishTank_to_URLhaus.png in the outputs/ directory.

Figure 5 Cross-Dataset Evaluation Dashboard: Train on URLhaus, Test on PhishTank
(Iteration 1, Full 24 Features)
Metric comparisons across all three classifiers evaluated on the full 20,000-row PhishTank dataset after training on URLhaus data, including confusion matrix heatmaps and feature importance rankings. Generated by cross_dataset_eval.py and saved as dashboard_cross_eval_URLhaus_to_PhishTank.png in the outputs/ directory.

4.3 Feature Selection Results (Iteration 2)

Building on the baseline results, Iteration 2 applies data-driven feature selection to identify and remove URL-string features that do not contribute consistently to classification performance across both threat intelligence sources. The feature selection procedure is described in full in Section 3.6.2. All three classifiers are retrained and re-evaluated using the reduced feature set under identical experimental conditions to Iteration 1.

4.3.1 Feature Selection Outcome

Random Forest importance scores averaged across both datasets produced the following selection outcome at threshold=0.01:

- Features retained: 13 of 24
- Features dropped: 11 of 24
- Dropped features:
    - num_hyphens
    - has_port
    - num_qmarks
    - num_equals
    - num_underscores
    - query_length
    - num_ampersands
    - has_www
    - num_percent
    - has_at_in_url
    - num_at

The 11 dropped features all scored below 0.01 mean importance across both datasets. Three of them, num_percent, has_at_in_url, and num_at, scored exactly 0.0000 on both datasets, indicating zero contribution to classification in either threat context. The remaining eight dropped features scored between 0.0002 and 0.0059 mean importance, representing collectively less than 2% of the total predictive weight distributed across the 24 features.

4.3.2 Within-dataset Impact on Feature Reduction

Removing 11 features had negligible impact on within-dataset performance across both datasets and all three models.

Table 8 *Within-Dataset Performance Comparison: Full 24 vs. Reduced 13 Features (Iteration 2) Note. Data sourced from feature_selection_results.csv in the outputs/ directory.*

Dataset	Model	Full 24 F1	Reduced 13 F1	Δ F1
PhishTank	Logistic Regression	0.9972	0.9972	0.0000
PhishTank	Random Forest	0.9980	0.9977	-0.0003
PhishTank	Decision Tree (depth=10)	0.9980	0.9980	0.0000
URLhaus	Logistic Regression	0.9997	0.9997	0.0000
URLhaus	Random Forest	0.9997	0.9997	0.0000
URLhaus	Decision Tree (depth=10)	1.0000	1.0000	0.0000

The largest within-dataset change was Random Forest on PhishTank declining by 0.0003 F1, which is negligible in magnitude. URLhaus within-dataset performance was completely unchanged across all three models. These results confirm that the 11 dropped features were genuinely redundant and contributed no meaningful discriminative signal to within-dataset classification.

4.3.3 Cross-Dataset Impact of Feature Reduction

The impact of feature reduction on cross-dataset generalization was small but consistently positive.

Table 9 *Cross-Dataset Performance Comparison: Full 24 vs. Reduced 13 Features (Iteration 2) Note. Data sourced from feature_selection_results.csv in the outputs/ directory.*

Dataset	Model	Full 24 F1	Reduced 13 F1	Δ F1	Δ Recall
PhishTank → URLhaus	Logistic Regression	0.9991	0.9995	+0.0004	+0.0007
PhishTank → URLhaus	Random Forest	1.0000	1.0000	0.0000	0.0000
PhishTank → URLhaus	Decision Tree (depth=10)	0.9996	0.9996	0.0000	0.0000
URLhaus → PhishTank	Logistic Regression	0.9730	0.9806	+0.0076	+0.0144
URLhaus → PhishTank	Random Forest	0.9828	0.9829	+0.0001	+0.0001
URLhaus → PhishTank	Decision Tree (depth=10)	0.9904	0.9904	0.0000	0.0000

The greatest improvement was Logistic Regression in the URLhaus to PhishTank direction, where recall improved from 0.9475 to 0.9619 (+0.0144) and F1-score improved from 0.9730 to 0.9806 (+0.0076). This improvement provides evidence relevant to research question five regarding measurable improvements through iterative optimization. Removing the low-importance noise features reduced the influence of low-importance features in the Logistic Regression linear decision boundary, improving its ability to generalize across the distribution shift (i.e., differences between training and test data distribution) between URLhaus training data and PhishTank test data. Tree-based models showed negligible changes in both directions, consistent with their

reduced sensitivity to irrelevant features through bootstrap sampling and threshold-based splitting mechanisms that inherently reduce sensitivity to low-importance inputs.

The Iteration 2 within-dataset and cross-dataset comparison dashboards are shown in Figure 6 and 7.

Figure 6 Feature Selection Impact Dashboard: Within-Dataset Evaluation (Iteration 2)

Side-by-side comparison of full 24-feature and reduced 13-feature performance across all three classifiers on both datasets, including feature importance bar charts with the 0.01 threshold line and ΔF1 annotations. Generated by feature_selection_eval.py and saved as dashboard_feature_selection_within.png in the outputs/ directory.

Figure 7 Feature Selection Impact Dashboard: Cross-Dataset Evaluation (Iteration 2)
Side-by-side comparison of full 24-feature and reduced 13-feature cross-dataset generalization performance across both evaluation directions, including ΔRecall charts showing improvement relative to the Iteration 1 baseline. Generated by feature_selection_eval.py and saved as dashboard_feature_selection_cross.png in the outputs/ directory.

4.4 Hyperparameter Tuning Results (Iteration 3)

Iteration 3 applies GridSearchCV with 5-fold stratified cross-validation scored by F1-score to identify the best-performing hyperparameter configuration within the defined search grid for each model on the Iteration 2 reduced 13-feature set. The grid search parameter spaces and methodology are described in Section 3.6.3. Results are compared against both Iteration 1 and Iteration 2 to produce a complete three-way experimental record.

4.4.1 Best Hyperparameter Configurations

Grid search identified the following best configurations for each model and dataset context:

Table 10 *Best Hyperparameter Configurations from Grid Search (Iteration 3) Note. Data sourced from best_params.csv in the outputs/ directory.*

Dataset	Model	Best Parameters	CV F1 Score
PhishTank	Logistic Regression	C=100.0	0.9967
PhishTank	Random Forest	n_estimators=100, max_depth=10	0.9971
PhishTank	Decision Tree	max_depth=15, min_samples_split=30	0.9968
URLhaus	Logistic Regression	C=100.0	0.9999
URLhaus	Random Forest	n_estimators=100, max_depth=None	0.9999
URLhaus	Decision Tree	max_depth=5, min_samples_split=10	0.9999

The near-perfect cross-validation performance observed on the URLhaus dataset (F1 ≈ 0.9999) warrants careful interpretation. This level of performance suggests that the dataset may contain highly distinguishable structural patterns relative to benign Tranco domains. One possible explanation is that URLhaus URLs include features such as IP-based addressing, irregular path structures, or obfuscation patterns that are less common in benign domains. These characteristics may create strong feature signals that enable models to achieve near-perfect classification without requiring complex decision boundaries.

While preprocessing controls were implemented to prevent data leakage (e.g., feature scaling restricted to training data, strict dataset separation, and no external feature resolution), the possibility of dataset-specific artifacts influencing performance cannot be fully excluded. As a result, cross-dataset evaluation remains the primary measure of generalization in this study, providing a more realistic assessment of model performance under distribution shift.

Two findings from the best parameter configurations are worth noting. First, the optimal Decision Tree depth differs substantially between the two datasets: PhishTank training favored max_depth=15, while URLhaus training favored max_depth=5. This indicates that PhishTank URL patterns require a more complex decision structure in order to capture the entropy-based obfuscation characteristics of credential-harvesting URLs, whereas URLhaus malware delivery URL patterns are separable with a much shallower tree.

Second, both Logistic Regression configurations selected C=100.0 for within-dataset evaluation, indicating that the reduced 13-feature set benefits from low regularization pressure, allowing the model to fully fit the training data with less regularization. In contrast, the URLhaus cross-dataset evaluation direction selected C=10.0, suggesting that slightly stronger regularization improves generalization under distribution shift to PhishTank test data.

4.4.2 Within-Dataset Performance After Tuning

Hyperparameter tuning produced negligible changes in within-dataset performance, suggesting that the system was already near the maximum performance observed under the current feature set and experimental conditions.

Table 11 *Within-Dataset Performance: Three-Way Comparison (Iterations 1, 2, and 3) Note. Data compiled from model_results.csv, feature_selection_results.csv, and hyperparameter_tuning_results.csv in the outputs/ directory.*

Dataset	Model	Iteration 1 F1	Iteration 2 F1	Iteration 3 F1	Δ F1 (1→3)
PhishTank	Logistic Regression	0.9972	0.9972	0.9970	-0.0002
PhishTank	Random Forest	0.9975	0.9977	0.9977	+0.0002
PhishTank	Decision Tree (depth=10)	0.9980	0.9980	0.9977	-0.0003
URLhaus	Logistic Regression	0.9997	0.9997	0.9997	0.0000
URLhaus	Random Forest	0.9997	0.9997	0.9997	0.0000
URLhaus	Decision Tree (depth=10)	1.0000	1.0000	1.0000	0.0000

URLhaus within-dataset performance remained completely stable across all three iterations for all models. PhishTank showed minor fluctuations within ±0.0003 F1, indicating minimal practical change in classification performance. These results indicate that the iterative experimental process did not degrade within-dataset performance while pursuing cross-dataset generalization improvements.

4.4.3 Cross-Dataset Performance After Tuning

The cross-dataset results from Iteration 3 reveal an important pattern that distinguishes model behavior between the two evaluation directions.

Table 12 *Cross-Dataset Performance: Three-Way Comparison (Iterations 1, 2, and 3) Note. Data compiled from cross_eval_results.csv, feature_selection_results.csv, and hyperparameter_tuning_results.csv in the outputs/ directory.*

Direction	Model	Iteration 1 F1	Iteration 2 F1	Iteration 3 F1	Δ F1 (1→3)	Δ Recall (1→3)
PhishTank → URLhaus	Logistic Regression	0.9991	0.9995	0.9993	+0.0002	+0.0007

PhishTank → URLhaus	Random Forest	1.0000	1.0000	0.9999	-0.0001	0.0000
PhishTank → URLhaus	Decision Tree (depth=10)	0.9996	0.9996	0.9989	-0.0007	0.0000
URLhaus → PhishTank	Logistic Regression	0.9730	0.9806	0.9835	+0.0105	+0.0200
URLhaus → PhishTank	Random Forest	0.9828	0.9829	0.9828	0.0000	0.0000
URLhaus → PhishTank	Decision Tree (depth=10)	0.9904	0.9904	0.9904	0.0000	0.0000

The PhishTank to URLhaus direction shows negligible variation across all three iterations for all models, suggesting that this direction was already at the maximum performance in Iteration 1 and that neither feature reduction nor hyperparameter tuning produced meaningful change. The small declines observed for Random Forest (-0.0001) and Decision Tree (-0.0007) in Iteration 3 are within the margin of variation expected from differences in hyperparameter configurations and do not provide strong evidence of meaningful performance degradation.

The URLhaus to PhishTank direction provides the clearest evidence of improvement across iterations. Logistic Regression improved consistently across all three iterations, achieving a cumulative F1 improvement of +0.0105 and a cumulative recall improvement of +0.0200 from Iteration 1 to Iteration 3. In practical terms, the Iteration 3 tuned Logistic Regression correctly identifies 9,675 malicious PhishTank URLs per 10,000 compared to 9,475 in Iteration 1, representing 200 additional detected threats per 10,000 evaluated. Random Forest and Decision Tree showed no meaningful change across iterations in this direction, with their Iteration 1 performance showing little additional improvement from feature reduction or hyperparameter tuning under the tested configuration.

The Iteration 3 three-way comparison dashboards are shown in Figure 8 and Figure 9. The complete three-iteration experimental record established in this section provides the quantitative foundation for the feature stability analysis in Section 4.5 and the final model selection decision in Section 4.6.

Figure 8 Hyperparameter Tuning Three-Way Comparison Dashboard: Within-Dataset Evaluation
(Iteration 3)
Three-way comparison of Iteration 1 baseline, Iteration 2 reduced features, and Iteration 3 tuned configurations across all three classifiers on both datasets, with best hyperparameter configurations shown in metrics tables. Generated by hyperparameter_tuning_eval.py and saved as dashboard_hyperparameter_tuning_within.png in the outputs/ directory.

Figure 9 Hyperparameter Tuning Three-Way Comparison Dashboard: Cross-Dataset Evaluation (Iteration 3)

Three-way comparison of all three iterations across both cross-dataset evaluation directions, including ΔRecall charts comparing Iteration 3 tuned models against the Iteration 1 baseline. Generated by hyperparameter_tuning_eval.py and saved as dashboard_hyperparameter_tuning_cross.png in the outputs/ directory.

4.5 Feature Stability Analysis

Feature stability analysis evaluates the consistency of predictive contribution across both datasets and both cross-dataset evaluation directions. In this analysis, stability is assessed using Random Forest importance scores from both datasets and the averaged importance threshold used in Iteration 2. A feature is considered stable if its importance score remains meaningful across both the PhishTank and URLhaus training contexts, indicating that it captures URL structural patterns associated with malicious behavior regardless of the specific threat category represented in the training data. Features that score highly on one dataset but negligibly on the other are considered dataset-specific and may contribute to distribution-specific decision boundaries that do not generalize well across threat sources.

4.5.1 Feature Importance Consistency Across Datasets

The feature importance comparison conducted in Iteration 2 using Random Forest importance scores averaged across both datasets produced the following stability classifications for all 24 features:

Table 13 *Feature Importance Scores and Stability Classification*
Note. Data sourced from feature_importances_comparison.csv in the outputs/ directory.

Feature	PhishTank	URLhaus	Mean	Classification
path_length	0.3218	0.2113	0.2666	Stable: top predictor both sources
num_slashes	0.2469	0.2051	0.2260	Stable: top predictor both sources
url_length	0.0900	0.1183	0.1041	Stable: consistent across sources
num_dots	0.0560	0.0971	0.0765	Stable: consistent across sources
subdomain_depth	0.0491	0.0936	0.0714	Stable: consistent across sources
has_https	0.0082	0.0847	0.0465	Moderately stable: stronger for URLhaus
num_digits	0.0312	0.0598	0.0455	Moderately stable: stronger for URLhaus
url_entropy	0.0791	0.0078	0.0434	Moderately stable: stronger for PhishTank
digit_ratio	0.0288	0.0425	0.0356	Moderately stable: consistent direction
hostname_length	0.0378	0.0134	0.0256	Moderately stable: stronger for PhishTank
special_char_ratio	0.0119	0.0255	0.0187	Moderately stable: consistent direction
hostname_entropy	0.0217	0.0078	0.0147	Moderately stable: stronger for PhishTank
has_ip	0.0000	0.0216	0.0108	Threshold survivor: URLhaus specific
num_hyphens	0.0111	0.0007	0.0059	Unstable: PhishTank specific, dropped
has_port	0.0000	0.0093	0.0047	Unstable: URLhaus specific, dropped
num_qmarks	0.0032	0.0001	0.0016	Unstable: near zero both sources, dropped
num_equals	0.0016	0.0001	0.0009	Unstable: near zero both sources, dropped
num_underscores	0.0000	0.0010	0.0005	Unstable: near zero both sources, dropped

query_length	0.0007	0.0001	0.0004	Unstable: near zero both sources, dropped
num_ampersands	0.0007	0.0000	0.0004	Unstable: near zero both sources, dropped
has_www	0.0003	0.0001	0.0002	Unstable: near zero both sources, dropped
num_percent	0.0000	0.0000	0.0000	Unstable: zero contribution, dropped
has_at_in_url	0.0000	0.0000	0.0000	Unstable: zero contribution, dropped
num_at	0.0000	0.0000	0.0000	Unstable: zero contribution, dropped

The two most stable features, path_length and num_slashes, together account for approximately 48.3% of the mean importance weight and maintain their top two ranking on both datasets independently. This consistency across structurally distinct threat sources indicates that URL path complexity and slash count are consistently weighted indicators of malicious URL structure in this experiment regardless of whether the threat is credential harvesting or malware delivery. The url_length feature ranks third by mean importance and shows a reverse pattern between sources, scoring slightly higher for URLhaus than PhishTank, suggesting that, in this dataset, URLhaus malware delivery URLs show more consistent length-based patterns while phishing URLs exhibit more extreme length variation.

4.5.2 Dataset-Specific Feature Behavior

Three features warrant specific attention due to their asymmetric importance scores across the two datasets.

url_entropy scores a 0.0791 for PhishTank but only a 0.0078 for URLhaus, a tenfold difference. This asymmetry is consistent with the structural characteristics of each threat category. Credential-harvesting phishing URLs in this dataset appear to include more randomized or obfuscated path components and obfuscated tokens designed to evade pattern matching, producing high character-level entropy. URLhaus malware delivery URLs, by contrast, often use shorter and more structurally predictable path patterns tied to specific malware families and botnet infrastructure, resulting in lower entropy variation across the dataset. This feature survived the Iteration 2 threshold because its PhishTank contribution was sufficiently high to elevate the mean above 0.01, but its near-zero URLhaus score indicates it is primarily a PhishTank-specific predictor.

has_https shows the inverse asymmetry, scoring 0.0082 from PhishTank but 0.0847 for URLhaus. This may reflect the documented adoption of HTTPS certificates by phishing sites, which has significantly reduced the discriminative power of the HTTPS indicator for credential-harvesting URLs since both malicious and legitimate sites commonly use HTTPS [12]. URLhaus malware delivery infrastructure, however, still frequently uses

plain HTTP connections, making has_https a more useful discriminator for that threat category.

has_ip scores a 0.0000 for PhishTank and a 0.0216 for URLhaus. In this dataset, raw IP addresses appear more characteristic of URLhaus malware delivery URLs than PhishTank URLs where operators may avoid domain registration to reduce attribution risk but are rarely observed in credential-harvesting phishing URLs which rely on deceptive domain names to mislead users into believing a site is legitimate. This feature is effectively a URLhaus-only predictor that survived the threshold solely due to its URLhaus contribution.

4.5.3 Implications for Cross-Dataset Generalization

The feature stability analysis helps explain the asymmetric cross-dataset generalization performance observed across all three iterations. The PhishTank to URLhaus direction consistently achieves near-perfect performance because the most stable features, path_length, num_slashes, and url_length, have sufficient predictive weight in the URLhaus context to maintain strong classification even when the model was trained only on PhishTank patterns. The URLhaus to PhishTank direction consistently shows lower recall because several features carry meaningful weight in PhishTank, particularly url_entropy and hostname_entropy, but have near-zero importance for URLhaus. Therefore, they receive insufficient weighting in URLhaus-trained models. When these models encounter PhishTank's entropy-based obfuscation patterns at test time, they lack the learned sensitivity to those signals that a PhishTank-trained model would possess.

The improvement observed for Logistic Regression across the three iterations in the URLhaus to PhishTank direction, from recall 0.9475 in Iteration 1 to 0.9675 in Iteration 3, is consistent with this explanation. Removing the 11 low-importance features in Iteration 2 reduced noise in the linear decision boundary. Additionally, the C=10.0 regularization parameter identified in Iteration 3 grid search for URLhaus cross-evaluation training provided additional boundary smoothing that helped the model generalize across the entropy-based patterns it had not been trained to recognize directly. The residual recall gap of approximately 3.25% suggests that URL-string features alone may not fully capture the structural differences between the two threat categories without direct exposure to PhishTank training examples.

4.6 Final Model Selection and System Lock-in

The final model configuration for the prototype demonstration is selected based on the results across three evaluation dimensions: within-dataset performance, cross-dataset generalization, and feature stability. System lock-in refers to fixing the final configuration, feature set, and preprocessing steps to ensure reproducible deployment. The selection prioritizes a configuration that balances classification performance, deployment feasibility, and interpretability across both threat intelligence sources.

4.6.1 Final Model Selection, Ensemble Configuration, and System Lock-in

Based on the three-iteration experimental record, the following final configuration is selected for the prototype system:

Primary Individual Model Recommendation: Decision Tree with Tuned Hyperparameters on the Reduced 13-Feature Set

This recommendation applies to the selection of a single interpretable model; however, the deployed system uses a majority-vote ensemble combining all three classifiers, as described below.

The Decision Tree achieves 0.9904 F1 and 0.9810 recall in the URLhaus-to-PhishTank cross-dataset direction. This represents the most challenging generalization condition evaluated in this study. It also achieves 0.9978 to 1.0000 F1 across within-dataset evaluation. Its performance is consistent across all three iterations with no degradation from feature reduction or hyperparameter tuning, indicating stable performance across experimental conditions.

The Iteration 3 tuned configuration of max_depth=5 and min_samples_split=10 for URLhaus training, and max_depth=15 and min_samples_split=30 for PhishTank training, provides improved flexibility compared to the baseline depth=10 constraint by allowing the model to adapt its structural complexity to the characteristics of each training source.

The Decision Tree's key advantage for deployment in the offline, privacy-preserving context defined by this project is its auditability. Each classification decision can be traced through a finite set of URL-string feature thresholds without requiring probabilistic interpretation or ensemble aggregation, making it one of the most transparent and operationally verifiable classifier among the three evaluated. In environments where classification decisions may need to be justified to analysts or audited for compliance, this interpretability is a practical deployment advantage that the ensemble models do not provide the same extent.

Supporting Recommendation: Random Forest as the Highest-Accuracy Alternative

Random Forest achieves the highest overall performance metrics across both datasets and both evaluation directions, with F1-scores raning from 0.9828 to 1.0000 across all evaluation scenarios. In the PhishTank to URLhaus direction, it achieves 1.0000 F1 in Iteration 1 and 2, and 0.9999 in Iteration 3. Its feature importance scores provide the quantitative basis for the feature stability analysis conducted in Section 4.5.

However, its ensemble structure requires aggregating predictions across 100 to 200 decision trees, reducing auditability and increasing the memory footprint relative to the single Decision Tree. For deployments where maximum accuracy is the primary requirement and interpretability is secondary, Random Forest is a suitable choice.

Logistic Regression as a Lightweight Linear Baseline

Logistic Regression demonstrated the greatest improvement observed across the three iterations in the most challenging evaluation direction, improving recall from 0.9475 in Iteration 1 to 0.9675 in Iteration 3 in the URLhaus to PhishTank cross-dataset direction. Its final tuned configuration with C=100.0 for PhishTank training and C=10.0 for URLhaus cross-evaluation training provides a strong linear baseline that is the fastest

and most memory-efficient classifier among the three. It is well-suited for deployment scenarios with severe computational constraints where the slight reduction in cross-dataset recall relative to tree-based models is an acceptable trade-off for inference speed and model simplicity.

Deployed Inference Configuration: Majority-Vote Ensemble

The deployed inference configuration uses a majority-vote ensemble (i.e., combining predictions from multiple models) consisting of Logistic Regression, Random Forest, and Decision Tree. Each classifier produces an independent prediction, and the final classification is determined by majority agreement across the three outputs.

This ensemble design combines the complementary strengths of each model, the nonlinear pattern recognition of Random Forest, the auditability of Decision Tree, and the linear baseline efficiency of Logistic Regression, which may reduce dependence on any single classifier's assumptions or training distribution sensitivity. Individual model outputs and confidence scores remain accessible, preserving interpretability for security analysts who require visibility into the classification basis.

4.6.2 Final Locked Configuration

The following configuration is locked for the prototype demonstration and reproducibility:

- Feature set: 13 features from the Iteration 2 reduced set
  - path_length
  - num_slashes
  - url_length
  - num_dots
  - subdomain_depth
  - has_https
  - num_digits
  - url_entropy
  - digit_ratio
  - hostname_length
  - special_char_ratio
  - hostname_entropy
  - has_ip
- Preprocessing: StandardScaler fitted on training data only, applied via transform to test and inference data. Tree-based models used unscaled features.
- Random seed: random_state=42 applied to all models, all train-test splits, and all cross-validation folds throughout the pipeline.
- Pipeline scripts: the eight-script single-responsibility pipeline (described in Chapter 3) with predict.py verified as operational for single-URL inference using the trained model artifacts stored in the outputs/ directory.

Table 14 *Final Locked Hyperparameter Configurations*

Note. Data sourced from best_params.csv in the outputs/ directory.

Model	Dataset Context	Final Parameters
Logistic Regression	PhishTank training	C=100.0, max_iter=1000, class_weight=balanced
Logistic Regression	URLhaus training/cross-eval	C=10.0, max_iter=1000, class_weight=balanced
Random Forest	PhishTank training	n_estimators=100, max_depth=10, class_weight=balanced
Random Forest	URLhaus training	n_estimators=100, max_depth=None, class_weight=balanced
Decision Tree	PhishTank training	max_depth=15, min_samples_split=30, class_weight=balanced
Decision Tree	URLhaus training	max_depth=5, min_samples_split=10, class_weight=balanced

4.6.3 Reproducibility Confirmation

The deterministic nature of the pipeline was confirmed by running both the Iteration 2 feature selection evaluation and the Iteration 3 hyperparameter tuning evaluation twice, each under identical input conditions. Both runs produced identical results across all metrics, feature importance scores, selected feature lists, and best hyperparameter configurations, confirming that the fixed random seed strategy applied throughout the pipeline supports reproducibility. This confirmation addresses the reproducibility requiremetns described in the SRS document and supports the project's claim that the experimental results are verifiable and independently replicable by future researchers.

4.7 Interpretation of Results

The three-iteration experimental record produces a coherent and interpretable set of findings that collectively address all five research questions defined in Chapter 1. This section synthesizes the results across all evaluation dimensions and explains what the observed patterns reveal about the behavior of lightweight, offline malicious URL detection classifiers operating under URL-string-only feature constraints.

4.7.1 Research Question 1: Viability of Offline URL-String-Only Detection

The results provide strong evidence supporting the first research question. All three classifiers achieved F1-scores above 0.997 on both datasets within matched training and test distributions using 24 features derived exclusively from URL strings without any DNS queries, WHOIS lookups, HTTP requests, or external API calls. The URLhaus dataset produced near-perfect performance with the Decision Tree achieving perfect 1.0000 F1 in Iteration 1. Even under the strictest evaluation condition, cross-dataset generalization in the harder URLhaus to PhishTank direction, the Decision Tree still maintained 0.9904 F1 across all iterations. These results confirm that URL-string-only features are sufficient to support strong malicious URL detection performance in a fully

offline, privacy-preserving environment, establishing a viable baseline for deployment in constrained contexts where external lookups are not feasible.

4.7.2 Research Question 2: Feature Predictive Stability

The feature stability analysis in Section 4.5 identifies a consistent pattern of predictive importance across the two threat intelligence sources. Five features demonstrate strong and consistent importance across both datasets: path_length, num_slashes, url_length, num_dots, and subdomain_depth. Together these features account for the majority of predictive weight in both training contexts and represent a core set of generalizable feature set for URL-string-based malicious URL detection. Eight additional features demonstrate moderate stability with meaningful contributions in both datasets but with notable asymmetries that reflect the structural differences between credential-harvesting and malware delivery URL patterns. The remaining 11 features were identified as unstable or redundant and were removed in Iteration 2 without meaningful performance impact, indicating that a parsimonious 13-feature set captures the essential discriminative information contained in the full 24-feature schema.

4.7.3 Research Question 3: Model Performance and Deployment

The three classifiers occupy distinct positions on the performance-interpretability-efficiency spectrum that have direct implications for deployment feasibility. Random Forest provides the highest accuracy and is the most resilient to dataset variation, achieving near-perfect or perfect performance in all evaluation scenarios. However, it requires aggregating predictions across 100 to 200 trees and provides limited decision auditability. Decision Tree provides fully auditable rule-based classification with competitive performance, 0.9904 F1 in the hardest generalization direction, and its best-performing hyperparameters were found to adapt naturally to dataset complexity, using deeper trees for PhishTank and shallower trees for URLhaus. Logistic Regression provides the fastest inference and smallest model footprint but shows the largest performance gap in the URLhaus to PhishTank cross-dataset direction. For deployments in offline, privacy-preserving environments where interpretability and auditability are operational requirements, the Decision Tree offers a strong trade-off between classification performance and deployment feasibility.

4.7.4 Research Question 4: Cross-Dataset Generalization Asymmetry

A key finding of this study is the directional asymmetry in cross-dataset generalization performance. Models trained on PhishTank and evaluated on URLhaus achieve near-perfect performance across all three iterations. While models trained on URLhaus and evaluated on PhishTank consistently exhibit a recall gap that iterative optimization narrowed but did not eliminate. This asymmetry is unlikely to be solely a modeling artifact and instead appears to reflect an underlying structural difference between the two threat categories. PhishTank represents credential-harvesting URLs characterized by entropy-based path obfuscation and deceptive domain construction. URLhaus represents malware delivery infrastructure characterized by IP-based hosts, numeric patterns, and subdomain complexity. The features most predictive for PhishTank, particularly url_entropy and hostname_entropy, carry near-zero importance for URLhaus, meaning URLhaus-trained models do not learn the sensitivity to entropy

signals that PhishTank test data requires. Conversely, the features most important for URLhaus, has_https, and has_ip, are learned by PhishTank-trained models in the context of their broader feature weighting, enabling more complete generalization in that direction. This finding has practical implications for the design of malicious URL detection systems: training data source selection significantly affects the breadth of threat categories a model can reliably detect, and systems trained exclusively on one threat type may exhibit systematic blind spots for structurally distinct threat categories.

4.7.5 Research Question 5: Measurable Improvements Through Iterative Optimization

The iterative methodology produced measurable and targeted improvements that directly answer research question five. Iteration 2 feature selection improved Logistic Regression recall in the URLhaus to PhishTank direction from 0.9475 to 0.9619 (+0.0144), while maintaining or marginally improving all other evaluation scenarios. Iteration 3 hyperparameter tuning further improved Logistic Regression recall in the same direction from 0.9619 to 0.9675 (+0.0056), producing a cumulative recall improvement of +0.0200 and a cumulative F1 improvement of +0.0105 across the full three-iteration process. Within-dataset performance remained stable or improved marginally across all iterations, confirming that the iterative refinements successfully targeted the generalization gap without introducing regression in controlled evaluation conditions. The iterative methodology is supported as an effective approach for systematically improving cross-dataset generalization in malicious URL detection systems operating under URL-string-only constraints.

4.8 Summary of Findings

The following key findings summarize the empirical results of the three-iteration evaluation:

- URL-string-only features support strong malicious URL detection performance in a fully offline environment. All three classifiers achieved F1-scores above 0.997 on both datasets within matched distributions, with the URLhaus Decision Tree reaching 1.0000 F1 in Iteration 1. These results indicate the viability of a privacy-preserving, offline detection framework without reliance on external data sources.
- path_length and num_slashes are the most stable and consistent predictors across both threat intelligence sources, together accounting for approximately 48.3% of mean importance weight. These features maintain their top-two ranking independently on both datasets, indicating that they are consistently weighted predictors for cross-source malicious URL detection in this study.
- A consistent asymmetry exists between the two cross-dataset evaluation directions. PhishTank to URLhaus generalization achieves near-perfect performance (F1 0.9991 to 1.0000) while URLhaus to PhishTank generalization shows a recall gap (F1 0.9730 to 0.9904 in Iteration 1). This pattern appears to reflect underlying structural differences between credential-harvesting and malware delivery URL categories rather than solely indicating model limitations.
- Feature selection in Iteration 2 showed that 11 of 24 features contributed minimally, accounting collectively for less than 2% of total predictive weight

across both datasets. Removing these features produced negligible within-dataset performance change while improving Logistic Regression cross-dataset recall by +0.0144 in the more challenging generalization direction, indicating that a reduced 13-feature set retains the essential predictive information of the full feature schema.

- Hyperparameter tuning in Iteration 3 produced its largest improvement for Logistic Regression in the URLhaus-to-PhishTank direction, further increasing recall by +0.0056. Tree-based models showed minimal change from tuning, suggesting that their performance was already near the maximum observed under the given feature set and experimental conditions.
- Across the three-iteration process, Logistic Regression improved by a cumulative +0.0200 recall and +0.0105 F1 in the most challenging evaluation direction, corresponding to approximately 200 additional correctly identified malicious URLs per 10,000 evaluated compared to the Iteration 1 baseline.
- The deployed inference component uses a majority-vote ensemble consisting of all three classifiers, integrating the interpretability of the Decision Tree, the accuracy of Random Forest, and the efficiency of Logistic Regression into a unified and classification approach.
- The pipeline demonstrates reproducible behavior under repeated execution. Independent re-execution of Iterations 2 and 3 under identical conditions produced identical results, indicating that the fixed random seed strategy, deterministic feature engineering, and stable classifier implementations support reproducibility of the experimental results.

CHAPTER 5: CONCLUSION AND DISCUSSION

5.1 Conclusion

This project set out to design, implement, and rigorously evaluate a privacy-preserving offline machine learning system for malicious URL detection that operates exclusively on features derived from the URL string, without reliance on external lookups, network queries, or runtime dependencies of any kind. The system was developed in response to a gap in existing research: while the phishing detection literature demonstrates strong performance under favorable conditions that include external data enrichment, the design space of fully offline, URL-string-only detection systems suitable for air-gapped, privacy-sensitive, and resource-constrained environments has received comparatively little systematic attention.

Experimental results across three iterative evaluation rounds demonstrate that URL-string-only features are sufficient to support high classification performance under controlled conditions. Using two structurally distinct real-world threat intelligence sources, PhishTank for credential-harvesting phishing URLs and URLhaus for malware delivery URLs, balanced against legitimate domains from the Tranco Top Sites list, all three classifiers achieved F1-scores above 0.997 within matched training and testing distributions, with Decision Tree achieving a perfect 1.0000 F1 on the URLhaus dataset in Iteration 1.

The five research questions defined in Chapter 1 were systematically addressed through the experimental methodology. URL-string-only features were shown to provide a viable foundation for offline malicious URL detection. Feature stability analysis identified path_length and num_slashes as the most consistent predictors across both datasets. Comparative model evaluation revealed clear trade-offs between performance, interpretability, and generalization. Cross-dataset evaluation identified a directional asymmetry in generalization behavior, and the iterative methodology demonstrated measurable improvement, with Logistic Regression achieving a cumulative recall improvement of +0.0200 and an F1 improvement of +0.0105 in the most challenging evaluation direction.

While individual model analysis identified the Decision Tree as the most interpretable classifier, the implemented system adopts a multi-model ensemble approach. Logistic Regression, Random Forest, and Decision Tree predictions are combined through a majority-vote mechanism, improving reliability while preserving interpretability through access to individual model outputs and confidence scores.

The system is delivered as a fully reproducible, modular, eight-script pipeline with a standalone inference component, and graphical user interface, supported by complete experimental artifacts including trained models, evaluation outputs, and feature importance rankings. Deterministic execution was confirmed through repeated runs producing identical results under identical conditions, satisfying the reproducibility requirements defined in the system design.

Overall, this project establishes a rigorous and reproducible baseline for evaluating lightweight machine learning under strict offline and privacy-preserving constraints,

while demonstrating the practical value of ensemble-based approaches for improving reliability in constrained deployment environments.

5.2 Discussion

The results of this project provide several insights into malicious URL detection under constrained environments, the effects of dataset variation, and the implications for real-world deployment.

5.2.1 The Viability and Limitations of URL-String-Only Detection

The strong within-dataset performance achieved across all three classifiers confirms that lexical and structural URL features contain substantial predictive information for both phishing and malware delivery detection. Achieving F1-scores above 0.997 using only 24 computationally inexpensive features, without DNS resolution, WHOIS lookups, or content analysis, demonstrates that strict offline constraints do not inherently prevent high classification performance.

However, cross-dataset evaluation introduces an important limitation. While generalization from PhishTank to URLhaus remained strong, the reverse direction consistently exhibited a recall gap ranging from approximately 1.9% to 5.25% in the baseline iteration.

This asymmetry has important implications for deployment, particularly in how models generalize across distinct threat distributions. Models trained on a single threat source may not fully capture the structural characteristics of other threat types, resulting in reduced detection performance under distribution shift.

These findings suggest that a model's effectiveness is inherently constrained by the diversity of tis training data. Single-source training may introduce systematic detection blind spots, reinforcing the importance of evaluating training data composition prior to deployment.

5.2.2 The Value of the Iterative Experimental Methodology

The three-iteration experimental framework proved effective for isolating and understanding performance drivers. By ensuring that each iteration modified only a single variable (either the feature set or the hyperparameters), this methodology enabled clear attribution of performance changes.

For example, Logistic Regression improved from 0.9475 recall in Iteration 1 to 0.9675 in Iteration 3 in the most challenging generalization direction. This improvement would not have been interpretable without controlled experimental isolation.

Equally important is the observed performance convergence of tree-based models, particularly Decision Tree and Random Forest. Their stability across iterations indicates that they had reached the highest performance achievable under the given feature constraints and dataset conditions. This suggests that further improvements require changes to feature representation or data composition rather than continued tuning.

5.2.3 Model Selection Trade-offs for Deployment

The three evaluated classifiers represent distinct trade-offs:

- Random Forest
  - Strongest overall performance and generalization stability
  - Higher complexity and reduced transparency
- Decision Tree
  - Fully interpretable and auditable rule-based decisions
  - Competitive performance across evaluation scenarios
- Logistic Regression
  - Lightweight, fast, and efficient
  - More sensitive to dataset variation

Rather than selecting a single model for deployment, the implemented system addresses these trade-offs through a majority-vote ensemble approach, where all three classifiers contribute equally to the final prediction.

This design combines the complementary strengths of each model:

- Random Forest improves modeling of complex feature interactions through nonlinear pattern recognition
- Decision Tree provides transparency and interpretability
- Logistic Regression contributes efficiency and a linear baseline perspective

Importantly, interpretability is preserved through access to individual model outputs enabling analysts to inspect and validate classification decisions.

5.2.4 Relationship to Existing Literature

The findings of this study align with and extend prior research. The strong within-dataset performance is consistent with established work demonstrating the effectiveness of lexical URL features in phishing detection. The observed cross-dataset performance degradation reinforces the dataset shift problem widely discussed in machine learning literature.

This project extends existing work by:

- Providing empirical evidence of directional generalization asymmetry
- Demonstrating feature stability as a key factor in transferability
- Establishing a fully offline evaluation framework, which remains underexplored despite its practical importance

By explicitly excluding external data enrichment, this research contributes a complementary perspective focused on constrained deployment scenarios, rather than maximum achievable accuracy.

5.3 Future Work

While this project establishes a rigorous baseline for offline malicious URL detection, it also identifies several important areas where future research can extend and strengthen the findings.

5.3.1 Feature Representation and Generalization Enhancement

The final system employs a reduced 13-feature set, derived through cross-dataset feature importance analysis, which demonstrated that a smaller, more focused feature space can maintain strong performance while improving generalization behavior. This result indicates that feature quality and transferability are more critical than feature quantity in constrained detection systems.

Future work should therefore focus on refining and extending feature representation, rather than expanding it indiscriminately. Potential directions include exploring more expressive representations such as character-level n-grams, token-based segmentation, or enhanced entropy-based measures that capture subtle structural relationships within URL strings.

In parallel, the cross-dataset generalization gap observed in this study suggests the need for targeted improvements in feature transferability. Further refinement of feature selection techniques, or incorporation of lightweight domain adaptation strategies, may help improve performance when models are applied to structurally different URL datasets while maintaining the system's offline and privacy-preserving constraints.

5.3.2 Multi-Source Training and Ensemble Optimization

This study focused on single-source training to isolate generalization behavior; however, training on combined datasets from multiple threat intelligence sources represents a natural extension. Such an approach may improve detection coverage and reduce blind spots associated with single-source models, although potential trade-offs between specialization and generalization should be evaluated.

Additionally, while the current system employs a majority-vote ensemble, future work should explore alternative ensemble strategies such as weighted voting or confidence-based aggregation. These approaches may further improve classification performance by dynamically leveraging the strengths of individual models while maintaining the system's offline and privacy-preserving constraints.

5.3.3 Temporal Evaluation and Practical Deployment

All datasets used in this study represent static snapshots of malicious URL distributions. Future work should evaluate model performance over time to assess how detection capability evolves as attacker techniques change. Understanding temporal generalization is critical for determining long-term deployment viability.

Finally, the prototype system should be evaluated in practical deployment scenarios. This includes benchmarking performance on constrained hardware, testing integration into environments such as browser-based or email filtering systems, and assessing usability for security analysts. These steps would help validate the system's operational effectiveness and applicability in real-world constrained environments.

REFERENCES

[1] FBI National Press Office, "FBI Releases Annual Internet Crime Report, Federal Bureau of Investigation," *Federal Bureau of Investigation*, Apr. 23, 2025. https://www.fbi.gov/news/press-releases/fbi-releases-annual-internet-crime-report

[2] "2025 Data Breach Investigations Report Executive Summary," Available: https://www.verizon.com/business/resources/reports/2025-dbir-executive-summary.pdf

[3] B. B. Gupta, K. Yadav, I. Razzak, K. Psannis, A. Castiglione, and X. Chang, "A novel approach for phishing URLs detection using lexical based machine learning in a real-time environment," *Computer Communications,* vol 175, pp. 47-57, Jul. 2021 doi: https://doi.org/10.1016/j.comcom.2021.04.023

[4] Cisco Talos Intelligence Group (2024). *PhishTank out of the Net into the Tank.* https://phishtank.org/developer_info.php

[5] "URLhaus, Malware URL exchange," *urlhaus.abuse.ch*. https://urlhaus.abuse.ch/

[6] "A research-oriented top sites ranking hardened against manipulation – Tranco," *tranco-list.eu*. https://tranco-list.eu/

[7] D. Sahoo, C. Liu, and Steven, "Malicious URL Detection using Machine Learning: A Survey," arXiv (Cornell University), Jan. 2017, doi: https://doi.org/10.48550/arXiv.1701.07179

[8] C. Catal, G. Giray, B. Tekinerdogan, S. Kumar, and S. Shukla, "Applications of deep learning for phishing detection: a systematic literature review," *Knowledge and Information Systems,* vol 64, no. 6, pp. 1457-1500, May 2022, doi: https://doi.org/10.1007/s10115-022-01672-x

[9] K. Haynes, H. Shirazi, and I. Ray, "Lightweight URL-based phishing detection using natural language processing transformers for mobile devices," *Procedia Computer Science,* vol. 191, pp. 127-134, 2021, doi: https://doi.org/10.1016/j.procs.2021.07.040

[10] F. Rashid, B. Doyle, S.C. Han, and S. Seneviratne, "Phishing URL detection generalisation using Unsupervised Domain Adaptation," *Computer Networks,* vol. 245, pp. 110398-110398, May 2024, doi: https://doi.org/10.1016/j.comnet.2024.110398

[11] R. Verma, and A. Das, "What's in a URL," *Proceedings of the 3rd ACM on International Workshop on Security and Privacy Analytics,* Mar. 2017, doi: https://doi.org/10.1145/3041008.3041016.

[12] N. F. Almujahid, M.A. Haq, and M. Alshehri, "Comparative evaluation of machine learning algorithms for phishing site detection," *Peer J Computer Science,* vol 10, pp. e2131-e2131, Jun. 2024, doi: https://doi.org/10.7717/peerj-cs.2131

[13] E.W. Dijkstra, "On the role of scientific through," 1974.

[14] R.C. Martin, *Agile software development: principles, patterns, and practices.* Upper Saddle River, N.J.: Prentice Hall, 2003.

[15] J. Wang, T. Kuo, L. Li, and A. Zeller, "Assessing and Restoring Reproducibility of Jupyter Notebooks," *2020 35th IEEE/ACM International Conference on Automated Software Engineering*, Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9286024

[16] C. Huyen, "*Designing Machine Learning Systems,"* Sebastopol, CA, USA: O'Reilly Media, 2022.

[17] T. M. Cover, and J.A. Thomas, "*Elements of information theory (2nd ed.),"* Wiley Publishers, 2006.

[18] M. Hussain, C. Cheng, R. Xu, and M. Afzal, "CNN-Fusion: An effective and lightweight phishing detection method based on multi-variant ConvNet*,"Information Sciences,* Feb. 2023, doi: https://doi.org/10.1016/j.ins.2023.02.039

[19] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research,* vol 12, no. 12, pp. 2825-2830, 2011, Available: https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf

[20] L. Bustio-Martinez, M.A. Alvarez-Carmona, V. Herrera-Semenets, C. Feregrino-Uribe, and R. Cumplido, "A lightweight data representation for phishing URLs detection in IoT environments," *Information Sciences,* vol. 603, pp. 42-59, Jul. 2022, doi: https://doi.org/10.1016/j.ins.2022.04.059

[21] P. Dhingra, "CTAL-TA Quality Characteristics: ISO 25010, Usability, Reliability, and Maintainability Testing," *Master Software Testing*, Sep. 28, 2025. https://mastersoftwaretesting.com/certification-guides/istqb/ctal-ta/ctal-ta-quality-characteristics#iso-25010-quality-model

[22] IBM, "What is ETL (extract, transform, load)?," *IBM*, Oct. 04, 2021. https://www.ibm.com/think/topics/etl

APPENDIX I: AI Use

This appendix

	Tool/Version	Query/Prompt	Output Summary	How It Was Used
1	OpenAI ChatGPT (GPT-5.3, 2026)	“Please proofread the pasted section only for grammar, punctuation, and sentence clarity at a master’s academic level standard, please format your response in bullet points thank you” (chapter 1)	Suggested corrections to capitalization of source, parallel structure in lists, avoid redundant phrasing, grammar, typos, and citation consistency.	Reviewed all suggestions independently. Accepted only minor grammatical corrections that did not alter my original works meaning. Corrected typos and capitalization.
2	OpenAI ChatGPT (GPT-5.3, 2026)	“Please proofread the pasted section only for grammar, punctuation, and sentence clarity at a master’s academic level standard, please format your response in bullet points thank you” (chapter 2)	Suggested corrections for grammatical parallelism, clarity improvement, typos, grammar, and hyphenations.	Reviewed all suggestions independently. Accepted minor grammatical corrections and phrasing that did not change my original works meaning.

				Corrected typos.
3	OpenAI ChatGPT (GPT-5.3, 2026)	“Please proofread the pasted section only for grammar, punctuation, and sentence clarity at a master’s academic level standard, please format your response in bullet points thank you” (chapter 3)	Suggested corrections for minor typos, article usage, sentence fragments, and consistency (hyphenation, capitalization).	Reviewed all suggestions independently. Accepted minor grammatical corrections, corrected typos, and updated a few phrasing issues that did not change my original works meaning.
4	OpenAI ChatGPT (GPT-5.3, 2026)	“Please proofread the pasted section only for grammar, punctuation, and sentence clarity at a master’s academic level standard, please format your response in bullet points thank you” (chapter 4)	Suggested correction for punctuation, typos, grammar corrections, and improved sentence clarity.	Reviewed all suggestions independently. Accepted minor sentence clarity improvements, grammatical corrections, and typos, accepting changes that did not change

				my original works meaning.
5	OpenAI ChatGPT (GPT-5.3, 2026)	“Please proofread the pasted section only for grammar, punctuation, and sentence clarity at a master’s academic level standard, please format your response in bullet points thank you” (chapter 5)	Suggested corrections for punctuation, sentence clarity, grammar corrections, typos, and improved readability.	Reviewed all suggestions independently. Accepted minor sentence clarity improvements, grammatical corrections, and typos, accepting changes that did not change my original works meaning.

APPENDIX II: System Pipeline and Code

This appendix contains IDE screenshots and code captures demonstrating the implemented pipeline modules, iteration evaluation scripts, and reproducible model training configuration used in the experiments presented in Chapters 3 and 4. All scripts are implemented in Python 3.11.9 within an isolated virtual environment and are organized as a modular, eight-script pipeline. Each subsection provides a representative code segment highlighting the core functionality of the corresponding module, including data ingestion, preprocessing, feature extraction, model training, cross-dataset evaluation, and inference. These materials are intended to support transparency, reproducibility, and independent verification of the system design and experimental results.

A.1 Dataset Conversion Module (convert_urlhaus.py)

Section Description:

This module parses the PhishTank XML dataset and converts verified phishing URLs into a structured CSV file containing url and label fields for downstream pipeline processing.

```
53      try:
54          tree = ET.parse(input_path) # try to parse the XML file into an Element
55          root = tree.getroot() # get the root element of the XML tree, which is
56      except ET.ParseError as e:  # catch parsing errors that occur if the XML fi
57          log.error("Could not parse XML file: %s", e) # if there's a parsing err
58          log.error("The file may be corrupted. Try downloading it again.") # com
59          sys.exit(1) # exit with error code 1 to indicate failure
60
61      log.info("XML loaded successfully. Extracting URLs...") # log that we've su
62
63      records = [] # this will hold the extracted URL records as dictionaries wit
64
65      for entry in root.iter("entry"): # PhishTank XML uses <entry> tage, iterate
```

Figure 10 XML Parsing
Parses the PhishTank XML input file using ElementTree, extracts the XML root element, handles malformed XML files, initializes the records list, and begins iterating through each <entry> element.

```
67          # Extracting URL
68          url_el = entry.find("url") # first try to
69          if url_el is None: # if <url> tag is not f
70              url_el = entry.find("phish_url") # if
71          if url_el is None or not url_el.text: # if
72              continue # if there's no URL found in
73
74          url = url_el.text.strip() # if we found a
75          if not url: # if the URL is empty after st
76              continue # if the URL is empty, we can
```

Figure 11 URL Extraction
Extracts the URL from either the <url> tag or the fallback <phish_url> tag, validates that the field is not empty, removes surrounding whitespace, and prepares the URL for record creation.

```
78          # Only keep verified entries for data quality
79          verified_el = entry.find("verified") # check if there's a
80          if verified_el is not None: # if the <verified> tag exists
81              if str(verified_el.text).strip().lower() != "yes": # i
82                  continue # if the entry is not verified, we skip i
83
84          records.append({"url": url, "label": 1}) # if we have a va
85
```

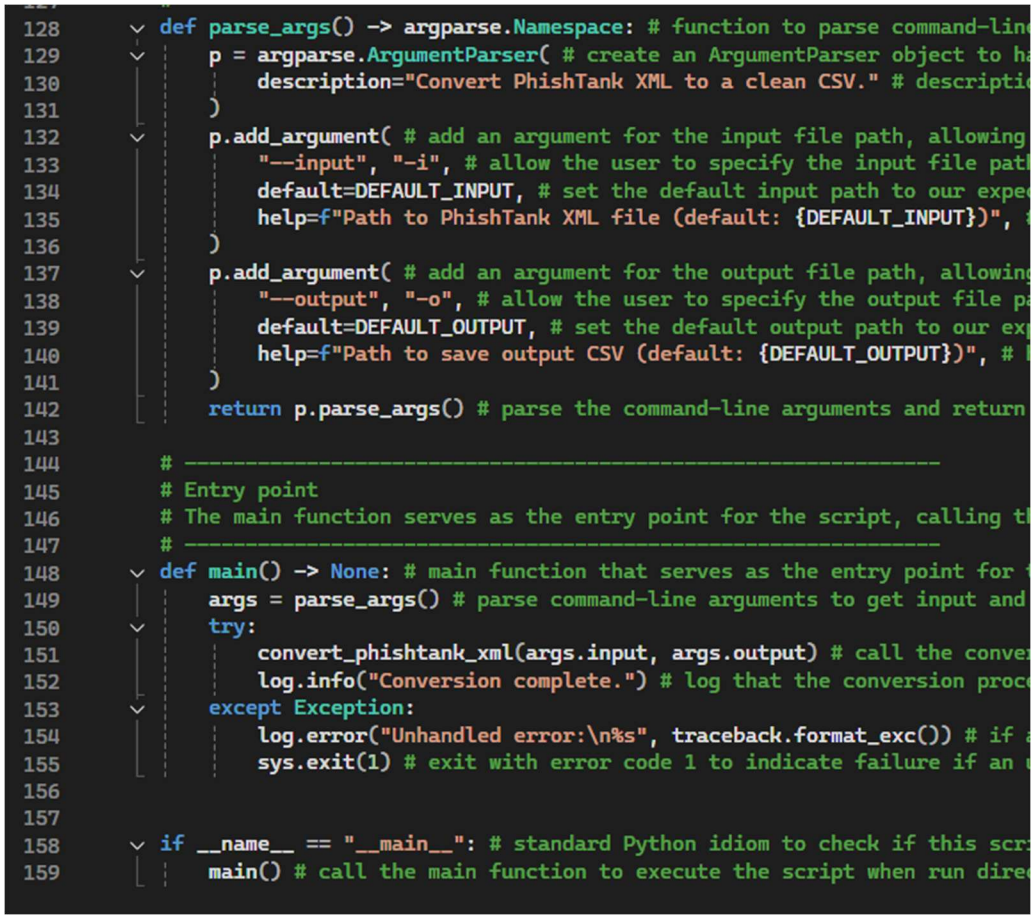
Figure 12 Verification Filter
Checks whether a PhishTank entry includes a <verified> tag and excludes entries that are not marked as verified, improving dataset quality before model training.

```
105     df = pd.DataFrame(records) # convert the list of records (diction
106
107     # Remove duplicates
108     before = len(df) # store the number of records before removing du
109     df = df.drop_duplicates(subset=["url"]).reset_index(drop=True) #
110     log.info(
111         "Removed %d duplicate URLs. Final count: %d",
112         before - len(df), len(df) # log how many duplicates were remo
113     )
```

Figure 13 DataFrame and Deduplication
Converts the extracted URL records into a pandas DataFrame, removes duplicate URLs, and resets the index so the output dataset contains only unique phishing URL entries.

```
115     os.makedirs(os.path.dirname(output_path) or ".", exist_ok=True) # en
116     df.to_csv(output_path, index=False) # save the DataFrame to a CSV fi
```

Figure 14 CSV Output
Ensures that the output directory exists and writes the cleaned PhishTank URL dataset to a CSV file without including an index column.



```
def parse_args() -> argparse.Namespace: # function to parse command-lin
    p = argparse.ArgumentParser( # create an ArgumentParser object to h
        description="Convert PhishTank XML to a clean CSV." # descripti
    )
    p.add_argument( # add an argument for the input file path, allowing
        "--input", "-i", # allow the user to specify the input file pat
        default=DEFAULT_INPUT, # set the default input path to our expe
        help=f"Path to PhishTank XML file (default: {DEFAULT_INPUT})",
    )
    p.add_argument( # add an argument for the output file path, allowin
        "--output", "-o", # allow the user to specify the output file p
        default=DEFAULT_OUTPUT, # set the default output path to our ex
        help=f"Path to save output CSV (default: {DEFAULT_OUTPUT})", #
    )
    return p.parse_args() # parse the command-line arguments and return

# ------------------------------------------------------------
# Entry point
# The main function serves as the entry point for the script, calling t
# ------------------------------------------------------------
def main() -> None: # main function that serves as the entry point for
    args = parse_args() # parse command-line arguments to get input and
    try:
        convert_phishtank_xml(args.input, args.output) # call the conve
        log.info("Conversion complete.") # log that the conversion proc
    except Exception:
        log.error("Unhandled error:\n%s", traceback.format_exc()) # if
        sys.exit(1) # exit with error code 1 to indicate failure if an


if __name__ == "__main__": # standard Python idiom to check if this scr
    main() # call the main function to execute the script when run dire
```

Figure 15 CLI and Entry Point
Defines the command-line interface for convert_phishtank.py, allowing the user to specify input and output file paths. It also provides the script entry point, which parses arguments, calls the conversion function, and handles unexpected execution errors.

A.2 Dataset Conversion Module (convert_urlhaus.py)

Section Description:

This module converts the URLhaus abuse.ch raw text download into a structured CSV file containing URL and label fields. It skips comment lines, handles encoding issues, extracts the URL column, removes invalid or duplicate entries, assigns malicious labels, and saves the cleaned dataset for downstream pipeline processing.

```
57      # Expected column names after skipping comment lines
58      URLHAUS_COLUMNS = [
59          "id", "dateadded", "url", "url_status", "last_online",
60          "threat", "tags", "urlhaus_link", "reporter",
61      ]
62
```

Figure 16 URLhaus Column Schema
Defines the expected URLhaus input columns after comment lines are skipped. The schema allows the script to correctly identify and extract the URL field from the raw URLhaus data.

```
84      try:
85          df = pd.read_csv( # read the
86              input_path, # path to the
87              comment="#", # skip lines
88              header=None, # no header
89              names=URLHAUS_COLUMNS, #
90              on_bad_lines="skip", # sk
91              encoding="utf-8", # try r
92          )
```

Figure 17 File Parsing
Reads the URLhaus text file with pandas, skips comment lines beginning with #, assigns the expected column names, skips, malformed rows, and uses UTF-8 encoding as the default input format.

```
93      except UnicodeDecodeError: # if utf-8 decoding fails,
94          df = pd.read_csv( # read the file again with lat
95              input_path, # same file path
96              comment="#", # same options for skipping com
97              header=None, # same as before, no header row
98              names=URLHAUS_COLUMNS, # same column names
99              on_bad_lines="skip", # same option to skip b
100             encoding="latin-1", # fallback encoding that
101         )
102         log.warning("Used latin-1 encoding fallback.") #
```

Figure 18 Encoding Fallback
Provides fallback parsing path when UTF-8 decoding fails. If a UnicodeDecodeError occurs, the script reloads the URLhaus files using Latin-1 encoding to prevent the conversion process from failing due to byte-sequence incompatibilities.

```
114         df = df[["url"]].dropna().copy() # select only
115         df["url"] = df["url"].astype(str).str.strip() #
116         df = df[df["url"] != ""] # remove any rows wher
117
```

Figure 19 URL Cleaning

Keeps only the URL column, removes missing URL values, converts URLs to strings, strips leading and trailing whitespace, and removes empty URL entries.

```
119         before = len(df) # store the number of rows before removing dupli
120         df = df.drop_duplicates(subset=["url"]).reset_index(drop=True) #
121         log.info(
122             "Removed %d duplicate URLs. Unique URLs remaining: %d",
123             before - len(df), len(df) # log how many duplicates were remo
124         )
125
126         df["label"] = 1 # add a new column 'label' and set it to 1 for al
127
```

Figure 20 Deduplicating and Labeling

Removes duplicate URLs, resets the DataFrame index, and assigns the malicious class label 1 to every retained URL because URLhaus entries are treated as malicious source data.

```
129         os.makedirs(os.path.dirname(output_path) or ".", exist_ok=True) # e
130         df.to_csv(output_path, index=False) # save the DataFrame to a CSV f
131
```

Figure 21 CSV Output

Ensures output directory exists and writes the cleaned URLhaus dataset to a CSV file without including the DataFrame index.

```python
def parse_args() -> argparse.Namespace: # function to parse command-lin
    p = argparse.ArgumentParser( # create an ArgumentParser object to h
        description=( # description of what this script does, shown whe
            "Convert URLhaus abuse.ch .txt download to a clean "
            "url + label CSV ready for build_dataset.py."
        )
    )
    p.add_argument( # add an argument for the input file path, allowing
        "--input", "-i", # allow the user to specify the input file pat
        default=DEFAULT_INPUT, # set the default input path to our expe
        help=f"Path to URLhaus .txt file (default: {DEFAULT_INPUT})", #
    )
    p.add_argument( # add an argument for the output file path, allowin
        "--output", "-o", # allow the user to specify the output file p
        default=DEFAULT_OUTPUT, # set the default output path to our ex
        help=f"Output CSV path (default: {DEFAULT_OUTPUT})", # help mes
    )
    return p.parse_args() # parse the command-line arguments and return

# ---------------------------------------------------------------------
# Entry point
# ---------------------------------------------------------------------
if __name__ == "__main__": # main function that servces as the entry po
    try:
        args = parse_args() # parse command-line arguments to get input
        convert(args.input, args.output) # call the conversion function
    except Exception:
        log.error("Unhandled error:\n%s", traceback.format_exc()) # if
        sys.exit(1) # exit with error code 1 to indicate failure if an

```

Figure 22 CLI and Entry Point
Defines the command-line interface for the script, allowing users to specify input and output file paths, and provides the standard Python entry point for direct execution.

A.3 Dataset Construction Module (build_dataset.py)

Section Description:

This module combines malicious URL datasets with legitimate URLs from the Tranco Top Sites list to create balanced datasets for model training. It applies class labeling, enforces equal class distribution, performs deterministic shuffling, and outputs finalized datasets for downstream processing.

```python
def load_phishtank(path: str) -> pd.DataFrame: # load the PhishTank C
    """Load the PhishTank CSV produced by convert_phishtank.py."""
    if not os.path.exists(path): # check if the file exists at the gi
        log.error("PhishTank file not found: %s", path) # log an erro
        log.error("Run convert_phishtank.py first.") # log a hint to
        sys.exit(1) # exit the program with an error code since we ca

    df = pd.read_csv(path, low_memory=False) # read the CSV file into

    if "url" not in df.columns: # check if the expected 'url' column
        log.error( # log an error if the 'url' column is missing, sho
            "No 'url' column in %s. Columns found: %s", # log the pat
            path, list(df.columns) # convert the columns to a list fo
        )
        sys.exit(1) # exit the program with an error code since we ca

    df = df[["url"]].dropna().drop_duplicates() # keep only the 'url'
    df["label"] = 1 # add a 'label' column and set it to 1 for all ro
    log.info("PhishTank  : loaded %d phishing URLs", len(df)) # log h
    return df.reset_index(drop=True) # reset the index of the DataFra

```

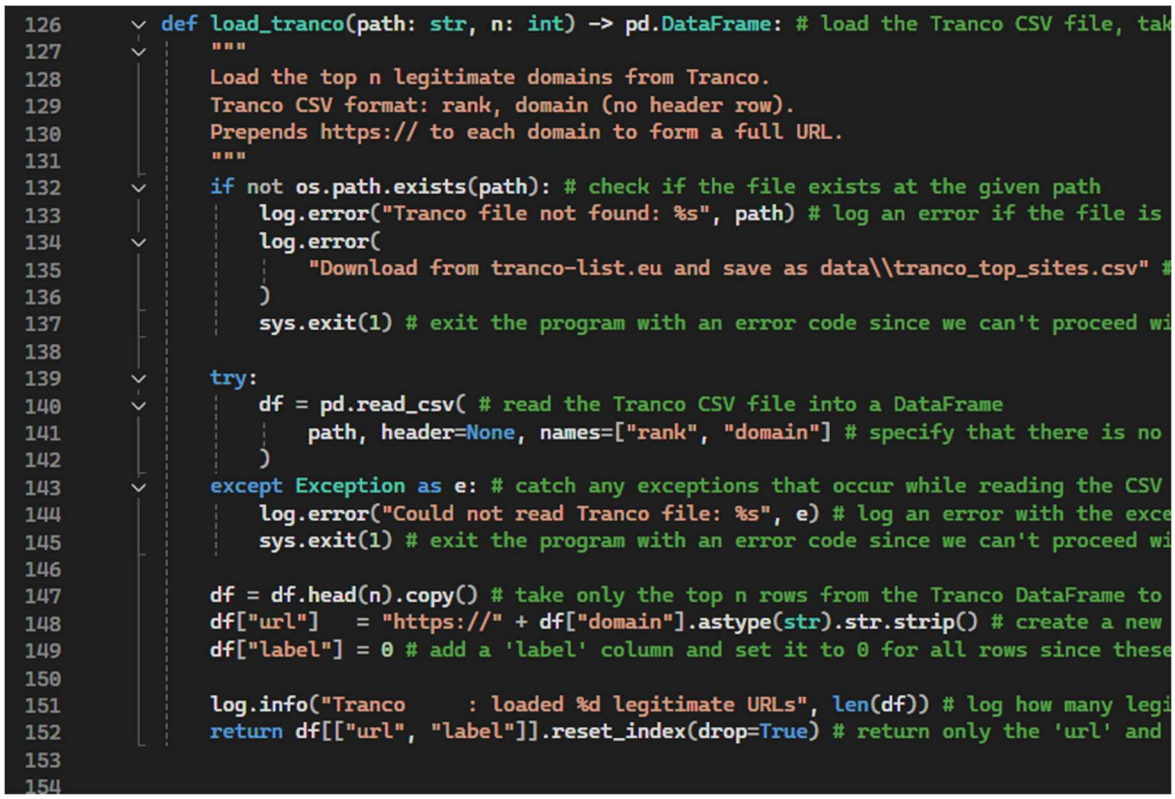
Figure 23 PhishTank Loading

Loads the PhishTank CSV produced by convert_phishtank.py, verifies that the required URL column exists, removes missing and deduplicate URLs, and assigns label 1 to mark the records as malicious.

```python
def load_urlhaus(path: str) -> pd.DataFrame: # load the URLhaus CSV file
    """
    Load the URLhaus CSV produced by convert_urlhaus.py.
    Expected columns: url, label (all 1s for malicious).
    Run convert_urlhaus.py first to generate this file.
    """
    if not os.path.exists(path): # check if the file exists at the given
        log.error("URLhaus CSV not found: %s", path) # log an error if th
        log.error("Run convert_urlhaus.py first to generate this file.")
        sys.exit(1) # exit the program with an error code since we can't

    df = pd.read_csv(path, low_memory=False) # read the CSV file into a D

    if "url" not in df.columns: # check if the expected 'url' column is p
        log.error( # log an error if the 'url' column is missing, showing
            "No 'url' column in %s. Columns found: %s", # log the path of
            path, list(df.columns) # convert the columns to a list for ea
        )
        sys.exit(1) # exit the program with an error code since we can't

    df = df[["url"]].dropna().drop_duplicates() # keep only the 'url' col
    df["label"] = 1 # add a 'label' column and set it to 1 for all rows s

    log.info("URLhaus    : loaded %d malicious URLs", len(df)) # log how
    return df.reset_index(drop=True) # reset the index of the DataFrame a

```

Figure 24 URLhaus Loading

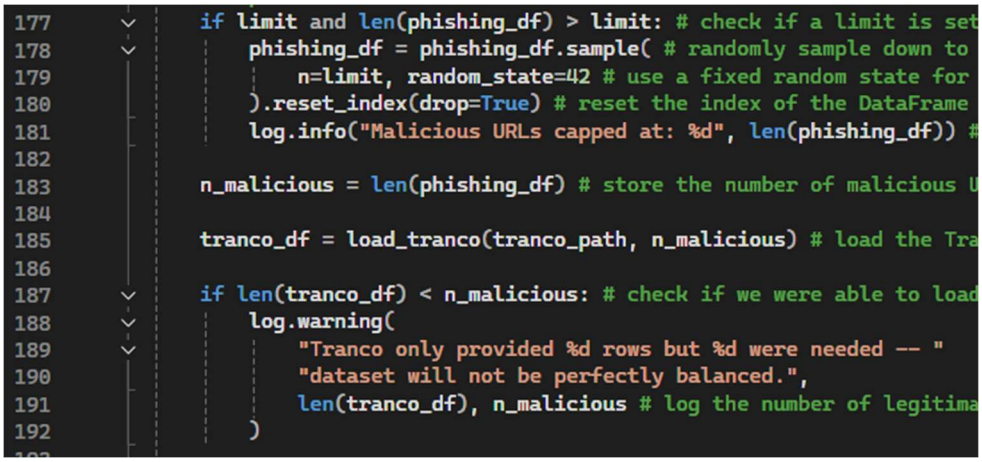
Loads the URLhaus CSV produced by convert_urlhaus.py, verifies the required URL column, removes missing and duplicate URL entries, and assigns label 1 because URLhaus is used as a malicious source.



```
126     def load_tranco(path: str, n: int) -> pd.DataFrame: # load the Tranco CSV file, tak
127         """
128         Load the top n legitimate domains from Tranco.
129         Tranco CSV format: rank, domain (no header row).
130         Prepends https:// to each domain to form a full URL.
131         """
132         if not os.path.exists(path): # check if the file exists at the given path
133             log.error("Tranco file not found: %s", path) # log an error if the file is
134             log.error(
135                 "Download from tranco-list.eu and save as data\\tranco_top_sites.csv" #
136             )
137             sys.exit(1) # exit the program with an error code since we can't proceed wi
138 
139         try:
140             df = pd.read_csv( # read the Tranco CSV file into a DataFrame
141                 path, header=None, names=["rank", "domain"] # specify that there is no
142             )
143         except Exception as e: # catch any exceptions that occur while reading the CSV
144             log.error("Could not read Tranco file: %s", e) # log an error with the exce
145             sys.exit(1) # exit the program with an error code since we can't proceed wi
146 
147         df = df.head(n).copy() # take only the top n rows from the Tranco DataFrame to
148         df["url"]   = "https://" + df["domain"].astype(str).str.strip() # create a new
149         df["label"] = 0 # add a 'label' column and set it to 0 for all rows since these
150 
151         log.info("Tranco     : loaded %d legitimate URLs", len(df)) # log how many legi
152         return df[["url", "label"]].reset_index(drop=True) # return only the 'url' and
153 
154 
```

Figure 25 Tranco Loading

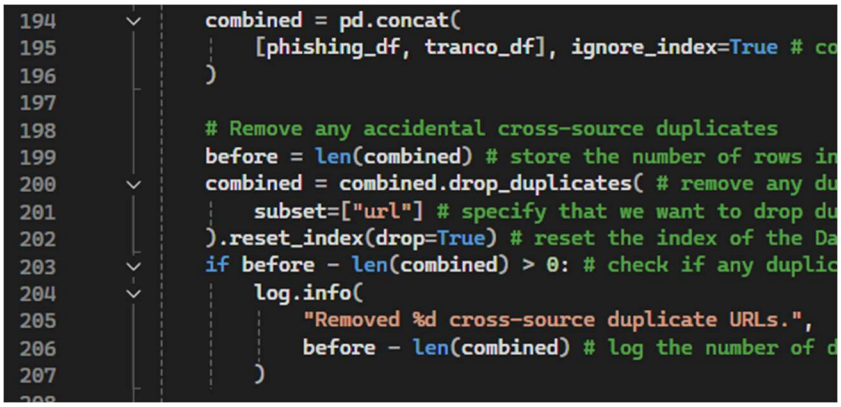
Loads the top n legitimate domains from the Tranco dataset, converts each domain into a URL by adding https://, assigns label 0, and returns only the URL and label column.



```
    if limit and len(phishing_df) > limit: # check if a limit is set
        phishing_df = phishing_df.sample( # randomly sample down to
            n=limit, random_state=42 # use a fixed random state for
        ).reset_index(drop=True) # reset the index of the DataFrame
        log.info("Malicious URLs capped at: %d", len(phishing_df)) #

    n_malicious = len(phishing_df) # store the number of malicious U

    tranco_df = load_tranco(tranco_path, n_malicious) # load the Tra

    if len(tranco_df) < n_malicious: # check if we were able to load
        log.warning(
            "Tranco only provided %d rows but %d were needed -- "
            "dataset will not be perfectly balanced.",
            len(tranco_df), n_malicious # log the number of legitima
        )
```

Figure 26 Class Balancing

Caps the malicious source at the configured limit using random_state=42, counts the retained malicious URLs, and loads the same number of legitimate Tranco URLs to create a balanced dataset.



```
    combined = pd.concat(
        [phishing_df, tranco_df], ignore_index=True # co
    )

    # Remove any accidental cross-source duplicates
    before = len(combined) # store the number of rows in
    combined = combined.drop_duplicates( # remove any du
        subset=["url"] # specify that we want to drop du
    ).reset_index(drop=True) # reset the index of the Da
    if before - len(combined) > 0: # check if any duplic
        log.info(
            "Removed %d cross-source duplicate URLs.",
            before - len(combined) # log the number of d
        )
```

Figure 27 Dataset Combination

Combines malicious and legitimate DataFrame into one dataset, removes accidental duplicate URLs across sources, and resets the index after deduplication.

```
210     combined = combined.sample(
211         frac=1, random_state=42 # shuffle the rows of the combined Data
212     ).reset_index(drop=True) # reset the index of the DataFrame after s
213
214     # Summary
215     n_mal   = int((combined["label"] == 1).sum()) # count how many mali
216     n_legit = int((combined["label"] == 0).sum()) # count how many legi
217     pct     = 100 * n_mal / len(combined) if len(combined) > 0 else 0 #
218
219     log.info("-" * 60) # log a separator line for better readability in
220     log.info("Dataset : %s", source_name) # log the name of the dataset
221     log.info("  Total rows  : %d", len(combined)) # log the total numbe
222     log.info("  Malicious   : %d  (%.1f%%)", n_mal, pct) # log the numb
223     log.info("  Legitimate  : %d  (%.1f%%)", n_legit, 100 - pct) # log
224     log.info("-" * 60) # log another separator line for better readabil
225
226     os.makedirs(os.path.dirname(output_path) or ".", exist_ok=True) # e
227     combined.to_csv(output_path, index=False) # save the combined DataF
228     log.info("Saved -> %s", os.path.abspath(output_path)) # log the abs
```

Figure 28 Deterministic Shuffle and Output

Shuffles the combined dataset using random_state=42, calculates the number of malicious and legitimate rows for summary reporting, ensures the output directory exists, and writes the balanced dataset to CSV.

```
290 try:
291     phishtank_df = load_phishtank(args.phishtank) #
292     urlhaus_df   = load_urlhaus(args.urlhaus) # load
293
294     # Dataset 1: PhishTank + Tranco
295     # This will build the first dataset by combining
296     build_one_dataset(
297         phishing_df=phishtank_df,
298         tranco_path=args.tranco,
299         output_path=args.out_phishtank,
300         source_name="PhishTank + Tranco",
301         limit=args.limit,
302     )
303
304     # Dataset 2: URLhaus + Tranco
305     # This will build the second dataset by combinin
306     build_one_dataset(
307         phishing_df=urlhaus_df,
308         tranco_path=args.tranco,
309         output_path=args.out_urlhaus,
310         source_name="URLhaus + Tranco",
311         limit=args.limit,
312     )
```

Figure 29 Dataset Generation Workflow

Orchestrates the construction of both final dataset by loading the PhishTank and URLhaus malicious sources, then calling build_one_dataset() once for PhishTank-Tranco dataset and once for the URLhaus-Tranco dataset.

A.4 Data Preparation Module (prepare_data.py)

Section Description:

This module cleans, validates, and standardizes the balanced datasets produced by build_dataset.py. It ensures that each dataset contains consistent URL and label columns, normalizes labels to binary values, removes invalid or duplicate rows, logs class balance, and saves cleaned files for feature extraction.

```
55  DEFAULT_DATASETS = [
56      os.path.join("data", "phishtank_tranco_dataset.csv"), # default input fil
57      os.path.join("data", "urlhaus_tranco_dataset.csv"), # default input file
58  ]
59
60  # Column name candidates across common dataset formats
61  URL_CANDIDATES   = ["url", "urls", "link", "domain", "address"] # common vari
62  LABEL_CANDIDATES = ["label", "class", "target", "type", "result", "status"] #
63
```

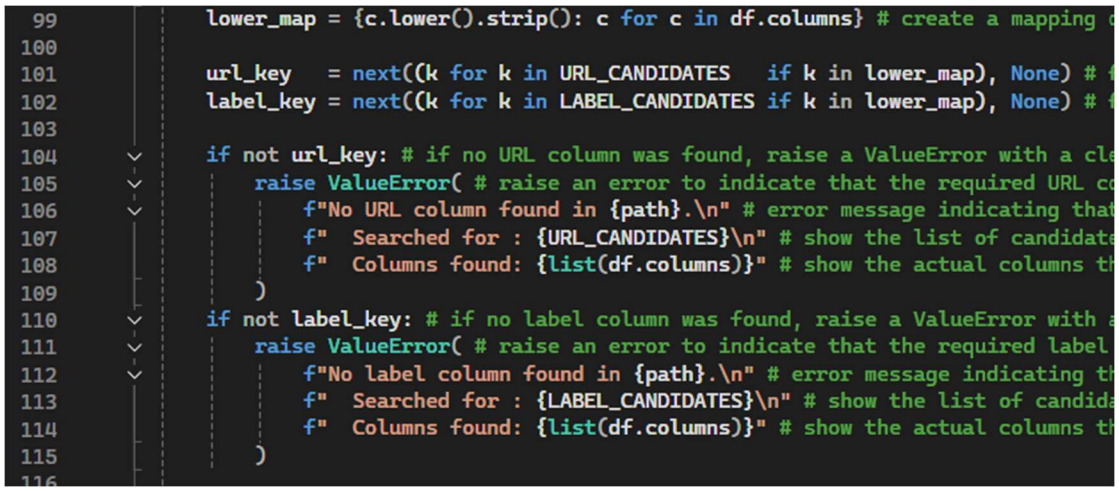
Figure 30 Dataset Defaults
Defines the default dataset processed by the script and lists accepted column-name variants for URL and label fields.

```
65  LABEL_MAP = {
66      "phishing":   1, "phish":     1, "bad":       1,
67      "malicious":  1, "spam":      1, "deceptive": 1, "1": 1,
68      "legitimate": 0, "benign":    0, "good":      0,
69      "safe":       0, "ham":       0, "0":         0,
70  }
71
```

Figure 31 Label Mapping
Maps common malicious and legitimate label formats into binary labels, allowing the cleaning step to standardize different dataset label conventions.

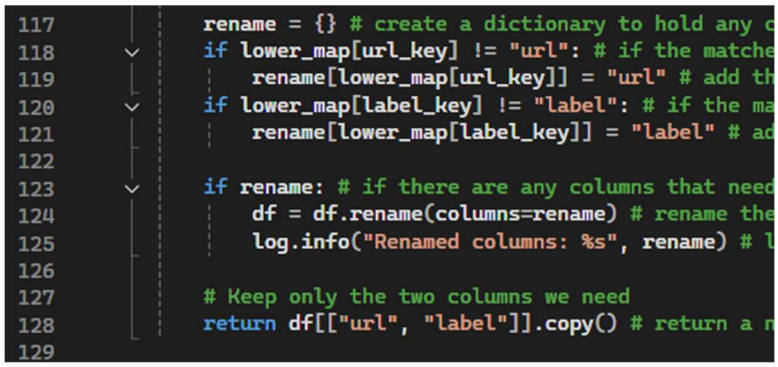
```
76  def load_csv(path: str) -> pd.DataFrame: # helper function to load CSV wit
77      """Load CSV with automatic UTF-8 / latin-1 encoding fallback."""
78      try: # first try to load with UTF-8 encoding (most common)
79          df = pd.read_csv(path, low_memory=False) # load CSV, disable low_m
80          log.info("Loaded %d rows from %s (utf-8)", len(df), path) # log ho
81          return df # return the loaded DataFrame
82      except UnicodeDecodeError: # if UTF-8 fails, try latin-1 encoding
83          df = pd.read_csv( # load CSV again with latin-1 encoding, skip bad
84              path, encoding="latin-1",
85              on_bad_lines="skip", low_memory=False
86          )
87          log.warning( # log a warning that UTF-8 failed and latin-1 was use
88              "Loaded %d rows from %s (latin-1 fallback)", len(df), path # l
89          )
90          return df # return the loaded DataFrame
```

Figure 32 CSV Loading
Loads each dataset as a pandas DataFrame and includes a Latin-1 fallback if UTF-8 decoding fails.



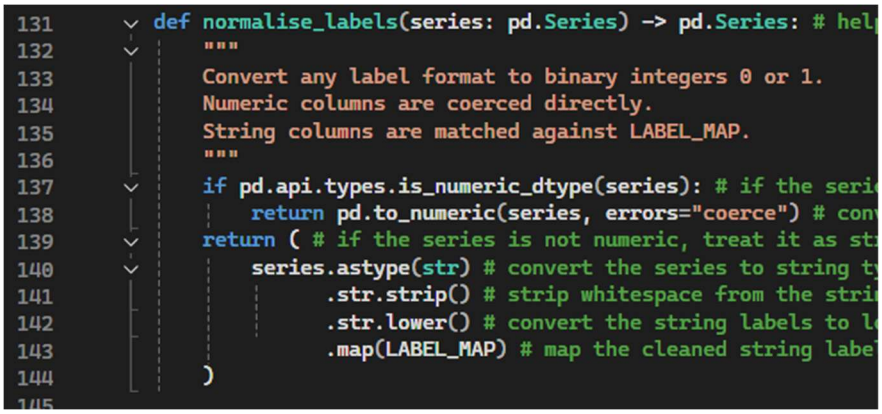
```
    lower_map = {c.lower().strip(): c for c in df.columns} # create a mapping c

    url_key   = next((k for k in URL_CANDIDATES   if k in lower_map), None) # f
    label_key = next((k for k in LABEL_CANDIDATES if k in lower_map), None) # f

    if not url_key: # if no URL column was found, raise a ValueError with a cle
        raise ValueError( # raise an error to indicate that the required URL co
            f"No URL column found in {path}.\n" # error message indicating that
            f"  Searched for : {URL_CANDIDATES}\n" # show the list of candidate
            f"  Columns found: {list(df.columns)}" # show the actual columns th
        )
    if not label_key: # if no label column was found, raise a ValueError with a
        raise ValueError( # raise an error to indicate that the required label
            f"No label column found in {path}.\n" # error message indicating th
            f"  Searched for : {LABEL_CANDIDATES}\n" # show the list of candida
            f"  Columns found: {list(df.columns)}" # show the actual columns th
        )
```

Figure 33 Column Standardization
Locates URL and label columns using accepted candidate names, raises an error if either field is missing, and renames matched columns to the URL and label format.



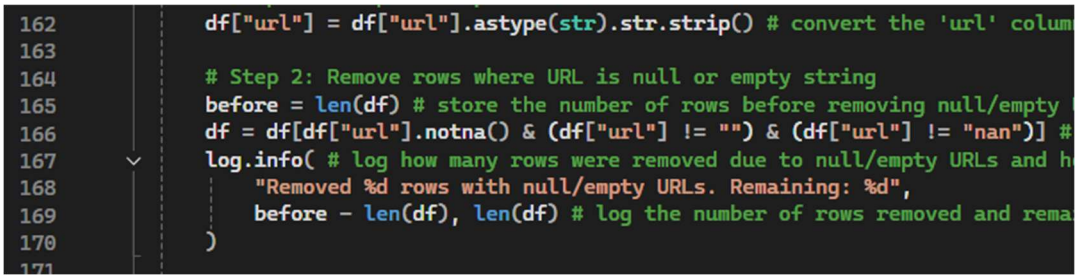
```
    rename = {} # create a dictionary to hold any c
    if lower_map[url_key] != "url": # if the matche
        rename[lower_map[url_key]] = "url" # add th
    if lower_map[label_key] != "label": # if the ma
        rename[lower_map[label_key]] = "label" # ad

    if rename: # if there are any columns that need
        df = df.rename(columns=rename) # rename the
        log.info("Renamed columns: %s", rename) # l

    # Keep only the two columns we need
    return df[["url", "label"]].copy() # return a n
```

Figure 34 Column Rename
Renames the detected URL and label columns to standardized names and returns only the two required fields used by downstream scripts.



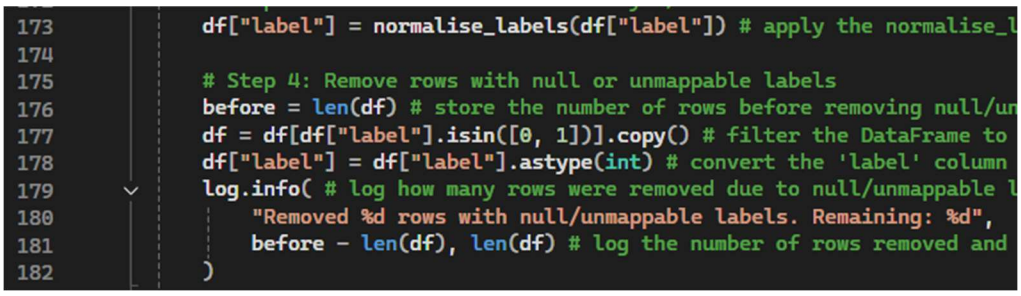
```
def normalise_labels(series: pd.Series) -> pd.Series: # hel
    """
    Convert any label format to binary integers 0 or 1.
    Numeric columns are coerced directly.
    String columns are matched against LABEL_MAP.
    """
    if pd.api.types.is_numeric_dtype(series): # if the seri
        return pd.to_numeric(series, errors="coerce") # con
    return ( # if the series is not numeric, treat it as st
        series.astype(str) # convert the series to string t
            .str.strip() # strip whitespace from the stri
            .str.lower() # convert the string labels to l
            .map(LABEL_MAP) # map the cleaned string labe
    )
```

Figure 35 Label Normalization
Converts numeric labels directly when possible and maps string-based labels to binary 0 or 1 values using the predefined label map.



```
    df["url"] = df["url"].astype(str).str.strip() # convert the 'url' colum

    # Step 2: Remove rows where URL is null or empty string
    before = len(df) # store the number of rows before removing null/empty
    df = df[df["url"].notna() & (df["url"] != "") & (df["url"] != "nan")] #
    log.info( # log how many rows were removed due to null/empty URLs and h
        "Removed %d rows with null/empty URLs. Remaining: %d",
        before - len(df), len(df) # log the number of rows removed and rema
    )
```

Figure 36 URL Cleaning
Strips whitespace from URL values and removes row where the URL is null, empty, or represented as the string "nan".



```
    df["label"] = normalise_labels(df["label"]) # apply the normalise_l

    # Step 4: Remove rows with null or unmappable labels
    before = len(df) # store the number of rows before removing null/un
    df = df[df["label"].isin([0, 1])].copy() # filter the DataFrame to
    df["label"] = df["label"].astype(int) # convert the 'label' column
    log.info( # log how many rows were removed due to null/unmappable l
        "Removed %d rows with null/unmappable labels. Remaining: %d",
        before - len(df), len(df) # log the number of rows removed and
    )
```

Figure 37 Label Validation
Normalizes labels to binary values, removes rows with null or unmappable labels, and converts the final label column to integer type.

```
185         before = len(df) # store the number of rows before removing duplicate U
186         df = df.drop_duplicates(subset=["url"]).reset_index(drop=True) # drop d
187         log.info( # log how many duplicate URLs were removed and how many remai
188             "Removed %d duplicate URLs. Remaining: %d",
189             before - len(df), len(df) # log the number of duplicate URLs remove
190         )
191
192         if len(df) == 0: # if no valid rows remain after all the cleaning steps
193             raise ValueError( # raise an error to indicate that the dataset is
194                 "No valid rows remain after cleaning. "
195                 "Check that the input file has correct URL and label values."
196             )
197
198         return df # return the cleaned DataFrame with only 'url' and 'label' co
199
```

Figure 38 Duplicate Removal

Removes duplicate URLs to prevent repeated entries from influencing downstream feature extraction and model training.

```
201     def log_class_balance(df: pd.DataFrame) -> None: # helper function to lo
202         """Log class distribution for verification."""
203         n_mal   = int((df["label"] == 1).sum()) # count the number of malici
204         n_legit = int((df["label"] == 0).sum()) # count the number of legiti
205         total   = len(df) # calculate the total number of samples in the cle
206         pct_mal = 100 * n_mal / total if total > 0 else 0 # calculate the pe
207
208         log.info("-" * 50)
209         log.info("Class balance summary:") # log a summary of the class bala
210         log.info("  Total rows  : %d", total) # log the total number of rows
211         log.info("  Malicious   : %d  (%.1f%%)", n_mal,   pct_mal) # log the
212         log.info("  Legitimate  : %d  (%.1f%%)", n_legit, 100 - pct_mal) # l
213         log.info("-" * 50)
214
```

Figure 39 Class Balance Logging

Counts malicious and legitimate samples after cleaning and logs the resulting class balance for verification.

```
261             try: # attempt to load, clean, and save the dataset, with error handlin
262                 df = load_csv(input_path) # load the input CSV file into a DataFram
263                 df = standardise_columns(df, input_path) # standardise the column n
264                 df = clean(df) # apply the full cleaning pipeline to the DataFrame
265                 log_class_balance(df) # log the class distribution of the cleaned d
266
267                 basename = os.path.splitext(os.path.basename(input_path))[0] # extr
268                 out_path = os.path.join(args.outdir, f"cleaned_{basename}.csv") # c
269
270                 df.to_csv(out_path, index=False) # save the cleaned DataFrame to a
271                 log.info("Saved -> %s", os.path.abspath(out_path)) # log the absolu
272                 success += 1 # increment the success counter if the dataset was cle
```

Figure 40 Cleaning Workflow

Applies the full preparation workflow to each dataset: loading the CSV, standardizing columns, cleaning values, logging class balance, and saving the cleaned output file.

A.5 Feature Engineering Module (extract_features.py)

Section Description:

This module generates the standardized 24-feature set from the cleaned dataset produced by prepare_data.py. All features are derived directly from the URL string, with no DNS, WHOIS, or external lookup dependency, ensuring both datasets use identical feature columns for training and cross-dataset evaluation.

```
68  DEFAULT_DATASETS = [
69      os.path.join("data", "cleaned_phishtank_tranco_dataset.csv"), #
70      os.path.join("data", "cleaned_urlhaus_tranco_dataset.csv"), # de
71  ]
```

Figure 41 Dataset Defaults
Defines the default cleaned dataset that are processed during feature extraction.

```
77  def load_csv(path: str) -> pd.DataFrame: # load a CSV file into a DataFram
78      """Load a cleaned CSV with UTF-8 / latin-1 encoding fallback."""
79      try: # first try to load with UTF-8 encoding (the most common encoding
80          df = pd.read_csv(path, low_memory=False) # load the CSV file at 'p
81          log.info("Loaded %d rows from %s", len(df), path) # log how many r
82          return df # return the loaded DataFrame to the caller
83      except UnicodeDecodeError: # if there was a UnicodeDecodeError (common
84          df = pd.read_csv( # load the CSV file again, but this time with en
85              path, encoding="latin-1", # skip bad lines that can't be parse
86              on_bad_lines="skip", low_memory=False # log a warning that we
87          )
88          log.warning( # log a warning that we had to fall back to latin-1 e
89              "Loaded %d rows from %s (latin-1 fallback)", len(df), path
90          )
91          return df # return the loaded DataFrame to the caller
```

Figure 42 CSV Loading
Loads a cleaned CSV file and provides a Latin-1 fallback if UTF-8 decoding fails.

```python
def _shannon_entropy(s: str) -> float: # compute the Shannon ent
    """
    Compute character-level Shannon entropy for a string.
    Higher entropy indicates more randomness – a common trait
    of algorithmically generated phishing URLs.
    """
    if not s: # if the string is empty or None, return 0.0 entro
        return 0.0 # compute the frequency of each character in
    freq = pd.Series(list(s)).value_counts(normalize=True) # cre
    return float(-(freq * np.log2(freq)).sum()) # compute the Sh
```

Figure 43 Entropy Calculation
Computes Shannon entropy for a URL string. Entropy is used to measure character-level randomness, which can be useful for detecting obfuscated or algorithmically generated URLs.

```python
    df = df.copy() # work on a copy of the input DataFrame to
    df["url"] = df["url"].astype(str).str.strip() # ensure the

    log.info("Extracting features from %d URLs ...", len(df))

    # Parse URL components once for reuse across feature groups
    def safe_parse(u): # helper function to parse a URL string
        return urlparse(u if "://" in u else "http://" + u) #

    parsed    = df["url"].apply(safe_parse) # apply the safe_pa
    hostname  = parsed.apply(lambda p: p.hostname or "") # ext
    path_part = parsed.apply(lambda p: p.path or "") # extract
    query     = parsed.apply(lambda p: p.query or "") # extract
```

Figure 44 URL Parsing
Normalizes URL input, safely parses URLs into components, and extracts hostname, path, and query portions for feature generation.

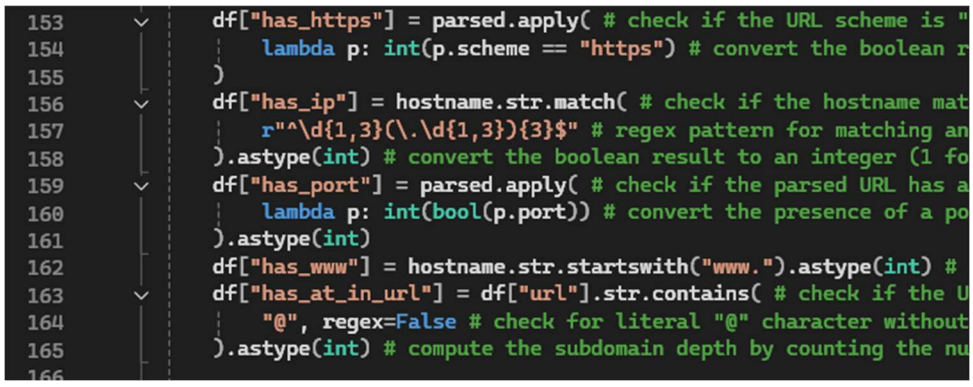
```python
    df["url_length"]      = df["url"].str.len() # c
    df["hostname_length"] = hostname.str.len() # co
    df["path_length"]     = path_part.str.len() # c
    df["query_length"]    = query.str.len() # comput
```

Figure 45 Length Features
Calculates URL length, hostname length, path length, and query length features.

```
    df["num_dots"]        = df["url"].str.count(r"\.") # c
    df["num_hyphens"]     = df["url"].str.count(r"\-") # c
    df["num_underscores"] = df["url"].str.count(r"\_") # c
    df["num_slashes"]     = df["url"].str.count(r"\/") # c
    df["num_qmarks"]      = df["url"].str.count(r"\?") # c
    df["num_equals"]      = df["url"].str.count(r"=") # cou
    df["num_ampersands"]  = df["url"].str.count(r"&") # cou
    df["num_at"]          = df["url"].str.count(r"@") # cou
    df["num_percent"]     = df["url"].str.count(r"%") # cou
    df["num_digits"]      = df["url"].str.count(r"\d") # c
```

Figure 46 Character Count Features

Counts selected characters within each URL, including dots, hyphens, underscores, slashes, question marks, equals signs, ampersands, at signs, percent signs, and digits.



```
    df["has_https"] = parsed.apply( # check if the URL scheme is "
        lambda p: int(p.scheme == "https") # convert the boolean r
    )
    df["has_ip"] = hostname.str.match( # check if the hostname mat
        r"^\d{1,3}(\.\d{1,3}){3}$" # regex pattern for matching an
    ).astype(int) # convert the boolean result to an integer (1 fo
    df["has_port"] = parsed.apply( # check if the parsed URL has a
        lambda p: int(bool(p.port)) # convert the presence of a po
    ).astype(int)
    df["has_www"] = hostname.str.startswith("www.").astype(int) #
    df["has_at_in_url"] = df["url"].str.contains( # check if the U
        "@", regex=False # check for literal "@" character without
    ).astype(int) # compute the subdomain depth by counting the nu
```

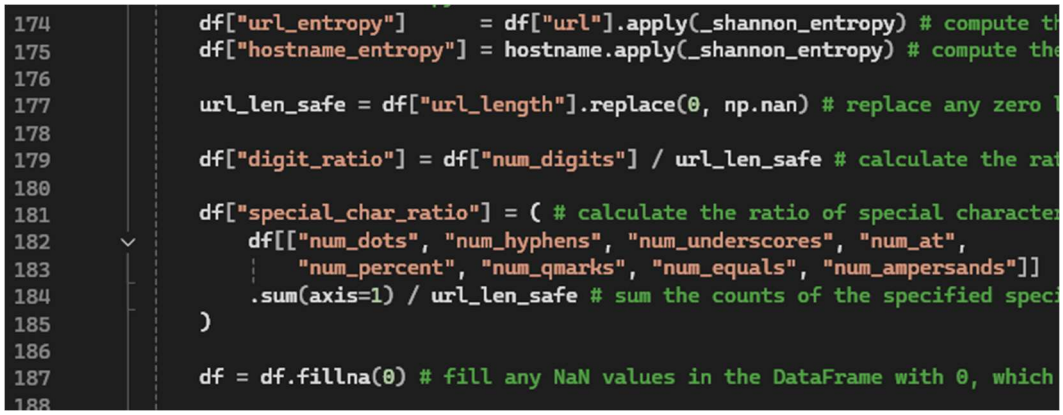
Figure 47 Structural Features

Generates structural and Boolean features, including HTTPS use, IP-based hostnames, port presence, www prefix, and use of @ in the URL.

```
def subdomain_depth(h): # helper function to compute the s
    parts = h.split(".") if h else [] # split the hostname
    return max(0, len(parts) - 2) # compute the subdomain

df["subdomain_depth"] = hostname.apply(subdomain_depth) #
```

Figure 48 Subdomain Depth

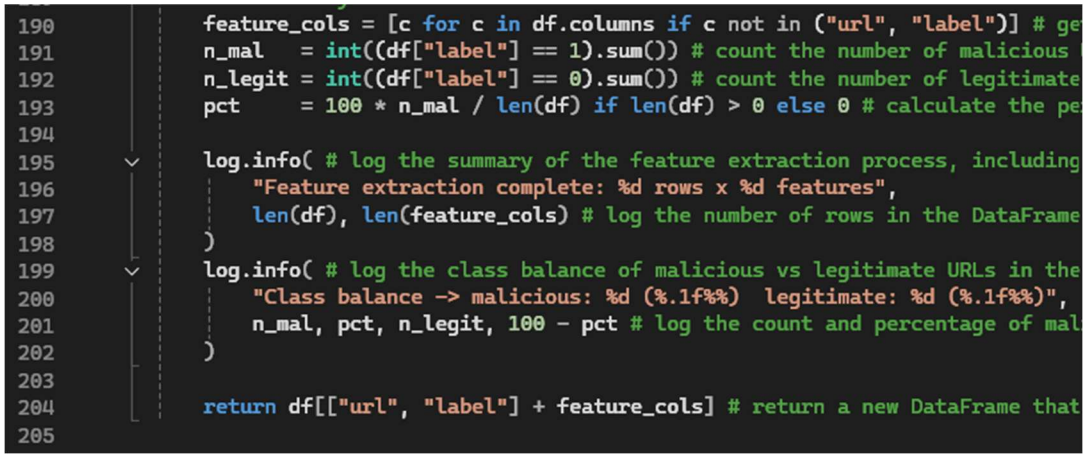
Calculates subdomain depth by splitting the hostname and counting domain components beyond the registered domain and top-level domain.



```
174     df["url_entropy"]      = df["url"].apply(_shannon_entropy) # compute th
175     df["hostname_entropy"] = hostname.apply(_shannon_entropy) # compute the
176
177     url_len_safe = df["url_length"].replace(0, np.nan) # replace any zero l
178
179     df["digit_ratio"] = df["num_digits"] / url_len_safe # calculate the rat
180
181     df["special_char_ratio"] = ( # calculate the ratio of special character
182         df[["num_dots", "num_hyphens", "num_underscores", "num_at",
183             "num_percent", "num_qmarks", "num_equals", "num_ampersands"]]
184         .sum(axis=1) / url_len_safe # sum the counts of the specified speci
185     )
186
187     df = df.fillna(0) # fill any NaN values in the DataFrame with 0, which
188
```

Figure 49 Entropy and Ratio Features

Calculates URL entropy, hostname entropy, digit ratio, and special-character ratio. These features represent randomness and character composition within the URL string.



```
190     feature_cols = [c for c in df.columns if c not in ("url", "label")] # ge
191     n_mal   = int((df["label"] == 1).sum()) # count the number of malicious
192     n_legit = int((df["label"] == 0).sum()) # count the number of legitimate
193     pct     = 100 * n_mal / len(df) if len(df) > 0 else 0 # calculate the pe
194
195     log.info( # log the summary of the feature extraction process, including
196         "Feature extraction complete: %d rows x %d features",
197         len(df), len(feature_cols) # log the number of rows in the DataFrame
198     )
199     log.info( # log the class balance of malicious vs legitimate URLs in the
200         "Class balance -> malicious: %d (%.1f%%)  legitimate: %d (%.1f%%)",
201         n_mal, pct, n_legit, 100 - pct # log the count and percentage of mal
202     )
203
204     return df[["url", "label"] + feature_cols] # return a new DataFrame that
205
```

Figure 50 Output Column Structure

Identifies feature columns, logs dataset summary information, and returns the final DataFrame containing URL, label, and all extracted feature columns.

```
    df = load_csv(path) # load the cleaned CSV file into a DataFrame us

    if "url" not in df.columns or "label" not in df.columns: # check if
        log.error( # log an error that the input file does not have the
            "Input file must have 'url' and 'label' columns. " # log an
            "Found: %s. Run prepare_data.py first.", list(df.columns) #
        )
        return # exit the function early since we cannot proceed with f

    df = extract_features(df) # extract features from the loaded DataFr

    # Output filename: features_<original_name>.csv
    # Strips the 'cleaned_' prefix since this is now the feature file
    basename = os.path.splitext(os.path.basename(path))[0] # get the ba
    basename = basename.replace("cleaned_", "") # remove the "cleaned_"
    out_path = os.path.join(outdir, f"features_{basename}.csv") # const

    os.makedirs(outdir, exist_ok=True) # create the output directory if
    df.to_csv(out_path, index=False) # save the resulting DataFrame wit
    log.info( # log the success of saving the feature-rich CSV file, in
        "Saved -> %s  (%d rows, %d columns)", # log the success of savi
        out_path, len(df), len(df.columns) # log the output path where
    )
    log.info("Next step: python src/train_models.py") # log a message s
```

Figure 51 Feature File Output
Processes a cleaned dataset, extracts URL-string features, constructs the output filename, and saves the feature-rich CSV file for model training.

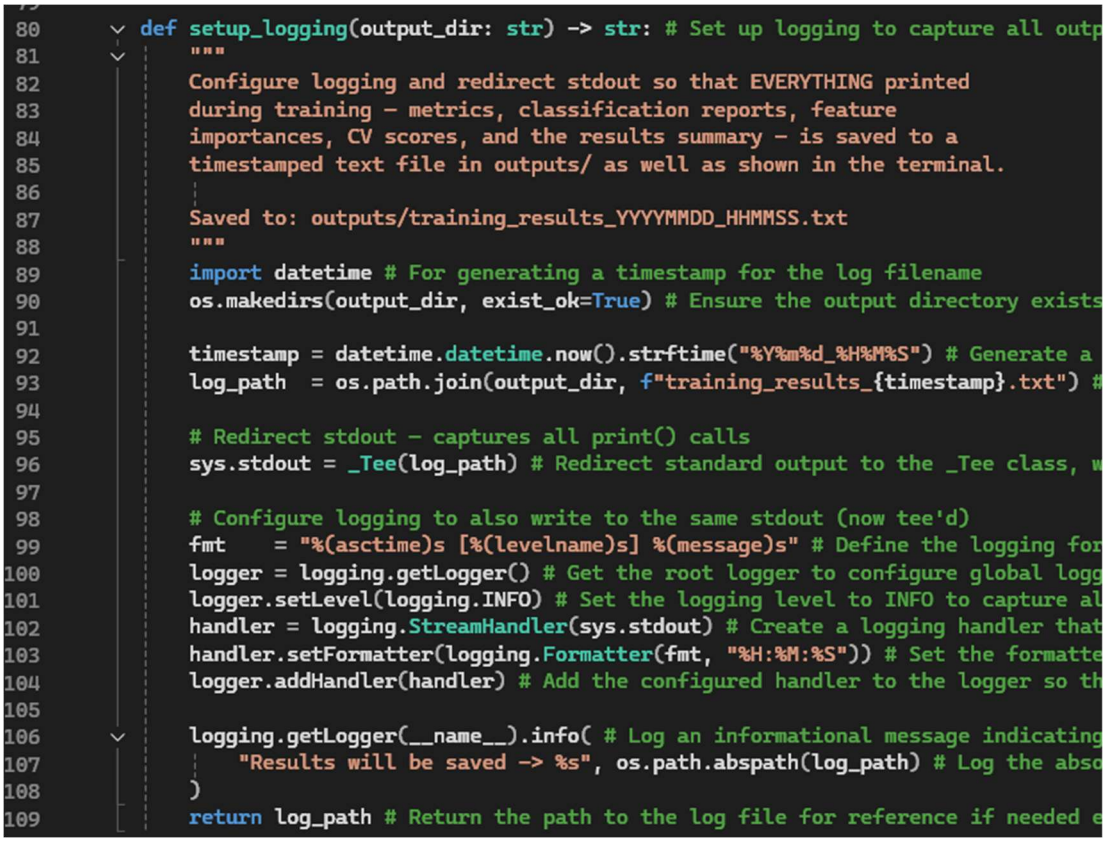
A.6 Model Training Module (train_models.py)

Section Description:

This module trains and evaluates Logistic Regression, Random Forest, and Decision Tree classifiers on the feature datasets produced by extract_features.py. It performs stratified train-test splitting, applies scaling for Logistic Regresssion, evaluates models using accuracy, precision, recall, F1-score, and confusion matrices, saves trained model artifacts, generates dashboards, and writes final results to outputs/model_results.csv.

```
FEATURE_FILES = [ # List of feature CSV files to load and process, p
    os.path.join("data", "features_phishtank_tranco_dataset.csv"),
    os.path.join("data", "features_urlhaus_tranco_dataset.csv"),
]

OUTPUT_DIR       = "outputs" # Directory where all outputs (dashboar
```

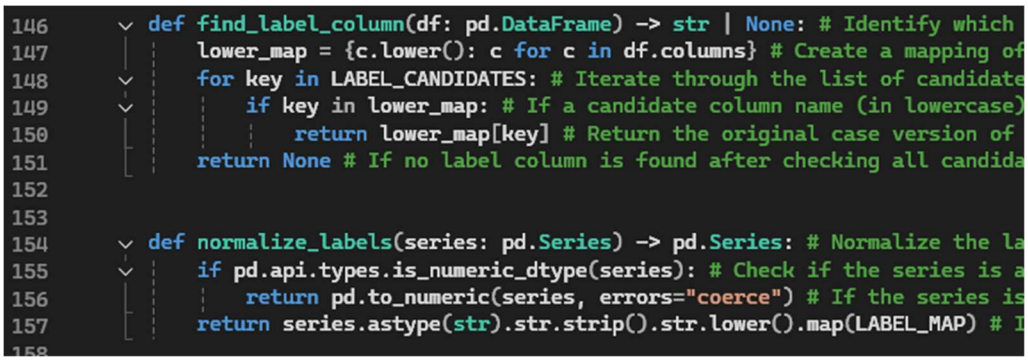
Figure 52 Feature File Inputs
Defines the feature dataset used for model training and the output directory where dashboards, model artifacts, logs, and result files are saved.



```
def setup_logging(output_dir: str) -> str: # Set up logging to capture all outp
    """
    Configure logging and redirect stdout so that EVERYTHING printed
    during training - metrics, classification reports, feature
    importances, CV scores, and the results summary - is saved to a
    timestamped text file in outputs/ as well as shown in the terminal.

    Saved to: outputs/training_results_YYYYMMDD_HHMMSS.txt
    """
    import datetime # For generating a timestamp for the log filename
    os.makedirs(output_dir, exist_ok=True) # Ensure the output directory exists

    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S") # Generate a
    log_path  = os.path.join(output_dir, f"training_results_{timestamp}.txt") #

    # Redirect stdout - captures all print() calls
    sys.stdout = _Tee(log_path) # Redirect standard output to the _Tee class, w

    # Configure logging to also write to the same stdout (now tee'd)
    fmt    = "%(asctime)s [%(levelname)s] %(message)s" # Define the logging for
    logger = logging.getLogger() # Get the root logger to configure global logg
    logger.setLevel(logging.INFO) # Set the logging level to INFO to capture al
    handler = logging.StreamHandler(sys.stdout) # Create a logging handler that
    handler.setFormatter(logging.Formatter(fmt, "%H:%M:%S")) # Set the formatte
    logger.addHandler(handler) # Add the configured handler to the logger so th

    logging.getLogger(__name__).info( # Log an informational message indicating
        "Results will be saved -> %s", os.path.abspath(log_path) # Log the abso
    )
    return log_path # Return the path to the log file for reference if needed e
```

Figure 53 Training Log Setup

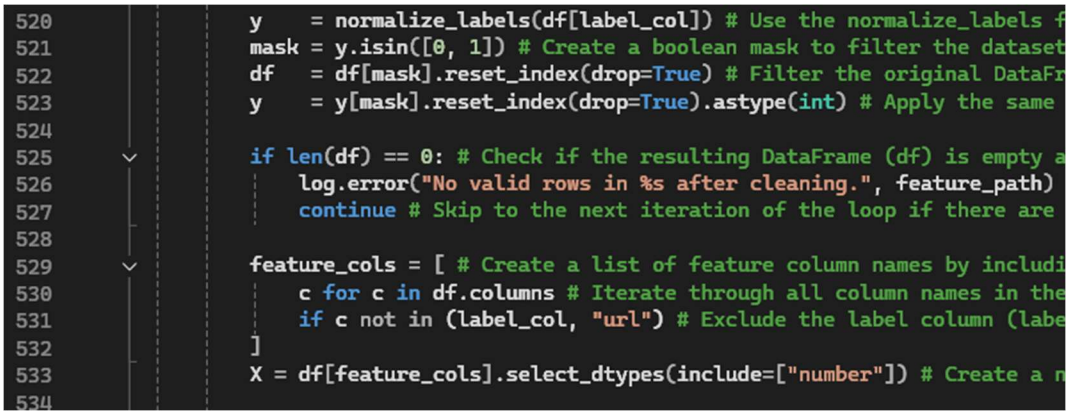
Creates a timestamped training log file and redirects terminal output so that training results are saved while still being displayed in the console.



```
def find_label_column(df: pd.DataFrame) -> str | None: # Identify which
    lower_map = {c.lower(): c for c in df.columns} # Create a mapping of
    for key in LABEL_CANDIDATES: # Iterate through the list of candidate
        if key in lower_map: # If a candidate column name (in lowercase)
            return lower_map[key] # Return the original case version of
    return None # If no label column is found after checking all candida


def normalize_labels(series: pd.Series) -> pd.Series: # Normalize the la
    if pd.api.types.is_numeric_dtype(series): # Check if the series is a
        return pd.to_numeric(series, errors="coerce") # If the series is
    return series.astype(str).str.strip().str.lower().map(LABEL_MAP) # I
```

Figure 54 Label Normalization

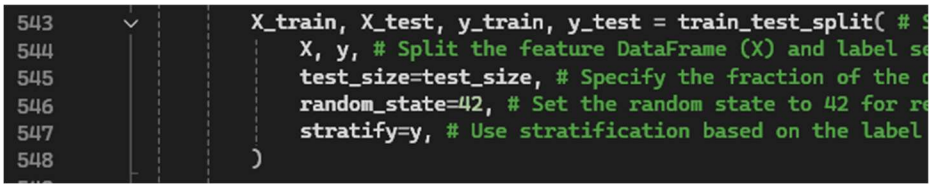
Identifies the label column and normalizes label values into binary classes, where 1 represents malicious URLs and 0 represents legitimate URLs.



```
520         y    = normalize_labels(df[label_col]) # Use the normalize_labels f
521         mask = y.isin([0, 1]) # Create a boolean mask to filter the dataset
522         df   = df[mask].reset_index(drop=True) # Filter the original DataFr
523         y    = y[mask].reset_index(drop=True).astype(int) # Apply the same
524
525         if len(df) == 0: # Check if the resulting DataFrame (df) is empty a
526             log.error("No valid rows in %s after cleaning.", feature_path)
527             continue # Skip to the next iteration of the loop if there are
528
529         feature_cols = [ # Create a list of feature column names by includi
530             c for c in df.columns # Iterate through all column names in the
531             if c not in (label_col, "url") # Exclude the label column (labe
532         ]
533         X = df[feature_cols].select_dtypes(include=["number"]) # Create a n
534
```

Figure 55 Feature and Label Preparation

Filters rows with binary labels, separates numeric feature columns from the label and URL fields, and prepares the feature matrix x and target vector y.



```
543         X_train, X_test, y_train, y_test = train_test_split( # S
544             X, y, # Split the feature DataFrame (X) and label se
545             test_size=test_size, # Specify the fraction of the d
546             random_state=42, # Set the random state to 42 for re
547             stratify=y, # Use stratification based on the label
548         )
```

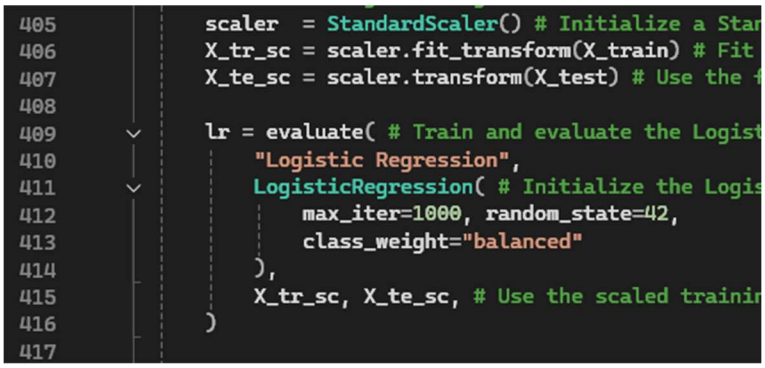
Figure 56 Stratified Train-Test Split

Splits the dataset into training and testing sets using stratification to preserve class distribution and random_state=42 to support reproducible results.

```
def evaluate(name, model, X_tr, X_te): # Define a nested functio
    model.fit(X_tr, y_train) # Fit the model to the training dat
    y_pred = model.predict(X_te) # Use the trained model to make
    cm = confusion_matrix(y_test, y_pred) # Compute the confusio

    result = { # Create a dictionary to store the results for th
        "dataset":   dataset_name,
        "model":     name,
        "accuracy":  round(accuracy_score(y_test, y_pred),  4),
        "precision": round(precision_score( # Calculate the prec
            y_test, y_pred, zero_division=0), 4),
        "recall":    round(recall_score( # Calculate the recall
            y_test, y_pred, zero_division=0), 4),
        "f1_score":  round(f1_score( # Calculate the F1-score of
            y_test, y_pred, zero_division=0), 4),
        "tn": int(cm[0][0]), "fp": int(cm[0][1]), # Extract the
        "fn": int(cm[1][0]), "tp": int(cm[1][1]), # Extract the
    }
    model_results.append(result) # Append the result dictionary
    all_results.append(result) # Append the result dictionary fo

    print(f"\n--- {name} ---")
    print(f"  Accuracy  : {result['accuracy']:.4f}") # Print the
    print(f"  Precision : {result['precision']:.4f}") # Print th
    print(f"  Recall    : {result['recall']:.4f}") # Print the r
    print(f"  F1-score  : {result['f1_score']:.4f}") # Print the
    print(classification_report( # Print the classification repo
        y_test, y_pred, # Generate the classification report by
        target_names=["Legitimate", "Malicious"], # Set the targ
        zero_division=0 # Handle cases where there are no positi
    ))
    return model # Return the trained model instance, which can
```

Figure 57 Model Evaluation Function

Trains a model, predicts labels on the test set, calculates evaluation metrics, stores confusion matrix values, and appends the results for dashboard generation and final reporting.



```
scaler  = StandardScaler() # Initialize a Star
X_tr_sc = scaler.fit_transform(X_train) # Fit
X_te_sc = scaler.transform(X_test) # Use the f

lr = evaluate( # Train and evaluate the Logist
    "Logistic Regression",
    LogisticRegression( # Initialize the Logis
        max_iter=1000, random_state=42,
        class_weight="balanced"
    ),
    X_tr_sc, X_te_sc, # Use the scaled trainin
)

```

Figure 58 Logistic Regression Training

Applies StandardScaler to the training data only, transforms the test data using the fitted scaler, and trains the Logistic Regression model.

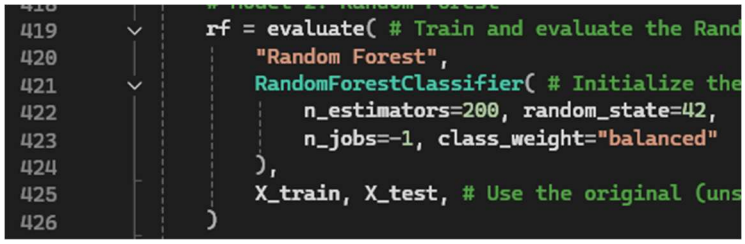


Figure 59 Random Forest Training
Trains the Random Forest classifier using 200 trees, balanced class weights, parallel processing, and a fixed random state for reproducibility.

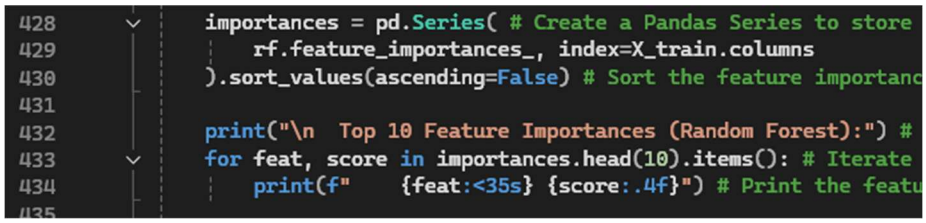


Figure 60 Feature Importance Extraction
Extracts and sorts Random Forest feature importance scores, allowing the system to identify which URL-string features contributed the strongest toward classification.

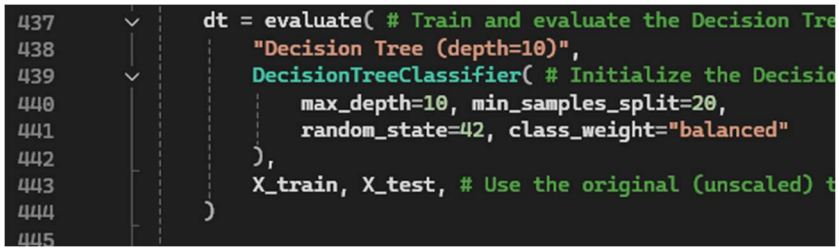


Figure 61 Decision Tree Training
Trains the Decision Tree classifier using a maximum depth of 10, minimum split size of 20, balanced class weights, and a fixed random state.

```
450     with open(os.path.join(OUTPUT_DIR, "model_lr.pkl"), "wb") as f: # Op
451         pickle.dump(lr, f) # Use pickle to serialize and save the traine
452
453     with open(os.path.join(OUTPUT_DIR, "model_rf.pkl"), "wb") as f: # Op
454         pickle.dump(rf, f) # Use pickle to serialize and save the traine
455
456     with open(os.path.join(OUTPUT_DIR, "model_dt.pkl"), "wb") as f: # Op
457         pickle.dump(dt, f) # Use pickle to serialize and save the traine
458
459     with open(os.path.join(OUTPUT_DIR, "scaler.pkl"), "wb") as f: # Oper
460         pickle.dump(scaler, f) # Use pickle to serialize and save the f
461
```

Figure 62 Model Artifact Saving

Saves the trained models and fitted scaler as pickle files so they can be reused later by the inference script without retraining.

```
483             cv = cross_val_score( # Perform 5-fold cross-validati
484                 model, X_cv, y_train, # Use the specified model a
485                 cv=StratifiedKFold(
486                     n_splits=5, shuffle=True, random_state=42), #
487                 scoring="f1", n_jobs=-1, # Set the scoring parame
488             )
489             print(f"    {name:<25s} mean={cv.mean():.4f}  " # Pri
490                   f"std={cv.std():.4f}") # Print the standard dev
```

Figure 63 Cross-Validation

Performs 5-fold stratified cross-validation on the training set using F1-scores as the evaluation metric.

```
492     show_dashboard( # Call the show_dashboard function to genera
493         dataset_name, model_results, importances, OUTPUT_DIR # P
494     )
495
```

Figure 64 Dashboard Generation

Creates and save the visual performance dashboard containing the model metrics table, grouped metric comparison chart, confusion matrices, and Random Forest features importance chart.

```
560     if all_results: # After processing all datasets, check if there are
561         results_df   = pd.DataFrame(all_results) # Create a DataFrame fr
562         results_path = os.path.join(OUTPUT_DIR, "model_results.csv") # D
563         results_df.to_csv(results_path, index=False) # Save the results
564         log.info( # Log an informational message indicating that all res
565             "All results saved -> %s",
566             os.path.abspath(results_path) # Log the absolute path to the
567         )
568
```

Figure 65 Results Output
Converts all model results into a DataFrame and saves the final training metrics to outputs/model_results.csv.

A.7 Cross-Dataset Evaluation Module (cross_dataset_eval.py)

Section Description:

This module evaluates cross-dataset generalization by training each model on one threat intelligence dataset and testing it on the other. It performs both evaluation directions, applies scaling only to training data for Logistic Regression, computes performance metrics, saves dashboard visualization, and exports structured cross-evaluation results.

```
71      PHISHTANK_FILE = os.path.join("data", "features_phishtank_tranco_dataset.csv") #
72      URLHAUS_FILE   = os.path.join("data", "features_urlhaus_tranco_dataset.csv") # 
73
74      OUTPUT_DIR       = "outputs" # directory to save results and dashboards
```

Figure 66 Cross-Evaluation Inputs
Defines the two feature datasets used for cross-dataset evaluation and the output directory where result files and dashboards are saved.

```
173         df        = load_dataset(path)  # Load the dataset from t
174         label_col = find_label_column(df)  # Identify the label c
175
176         if label_col is None:  # If no label column is found in t
177             log.error("No label column found in %s", path)  # If
178             sys.exit(1)  # Exit the program with a non-zero statu
179
180         y    = normalize_labels(df[label_col])  # Normalize the l
181         mask = y.isin([0, 1])   # Create a boolean mask to filter
182         df   = df[mask].reset_index(drop=True)   # Filter the ori
183         y    = y[mask].reset_index(drop=True).astype(int)  # Appl
184
185         feature_cols = [  # Select all columns that are not the l
186             c for c in df.columns  # Create a list of feature col
187             if c not in (label_col, "url")  # Exclude the label c
188         ]
189         X = df[feature_cols].select_dtypes(include=["number"])  #
190
191         log.info(  # Log the number of rows, features, and class
192             "Loaded %s: %d rows, %d features | "
193             "malicious: %d (%.1f%%)  legitimate: %d (%.1f%%)",
194             os.path.basename(path), len(X), len(X.columns),  # Lo
195             int(y.sum()), 100 * y.mean(),  # Log the number and p
196             int((y == 0).sum()), 100 * (1 - y.mean()),  # Log the
197         )
198         return X, y  # Return the feature matrix X and label vect
199
```

Figure 67 Dataset Preparation

Loads a feature CSV, identifies and normalizes the label column, filters valid binary labels, and returns the numeric feature matrix and target vector for training and testing.

```
220         direction = f"Train: {train_name}  →  Test: {test_name}"
221
222         print(f"\n{'='*60}")
223         print(f"CROSS-DATASET EVALUATION")
224         print(f"  {direction}") # Log the direction of the cross-e
225         print(f"  Train rows : {len(X_train)}") # Log the number o
226         print(f"  Test rows  : {len(X_test)}") # Log the number of
227         print(f"  Features   : {X_train.shape[1]}") # Log the numb
228         print(f"{'='*60}")
```

Figure 68 Cross-Evaluation Direction

Defines the cross-dataset evaluation direction and reports the training dataset, testing dataset, number of rows, and feature count.

```
232     def evaluate(name, model, X_tr, X_te): # Helper function to trai
233         model.fit(X_tr, y_train) # Train the model on the training da
234         y_pred = model.predict(X_te) # Use the trained model to make
235         cm     = confusion_matrix(y_test, y_pred) # Compute the confu
236
237         result = { # Compile the results for this model into a dictio
238             "train_on":  train_name, # Include the name of the traini
239             "test_on":   test_name, # Include the name of the testing
240             "model":     name, # Include the name of the model in the
241             "accuracy":  round(accuracy_score(y_test, y_pred),  4), #
242             "precision": round(precision_score( # Compute the precisi
243                 y_test, y_pred, zero_division=0), 4), # Compute the r
244             "recall":    round(recall_score( # Compute the F1-score o
245                 y_test, y_pred, zero_division=0), 4), # Compute the F
246             "f1_score":  round(f1_score( # Compute the F1-score of th
247                 y_test, y_pred, zero_division=0), 4), # Compute the F
248             "tn": int(cm[0][0]), "fp": int(cm[0][1]), # Include the c
249             "fn": int(cm[1][0]), "tp": int(cm[1][1]), # Include the c
250         }
251         model_results.append(result) # Append the results dictionary
252         all_results.append(result) # Append the results dictionary fo
253
254         print(f"\n--- {name} ---")
255         print(f"  Accuracy  : {result['accuracy']:.4f}")
256         print(f"  Precision : {result['precision']:.4f}")
257         print(f"  Recall    : {result['recall']:.4f}")
258         print(f"  F1-score  : {result['f1_score']:.4f}")
259         print(classification_report( # Print a detailed classificatio
260             y_test, y_pred,
261             target_names=["Legitimate", "Malicious"],
262             zero_division=0
263         ))
264         return model # Return the trained model after evaluation, whi
```

Figure 69 Model Evaluation Function

Trains a model on the full training dataset, predicts labels on the separate test dataset, calculates evaluation metrics, stores confusion matrix values, and appends the results for dashboard creation and final reporting.

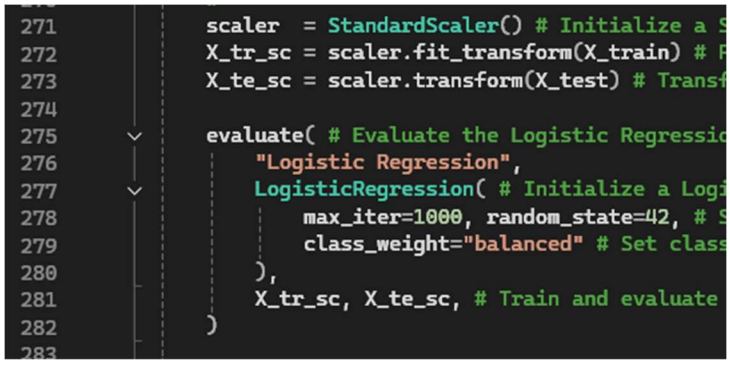


Figure 70 Logistic Regression Scaling

Fits StandardScaler only on the training dataset and then applies the same scaler to the test dataset, preventing data leakage during cross-dataset evaluation.

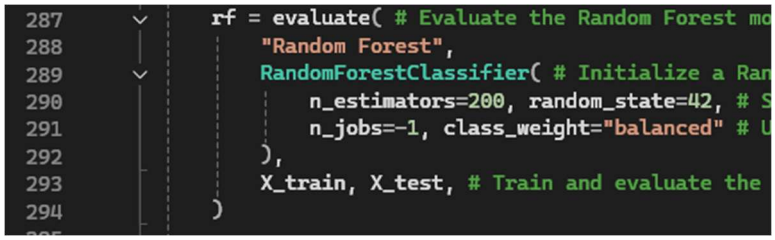


Figure 71 Random Forest Evaluation

Trains and evaluates the Random Forest classifier on the cross-dataset direction using unscaled features, balanced class weights, parallel processing, and a fixed random state.

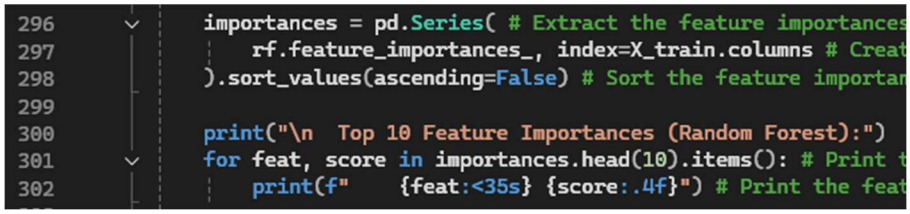


Figure 72 Feature Importance Extraction

Extracts and sorts Random Forest feature importance values from the model trained on the source dataset.

```
307     evaluate( # Evaluate the Decision Tree model using t
308         "Decision Tree (depth=10)",
309         DecisionTreeClassifier( # Initialize a Decision
310             max_depth=10, min_samples_split=20, # Set ma
311             random_state=42, class_weight="balanced" # S
312         ),
313         X_train, X_test, # Train and evaluate the Decis
314     )
```

Figure 73 Decision Tree Evaluation

Trains and evaluates the Decision Tree classifier using unscaled features, constrained tree depth, minimum split size, balanced class weights, and a fixed random state.

```
503     save_path = os.path.join(output_dir, save_name) #
504     plt.savefig( # Save the dashboard figure to the s
505         save_path, dpi=150, # Set the resolution of t
506         bbox_inches="tight", # Use tight bounding box
507         facecolor=fig.get_facecolor() # Set the face
508     )
```

Figure 74 Dashboard Output

Saves the cross-dataset dashboard as a PNG file using a filename that identifies the training and testing direction.

```
523         X_phish, y_phish = prepare(PHISHTANK_FILE)
524         X_url,   y_url   = prepare(URLHAUS_FILE) #
525
526         # ----------------------------------------
527         # Direction 1: Train on PhishTank → Test or
528         # ----------------------------------------
529         results_1, imp_1 = run_cross_eval( # Call t
530             train_name="PhishTank", # Set the name
531             test_name="URLhaus", # Set the name of
532             X_train=X_phish, y_train=y_phish, # Pro
533             X_test=X_url,   y_test=y_url, # Provide
534             all_results=all_results, # Pass the all
535         )
536         show_dashboard( # Call the show_dashboard f
537             train_name="PhishTank", # Set the name
538             test_name="URLhaus", # Set the name of
539             model_results=results_1, # Provide the
540             importances=imp_1, # Provide the featu
541             output_dir=OUTPUT_DIR,# Provide the fea
542         )
543
544         # ----------------------------------------
545         # Direction 2: Train on URLhaus → Test on P
546         # ----------------------------------------
547         results_2, imp_2 = run_cross_eval( # Call t
548             train_name="URLhaus",
549             test_name="PhishTank",
550             X_train=X_url,   y_train=y_url, # Provi
551             X_test=X_phish, y_test=y_phish, # Provi
552             all_results=all_results, # Pass the all
553         )
```

Figure 75 Two-Direction Evaluation Workflow
Prepares both datasets and runs cross-dataset evaluation in both directions: PhishTank to URLhaus and URLhaus to PhishTank.

```
565         results_df   = pd.DataFrame(all_results) # Create a pandas DataFrame from t
566         results_path = os.path.join(OUTPUT_DIR, "cross_eval_results.csv") # Define
567         results_df.to_csv(results_path, index=False) # Save the results DataFrame t
568         log.info("Cross-eval results saved -> %s", os.path.abspath(results_path)) #
569
```

Figure 76 Results Export
Saves the combined cross-dataset evaluation results to outputs/cross_eval_results.csv.

A.8 Inference Module (predict.py)

Section Description:

This module performs single-URL inference using the trained model artifacts produced by train_models.py. It loads the saved Logistic Regression, Random Forest, Decision Tree, and StandardScaler objects, extracts the same 24 URL-string features used

during training, applies model-specific preprocessing, and returns individual model predictions with a majority-vote classification.

```
64  OUTPUT_DIR  = "outputs"
65  MODEL_LR    = os.path.join(OUTPUT_DIR, "model_lr.pkl")
66  MODEL_RF    = os.path.join(OUTPUT_DIR, "model_rf.pkl")
67  MODEL_DT    = os.path.join(OUTPUT_DIR, "model_dt.pkl")
68  SCALER_PATH = os.path.join(OUTPUT_DIR, "scaler.pkl")
69
```

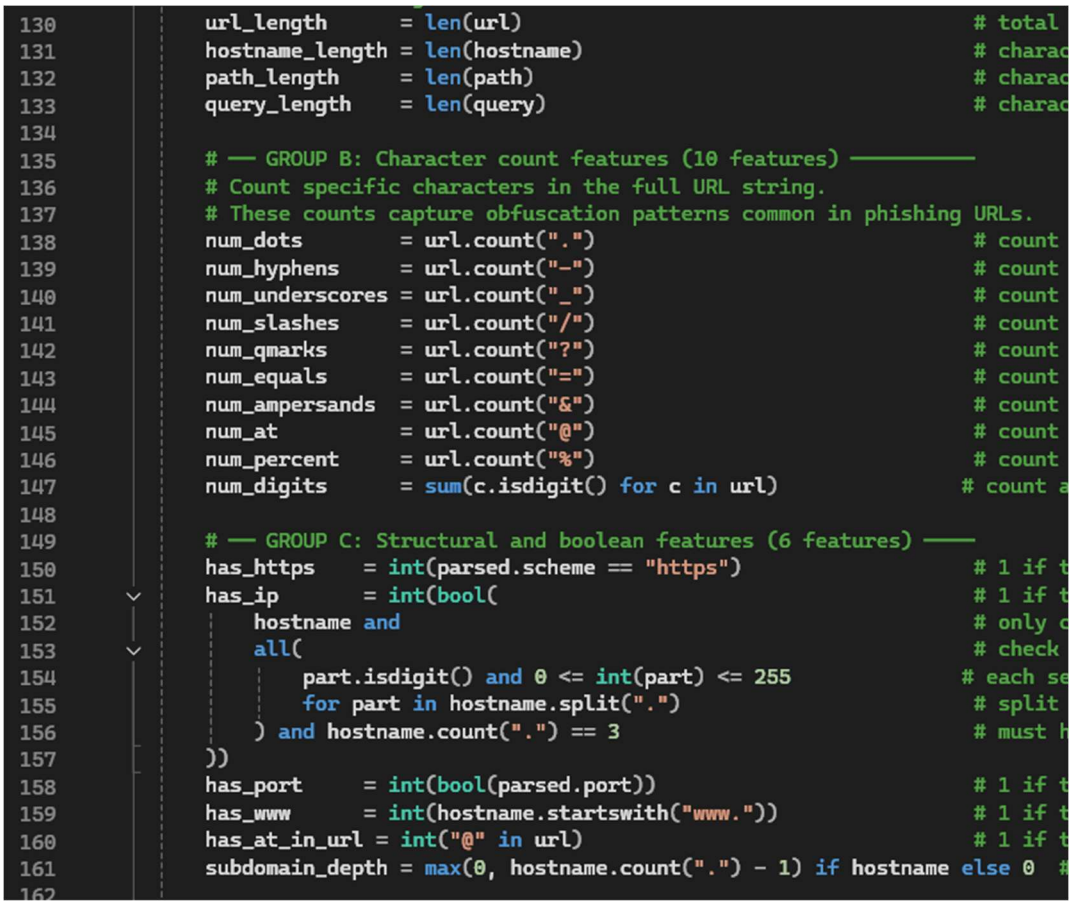
Figure 77 Model Artifact Paths
Defines the paths to the trained models and scaler that must exist before inference can run.

```
88  def _shannon_entropy(s: str) -> float:
89      """
90      Compute character-level Shannon entropy for a string.
91      Formula: H = -sum(p(x) * log2(p(x))) for each unique character x.
92      Higher entropy indicates more randomness in the character distribution,
93      which is a common trait of algorithmically generated phishing URLs.
94      Returns 0.0 for empty strings to avoid errors on empty URL components.
95      Identical to the _shannon_entropy function in extract_features.py.
96      """
97      if not s:                                                    # if the
98          return 0.0                                               # return
99      freq = pd.Series(list(s)).value_counts(normalize=True)       # count ho
100     return float(-(freq * np.log2(freq)).sum())                  # apply th
```

Figure 78 Single-URL Entropy
Computes Shannon entropy for the input URL string or hostname, matching the entropy logic used during feature extraction.

```
118     url = str(url).strip()                                       #
119
120     # ── Parse the URL into components using urllib.parse ──────────
121     # If the URL has no scheme (e.g., "example.com/path"), prepend
122     # "http://" so urlparse can correctly identify the hostname and
123     # This mirrors the safe_parse logic in extract_features.py exact
124     parsed   = urlparse(url if "://" in url else "http://" + url) #
125     hostname = parsed.hostname or ""                             #
126     path     = parsed.path     or ""                             #
127     query    = parsed.query    or ""                             #
128
```

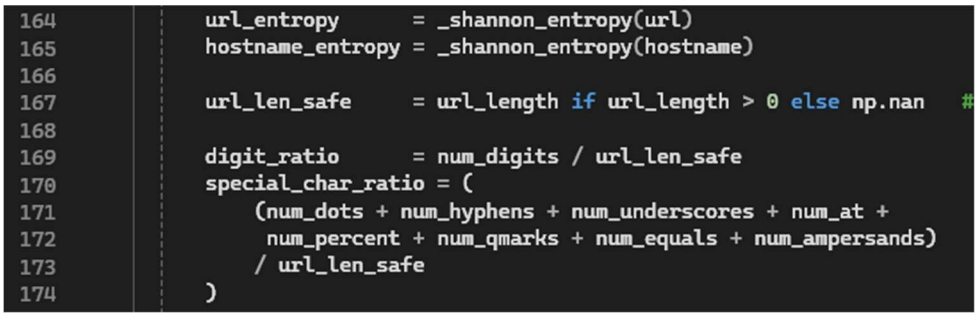
Figure 79 URL Parsing
Converts the supplied URL into a stripped string, safely parses it with urlparse, and extracts hostname, path, and query component.



```
    url_length      = len(url)                                          # total
    hostname_length = len(hostname)                                     # charac
    path_length     = len(path)                                         # charac
    query_length    = len(query)                                        # charac

    # ── GROUP B: Character count features (10 features) ──────────
    # Count specific characters in the full URL string.
    # These counts capture obfuscation patterns common in phishing URLs.
    num_dots        = url.count(".")                                    # count
    num_hyphens     = url.count("-")                                    # count
    num_underscores = url.count("_")                                    # count
    num_slashes     = url.count("/")                                    # count
    num_qmarks      = url.count("?")                                    # count
    num_equals      = url.count("=")                                    # count
    num_ampersands  = url.count("&")                                    # count
    num_at          = url.count("@")                                    # count
    num_percent     = url.count("%")                                    # count
    num_digits      = sum(c.isdigit() for c in url)                    # count a

    # ── GROUP C: Structural and boolean features (6 features) ────
    has_https    = int(parsed.scheme == "https")                        # 1 if t
    has_ip       = int(bool(                                            # 1 if t
        hostname and                                                    # only c
        all(                                                            # check
            part.isdigit() and 0 <= int(part) <= 255                   # each se
            for part in hostname.split(".")                             # split
        ) and hostname.count(".") == 3                                  # must h
    ))
    has_port     = int(bool(parsed.port))                               # 1 if t
    has_www      = int(hostname.startswith("www."))                     # 1 if t
    has_at_in_url = int("@" in url)                                     # 1 if t
    subdomain_depth = max(0, hostname.count(".") - 1) if hostname else 0  #
```

Figure 80 Single-URL Feature Extraction

Calculates length, character-count, structural, entropy, and ratio features for one user-supplied URL.



```
    url_entropy      = _shannon_entropy(url)
    hostname_entropy = _shannon_entropy(hostname)

    url_len_safe     = url_length if url_length > 0 else np.nan   #

    digit_ratio      = num_digits / url_len_safe
    special_char_ratio = (
        (num_dots + num_hyphens + num_underscores + num_at +
         num_percent + num_qmarks + num_equals + num_ampersands)
        / url_len_safe
    )
```

Figure 81 Entropy and Ratio Features

Calculates URL entropy, hostname entropy, digit ratio, and special-character ratio for the input URL.

```
    features = {
        # Group A - Length features (4)
        "url_length":        url_length,       # to
        "hostname_length":   hostname_length,  # ho
        "path_length":       path_length,      # pa
        "query_length":      query_length,     # qu
        # Group B - Character count features (10)
        "num_dots":          num_dots,         # co
        "num_hyphens":       num_hyphens,      # co
        "num_underscores":   num_underscores,  # co
        "num_slashes":       num_slashes,      # co
        "num_qmarks":        num_qmarks,       # co
        "num_equals":        num_equals,       # co
        "num_ampersands":    num_ampersands,   # co
        "num_at":            num_at,           # co
        "num_percent":       num_percent,      # co
        "num_digits":        num_digits,       # co
        # Group C - Structural/boolean features (6)
        "has_https":         has_https,        # 1
        "has_ip":            has_ip,           # 1
        "has_port":          has_port,         # 1
        "has_www":           has_www,          # 1
        "has_at_in_url":     has_at_in_url,    # 1
        "subdomain_depth":   subdomain_depth,  # nu
        # Group D - Entropy and ratio features (4)
        "url_entropy":       url_entropy,      # Sh
        "hostname_entropy":  hostname_entropy, # Sh
        "digit_ratio":       digit_ratio,      # pr
        "special_char_ratio": special_char_ratio, #
    }

    df = pd.DataFrame([features])              # wr
    df = df.fillna(0)                          # re
    return df                                  # re
```

Figure 82 Feature DataFrame Assembly

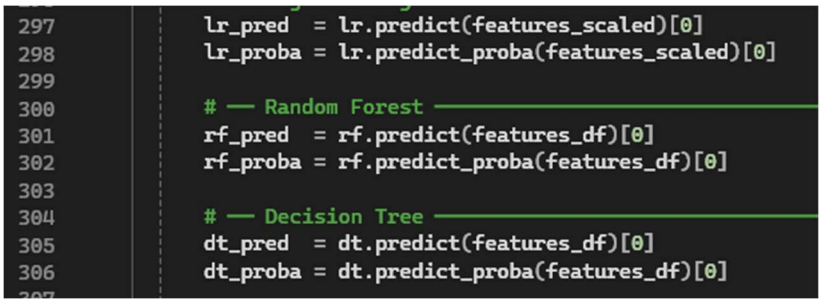
Assembles all 24 calculated values into a single-row DataFrame with the same feature schema expected by the trained classifiers.

```
    required = {                                # dictionary mappi
        "Logistic Regression": MODEL_LR,        # path to the Logis
        "Random Forest":       MODEL_RF,        # path to the Rando
        "Decision Tree":       MODEL_DT,        # path to the Decis
        "StandardScaler":      SCALER_PATH,     # path to the Stand
    }

    missing = [                                 # build a list of a
        f"  {name}: {path}"                     # format each missi
        for name, path in required.items()      # iterate through a
        if not os.path.exists(path)             # include in the mi
    ]

    if missing:                                 # if any required f
        log.error(                              # log an error mess
            "Required model files not found:\n%s\n"
            "Run train_models.py first to generate these files.",
            "\n".join(missing)                  # join all missing
        )
        sys.exit(1)                             # exit with error c

    log.info("Loading model artifacts from %s/", OUTPUT_DIR)  # lo

    with open(MODEL_LR, "rb") as f:             # open the Logistic
        lr = pickle.load(f)                     # deserialise the L

    with open(MODEL_RF, "rb") as f:             # open the Random F
        rf = pickle.load(f)                     # deserialise the R

    with open(MODEL_DT, "rb") as f:             # open the Decision
        dt = pickle.load(f)                     # deserialise the D

    with open(SCALER_PATH, "rb") as f:          # open the Standard
        scaler = pickle.load(f)                 # deserialise the S

    log.info("All model artifacts loaded successfully.")       # l
    return lr, rf, dt, scaler                   # return all four l
```

Figure 83 Model Loading
Verifies that all required model artifacts exist, loads each trained classifier and scaler from disk, and returns them for inference.

```
    features_df = extract_features_single(url)

    log.info("Running classifiers...")

    # Apply the scaler to the feature vector for Log
    # transform() is used — NOT fit_transform() — be
    # was already fitted on training data. Re-fittin
    # use the single test URL's statistics, which is
    features_scaled = scaler.transform(features_df)
```

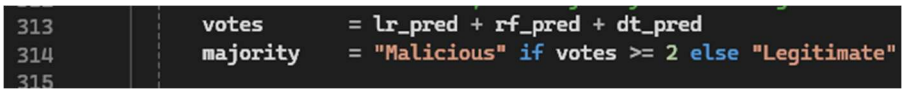
Figure 84 Inference Preprocessing
Extracts the 24 URL-string features form the input URL and applies the saved scaler only for Logistic Regression inference.



```
297     lr_pred  = lr.predict(features_scaled)[0]
298     lr_proba = lr.predict_proba(features_scaled)[0]
299
300     # ── Random Forest ──────────────────────────────
301     rf_pred  = rf.predict(features_df)[0]
302     rf_proba = rf.predict_proba(features_df)[0]
303
304     # ── Decision Tree ──────────────────────────────
305     dt_pred  = dt.predict(features_df)[0]
306     dt_proba = dt.predict_proba(features_df)[0]
```

Figure 85 Model Predictions

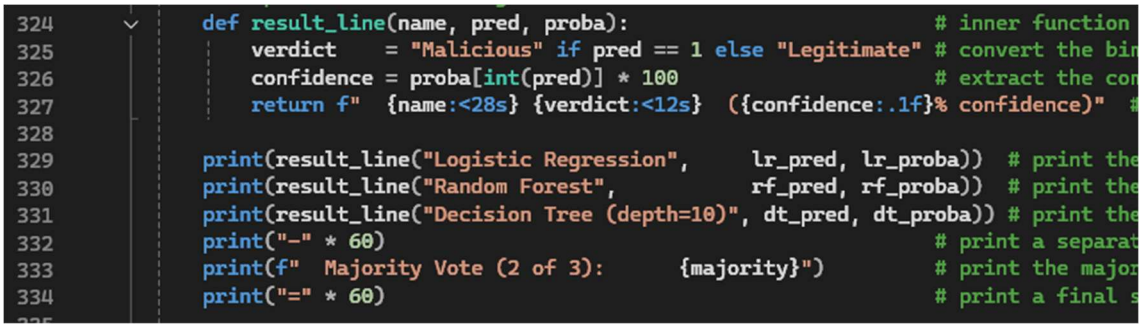
Runs the prepared URL features vector through all three trained classifiers and collects both predictions and class probabilities.



```
313     votes       = lr_pred + rf_pred + dt_pred
314     majority    = "Malicious" if votes >= 2 else "Legitimate"
315
```

Figure 86 Majority Vote

Combines the three binary classifier outputs and assigns the final majority-vote verdict.



```
324     def result_line(name, pred, proba):                          # inner function
325         verdict    = "Malicious" if pred == 1 else "Legitimate" # convert the bi
326         confidence = proba[int(pred)] * 100                      # extract the co
327         return f"  {name:<28s} {verdict:<12s}  ({confidence:.1f}% confidence)"  #
328
329     print(result_line("Logistic Regression",     lr_pred, lr_proba))  # print the
330     print(result_line("Random Forest",           rf_pred, rf_proba))  # print the
331     print(result_line("Decision Tree (depth=10)", dt_pred, dt_proba)) # print the
332     print("-" * 60)                                                  # print a separat
333     print(f"  Majority Vote (2 of 3):      {majority}")            # print the majo
334     print("=" * 60)                                                  # print a final
```

Figure 87 Result Formatting

Formats individual model predictions into readable output lines showing the model name, verdict, and confidence score.

```python
def parse_args() -> argparse.Namespace:                                    # funct
    """Parse the --url command-line argument."""
    p = argparse.ArgumentParser(                                           # creat
        description=(                                                      # descr
            "Classify a single URL as Malicious or Legitimate "
            "using trained model artifacts from train_models.py. "
            "No network connections are made during classification."
        )
    )
    p.add_argument(                                                        # defin
        "--url",                                                           # flag
        required=True,                                                     # this
        help=(                                                             # help
            "The URL to classify. "
            "Example: --url https://example.com"
        ),
    )
    return p.parse_args()                                                  # parse

```

Figure 88 CLI and Entry Point
Defines the required command-line argument for URL classification and provides user guidance through the help interface.

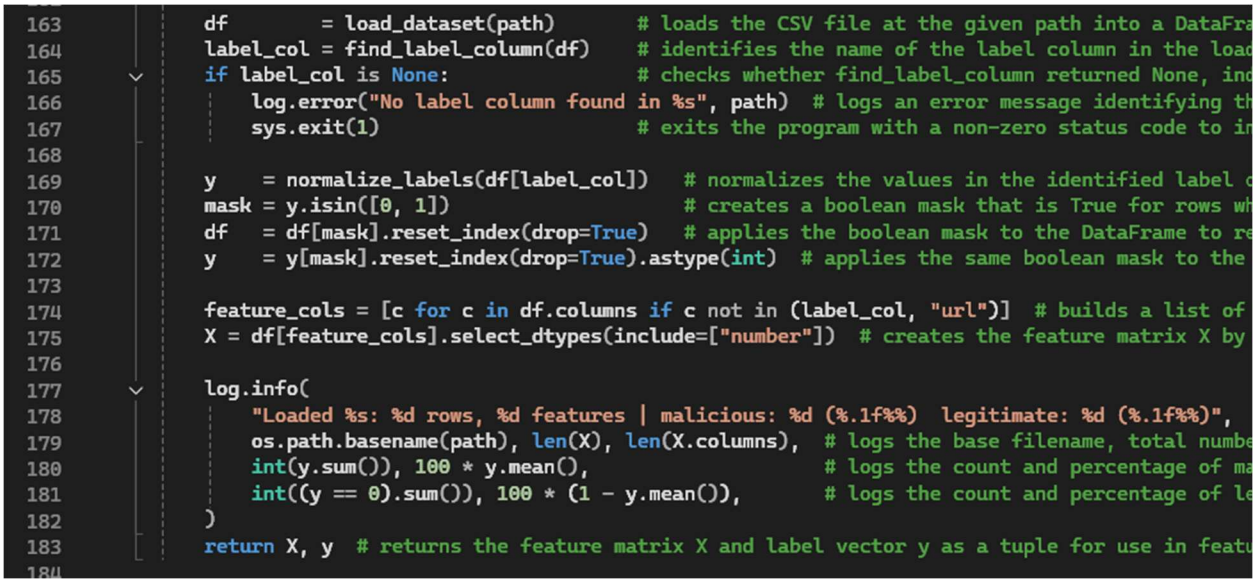
A.9 Feature Selection Evaluation Module (feature_selection_eval.py)

Section Description:

This module implements Iteration 2 by selecting features based on averaged Random Forest importance scores across both datasets. It compares full-feature and reduced-feature performance using within-dataset and cross-dataset evaluations, then saves comparison results, feature importance rankings, and dashboards.

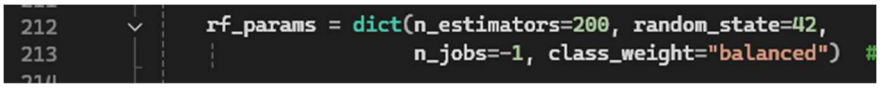
```python
PHISHTANK_FILE = os.path.join("data", "features_phishtank_tranco_dataset.csv")  #
URLHAUS_FILE   = os.path.join("data", "features_urlhaus_tranco_dataset.csv")    #
OUTPUT_DIR     = "outputs"  # defines the output directory name where all results
```

Figure 89 Iteration 2 Inputs
Defines the two feature datasets used for Iteration 2 feature selection and the output directory for results.



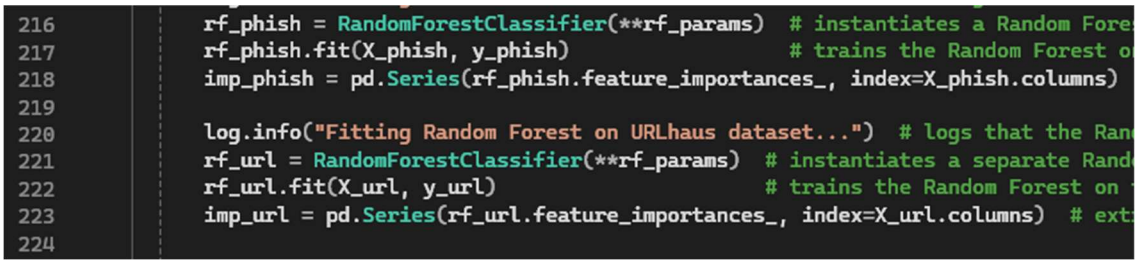
```
    df        = load_dataset(path)       # loads the CSV file at the given path into a DataFra
    label_col = find_label_column(df)    # identifies the name of the label column in the load
    if label_col is None:                # checks whether find_label_column returned None, ind
        log.error("No label column found in %s", path)  # logs an error message identifying th
        sys.exit(1)                      # exits the program with a non-zero status code to in

    y    = normalize_labels(df[label_col])   # normalizes the values in the identified label c
    mask = y.isin([0, 1])                    # creates a boolean mask that is True for rows wh
    df   = df[mask].reset_index(drop=True)   # applies the boolean mask to the DataFrame to re
    y    = y[mask].reset_index(drop=True).astype(int)  # applies the same boolean mask to the

    feature_cols = [c for c in df.columns if c not in (label_col, "url")]  # builds a list of
    X = df[feature_cols].select_dtypes(include=["number"])  # creates the feature matrix X by

    log.info(
        "Loaded %s: %d rows, %d features | malicious: %d (%.1f%%)  legitimate: %d (%.1f%%)",
        os.path.basename(path), len(X), len(X.columns),  # logs the base filename, total numbe
        int(y.sum()), 100 * y.mean(),                    # logs the count and percentage of ma
        int((y == 0).sum()), 100 * (1 - y.mean()),       # logs the count and percentage of le
    )
    return X, y  # returns the feature matrix X and label vector y as a tuple for use in featu
```

Figure 90 Dataset Preparation
Loads a feature CSV, identifies and normalizes the label column, removes invalid labels, and returns the numeric feature matrix and target labels.



```
    rf_params = dict(n_estimators=200, random_state=42,
                     n_jobs=-1, class_weight="balanced")  #
```

Figure 91 Random Forest Importance Setup
Defines the Random Forest configuration used to calculate feature importance scores consistently across both datasets.



```
    rf_phish = RandomForestClassifier(**rf_params)  # instantiates a Random Fore
    rf_phish.fit(X_phish, y_phish)                  # trains the Random Forest o
    imp_phish = pd.Series(rf_phish.feature_importances_, index=X_phish.columns)

    log.info("Fitting Random Forest on URLhaus dataset...")  # logs that the Ran
    rf_url = RandomForestClassifier(**rf_params)  # instantiates a separate Rand
    rf_url.fit(X_url, y_url)                      # trains the Random Forest on
    imp_url = pd.Series(rf_url.feature_importances_, index=X_url.columns)  # ext
```

Figure 92 Feature Importance Extraction
Trains separate Random Forest models on the PhishTank and URLhaus datasets and extracts feature importance scores from each model.

```
importance_df = pd.DataFrame({
    "phishtank":  imp_phish,  # adds the PhishTank importance scores as a column named phishtank in the comparis
    "urlhaus":    imp_url,    # adds the URLhaus importance scores as a column named urlhaus in the comparison D
})
importance_df["mean"] = importance_df.mean(axis=1)            # computes the mean importance score for each fea
importance_df = importance_df.sort_values("mean", ascending=False)  # sorts the importance DataFrame in descendi

print("\n" + "=" * 60)
print("FEATURE IMPORTANCE COMPARISON (all 24 features)")
print("=" * 60)
print(f"{'Feature':<30s} {'PhishTank':>12s} {'URLhaus':>12s} {'Mean':>12s}  {'Keep?':>6s}")  # prints the header
print("-" * 75)  # prints a horizontal separator line below the header row for visual clarity
for feat, row in importance_df.iterrows():                          # iterates over each row in the sorted imp
    keep = "YES" if row["mean"] >= threshold else "no"              # determines whether this feature should be
    print(f"  {feat:<28s} {row['phishtank']:>12.4f} {row['urlhaus']:>12.4f} {row['mean']:>12.4f}  {keep:>6s}")

selected = importance_df[importance_df["mean"] >= threshold].index.tolist()  # creates the list of selected feat
```

Figure 93 Feature Selection Threshold

Averages feature importance scores across both datasets, sorts the results, and selects features whose mean importance meets or exceeds the threshold.

```
    scaler  = StandardScaler()           # instantiates a StandardScaler that will norma
    X_tr_sc = scaler.fit_transform(X_train)  # fits the scaler on the training data and
    X_te_sc = scaler.transform(X_test)       # applies the scaler fitted on training da
    _eval("Logistic Regression",
          LogisticRegression(max_iter=1000, random_state=42, class_weight="balanced"),
          X_tr_sc, X_te_sc)  # passes the scaled training and test feature matrices to

    # Random Forest
    _eval("Random Forest",
          RandomForestClassifier(n_estimators=200, random_state=42,
                                 n_jobs=-1, class_weight="balanced"),  # instantiates Ra
          X_train, X_test)  # passes the unscaled training and test feature matrices to

    # Decision Tree
    _eval("Decision Tree (depth=10)",
          DecisionTreeClassifier(max_depth=10, min_samples_split=20,
                                 random_state=42, class_weight="balanced"),  # instantia
          X_train, X_test)  # passes the unscaled training and test feature matrices to
```

Figure 94 Model Evaluation

Trains and evaluates Logistic Regression, Random Forest, and Decision Tree models on either the full or reduced feature set.

```
280         r = {
281             "feature_set": label,       # stores the feature set label (Full or Reduced w
282             "train_on":    train_name,  # stores the name of the training dataset to iden
283             "test_on":     test_name,   # stores the name of the test dataset to identify
284             "model":       name,        # stores the name of the model being evaluated to
285             "accuracy":    round(accuracy_score(y_test, y_pred),  4),
286             "precision":   round(precision_score(y_test, y_pred, zero_division=0), 4),
287             "recall":      round(recall_score(y_test, y_pred,    zero_division=0), 4),
288             "f1_score":    round(f1_score(y_test, y_pred,        zero_division=0), 4),
289             "tn": int(cm[0][0]), "fp": int(cm[0][1]),  # extracts the true negative count
290             "fn": int(cm[1][0]), "tp": int(cm[1][1]),  # extracts the false negative coun
291             "n_features":  X_tr.shape[1],  # stores the number of features in the trainin
292         }
```

Figure 95 Result Dictionary

Stores the evaluation results for each model, including feature set type, train/test dataset names, performance metrics, confusion matrix values, and number of features used.

```
628     X_phish, y_phish = prepare(PHISHTANK_FILE)  # loads the PhishTan
629     X_url,   y_url   = prepare(URLHAUS_FILE)    # loads the URLhaus
630
631     # Align columns (both datasets share the same 24 features)
632     shared_cols = sorted(set(X_phish.columns) & set(X_url.columns))
633     X_phish = X_phish[shared_cols]  # reindexes the PhishTank featur
634     X_url   = X_url[shared_cols]    # reindexes the URLhaus feature
635
```

Figure 96 Column Alignment

Aligns the PhishTank and URLhaus feature matrices so both datasets use the same shared feature columns in the same order.

```
639     selected, importance_df = select_features(
640         X_phish, y_phish, X_url, y_url, threshold=threshold
641     )  # calls the feature selection function with both full datasets and th
642
643     if len(selected) == 0:
644         log.error("No features survived the threshold! Lower --threshold.")
645         sys.exit(1)  # exits with a non-zero status code if no features were
```

Figure 97 Feature Selection Exectution

Runs the feature selection process and stops execution if no features survive the threshold.

```
    X_tr_f, X_te_f, y_tr, y_te = train_test_split(
        X_full, y_full,
        test_size=test_size, random_state=42, stratify=y_full
    )  # splits the full feature dataset into 80/20 training and test s

    # -- Full feature set
    print(f"\n--- {label}  |  Full ({len(shared_cols)} features) ---")
    f_res = evaluate_models(
        X_tr_f, y_tr, X_te_f, y_te,
        label=f"Full ({len(shared_cols)})",
        train_name=label, test_name=label,
        results_list=all_results,
    )  # calls evaluate_models to train and evaluate all three classifi
    full_within[ds_name] = f_res  # stores the full feature set results
    for r in f_res:
        print(f"  {r['model']:<35s}  Acc={r['accuracy']:.4f}  "
              f"Prec={r['precision']:.4f}  Rec={r['recall']:.4f}  "
              f"F1={r['f1_score']:.4f}")  # prints a one-line summary o

    # -- Reduced feature set
    X_tr_r = X_tr_f[selected]  # creates the reduced training feature m
    X_te_r = X_te_f[selected]  # creates the reduced test feature matri
    print(f"\n--- {label}  |  Reduced ({len(selected)} features) ---")
    r_res = evaluate_models(
        X_tr_r, y_tr, X_te_r, y_te,
        label=f"Reduced ({len(selected)})",
        train_name=label, test_name=label,
        results_list=all_results,
    )  # calls evaluate_models to train and evaluate all three classifi
```

Figure 98 Within-Dataset Comparison
Performs within-dataset evaluation using both the full feature set and the reduced feature set selected in Iteration 2.

```
    f_res = evaluate_models(
        X_tr_full, y_tr, X_te_full, y_te,
        label=f"Full ({len(shared_cols)})",
        train_name=train_name, test_name=test_name,
        results_list=all_results,
    )  # calls evaluate_models to train on the full training dataset
    full_cross[key] = f_res  # stores the full feature set cross-eval
    for r in f_res:
        print(f"  {r['model']:<35s}  Acc={r['accuracy']:.4f}  "
              f"Prec={r['precision']:.4f}  Rec={r['recall']:.4f}  "
              f"F1={r['f1_score']:.4f}")  # prints a one-line summary

    # Reduced
    X_tr_r = X_tr_full[selected]  # creates the reduced training feat
    X_te_r = X_te_full[selected]  # creates the reduced test feature
    print(f"\n--- Cross: Train {train_name} → Test {test_name}"
          f"  |  Reduced ({len(selected)} features) ---")  # prints a
    r_res = evaluate_models(
        X_tr_r, y_tr, X_te_r, y_te,
        label=f"Reduced ({len(selected)})",
        train_name=train_name, test_name=test_name,
        results_list=all_results,
    )  # calls evaluate_models to train on the reduced training datas
```

Figure 99 Cross-Dataset Comparison
Evaluates cross-dataset performance using both the full feature set and the reduced feature set for each train-test direction.

```
    results_df   = pd.DataFrame(all_results)  # converts the accumulated list c
    results_path = os.path.join(OUTPUT_DIR, "feature_selection_results.csv")  #
    results_df.to_csv(results_path, index=False)  # saves the results DataFrame
    log.info("Results saved -> %s", os.path.abspath(results_path))  # logs the

    # Save importance table
    imp_path = os.path.join(OUTPUT_DIR, "feature_importances_comparison.csv")
    importance_df.to_csv(imp_path)  # saves the feature importance comparison D
```

Figure 100 Results and Importance Export
Saves the full feature-selection evaluation results and the feature-importance comparison table to CSV files.

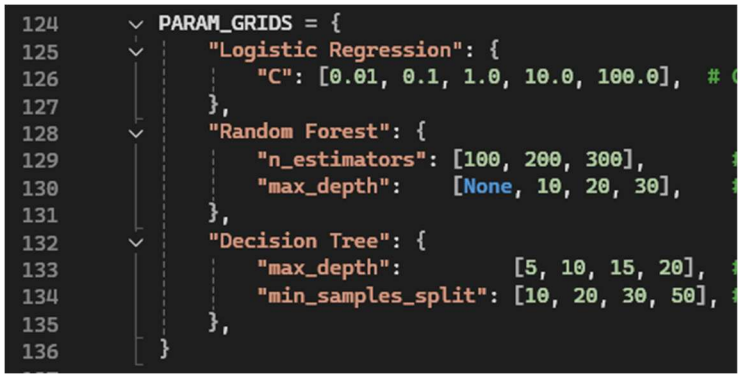
A.10 Hyperparameter Tuning Evaluation Module (hyperparameter_tuning_eval.py)

Section Description:

This module implements Iteration 3 by applying GridSearchCV hyperparameter tuning to the reduced 13-feature set selected in Iteration 2. It tunes Logistic Regression, Random Forest, and Decision Tree models using 5-fold stratified cross-validation, evaluates tuned models within-dataset and cross-dataset, and saves comparison results, best parameter configurations, and dashboards.

```
REDUCED_FEATURES = [
    "path_length",       #
    "num_slashes",       #
    "url_length",        #
    "num_dots",          #
    "subdomain_depth",   #
    "has_https",         #
    "num_digits",        #
    "url_entropy",       #
    "digit_ratio",       #
    "hostname_length",   #
    "special_char_ratio",#
    "hostname_entropy",  #
    "has_ip",            #
]
```

Figure 101 Reduced Feature Set
Defines the 13-feature subset retained from Iteration 2 and used for all Iteration 3 tuning and evaluation.



```
PARAM_GRIDS = {
    "Logistic Regression": {
        "C": [0.01, 0.1, 1.0, 10.0, 100.0],  # (
    },
    "Random Forest": {
        "n_estimators": [100, 200, 300],
        "max_depth":    [None, 10, 20, 30],
    },
    "Decision Tree": {
        "max_depth":         [5, 10, 15, 20],
        "min_samples_split": [10, 20, 30, 50],
    },
}
```

Figure 102 Parameter Grids

Defines the hyperparameter search spaces used by GridSearchCV for each classifier.

```
ITER1_WITHIN = {
    "features_phishtank_tranco_dataset": [
        {"model": "Logistic Regression",      "accuracy": 0.9972, "precision": 0.9990, "recall": 0.9955, "f1_score": 0.9972, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
        {"model": "Random Forest",            "accuracy": 0.9975, "precision": 0.9980, "recall": 0.9970, "f1_score": 0.9975, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
        {"model": "Decision Tree (depth=10)", "accuracy": 0.9980, "precision": 0.9985, "recall": 0.9975, "f1_score": 0.9980, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
    ],
    "features_urlhaus_tranco_dataset": [
        {"model": "Logistic Regression",      "accuracy": 0.9998, "precision": 1.0000, "recall": 0.9995, "f1_score": 0.9997, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
        {"model": "Random Forest",            "accuracy": 0.9998, "precision": 1.0000, "recall": 0.9995, "f1_score": 0.9997, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
        {"model": "Decision Tree (depth=10)", "accuracy": 1.0000, "precision": 1.0000, "recall": 1.0000, "f1_score": 1.0000, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
    ],
}

ITER1_CROSS = {
    ("PhishTank", "URLhaus"): [
        {"model": "Logistic Regression",      "accuracy": 0.9991, "precision": 0.9989, "recall": 0.9993, "f1_score": 0.9991, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
        {"model": "Random Forest",            "accuracy": 1.0000, "precision": 0.9999, "recall": 1.0000, "f1_score": 1.0000, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
        {"model": "Decision Tree (depth=10)", "accuracy": 0.9996, "precision": 0.9991, "recall": 1.0000, "f1_score": 0.9996, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
    ],
    ("URLhaus", "PhishTank"): [
        {"model": "Logistic Regression",      "accuracy": 0.9738, "precision": 1.0000, "recall": 0.9475, "f1_score": 0.9730, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
        {"model": "Random Forest",            "accuracy": 0.9831, "precision": 1.0000, "recall": 0.9662, "f1_score": 0.9828, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
        {"model": "Decision Tree (depth=10)", "accuracy": 0.9905, "precision": 1.0000, "recall": 0.9810, "f1_score": 0.9904, "n_features": 24, "iteration": "Iteration 1 (Full 24)"},
    ],
}
```

Figure 103 Baseline Comparison Data

Stores prior Iteration 1 and Iteration 2 results so Iteration 3 tuned models can be compared against both earlier configurations.

```
    df        = load_dataset(path)       # loads the CSV file into a DataFrame using the load
    label_col = find_label_column(df)    # identifies the name of the label column using the
    if label_col is None:                # checks whether no recognizable label column was fo
        log.error("No label column found in %s", path)  # logs an error identifying the file
        sys.exit(1)                      # exits with a non-zero status code to indicate a fa

    y    = normalize_labels(df[label_col])   # normalizes the label column values to binary i
    mask = y.isin([0, 1])                    # creates a boolean mask that is True for rows w
    df   = df[mask].reset_index(drop=True)   # applies the boolean mask to retain only rows w
    y    = y[mask].reset_index(drop=True).astype(int)  # applies the same mask to the label s

    feature_cols = [c for c in df.columns if c not in (label_col, "url")]  # builds a list of
    X = df[feature_cols].select_dtypes(include=["number"])  # creates the feature matrix X by

    log.info(
        "Loaded %s: %d rows, %d features | malicious: %d (%.1f%%)  legitimate: %d (%.1f%%)",
        os.path.basename(path), len(X), len(X.columns),  # logs the base filename, total row
        int(y.sum()), 100 * y.mean(),                    # logs the count and percentage of m
        int((y == 0).sum()), 100 * (1 - y.mean()),       # logs the count and percentage of l
    )
    return X, y  # returns the feature matrix X and label vector y as a tuple
```

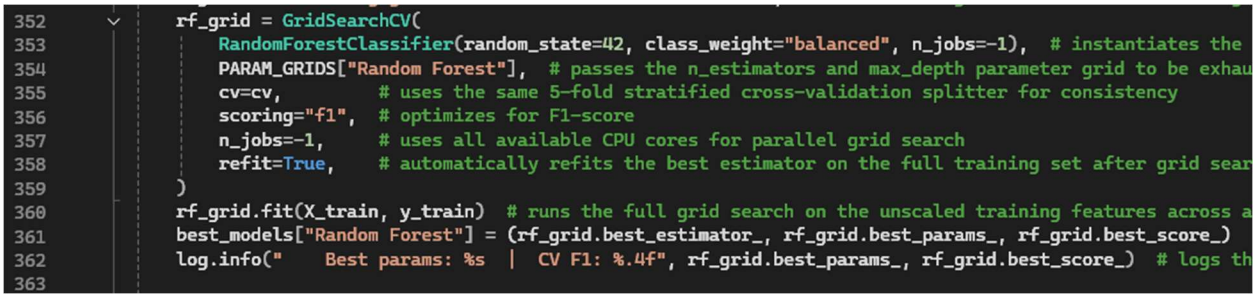
Figure 104 Dataset Preparation
Loads a feature CSV, identifies the label column, normalizes labels to binary values, filters invalid labels, and returns the numeric feature matrix and target vector.

```
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    best_models = {}  # initializes an empty dictionary to store the
```

Figure 105 Grid Search Setup
Creates a 5-fold stratified cross-validation object and initializes the dictionary used to store the best model configurations.

```
X_tr_sc = scaler.transform(X_train)  # transforms the training features using the already-fitted scaler wit

lr_grid = GridSearchCV(
    LogisticRegression(random_state=42, class_weight="balanced", max_iter=2000),  # instantiates the base L
    PARAM_GRIDS["Logistic Regression"],  # passes the C parameter grid defining the regularization strength
    cv=cv,            # uses the 5-fold stratified cross-validation splitter defined above
    scoring="f1",     # optimizes for F1-score consistent with the project's evaluation methodology and emp
    n_jobs=-1,        # uses all available CPU cores for parallel grid search to reduce computation time
    refit=True,       # automatically refits the best estimator on the full training set after grid search
)
lr_grid.fit(X_tr_sc, y_train)  # runs the full grid search on the scaled training features, training and ev
best_models["Logistic Regression"] = (lr_grid.best_estimator_, lr_grid.best_params_, lr_grid.best_score_)
log.info("    Best params: %s  |  CV F1: %.4f", lr_grid.best_params_, lr_grid.best_score_)  # logs the best
```

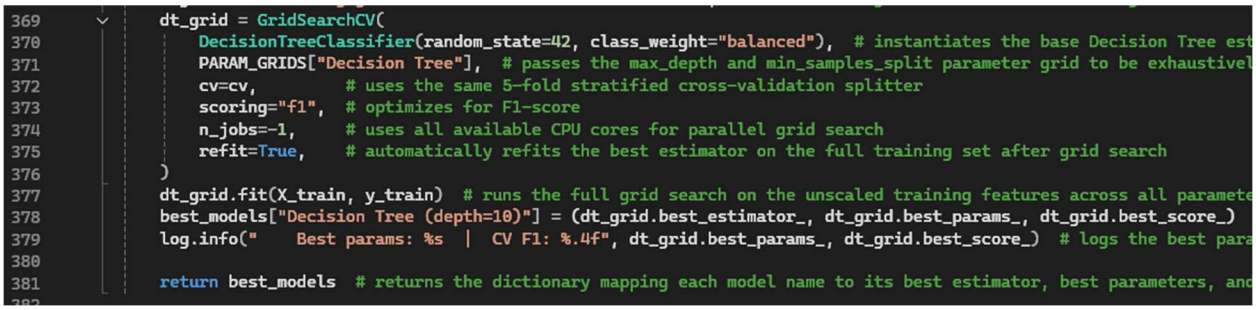
Figure 106 Logistic Regression Tuning
Transforms the reduced training features using a scaler fitted on training data and runs GridSearchCV for Logistic Regression using F1-score as the optimization metric.



```
352     rf_grid = GridSearchCV(
353         RandomForestClassifier(random_state=42, class_weight="balanced", n_jobs=-1),  # instantiates the
354         PARAM_GRIDS["Random Forest"],  # passes the n_estimators and max_depth parameter grid to be exhau
355         cv=cv,          # uses the same 5-fold stratified cross-validation splitter for consistency
356         scoring="f1",  # optimizes for F1-score
357         n_jobs=-1,      # uses all available CPU cores for parallel grid search
358         refit=True,     # automatically refits the best estimator on the full training set after grid sear
359     )
360     rf_grid.fit(X_train, y_train)  # runs the full grid search on the unscaled training features across a
361     best_models["Random Forest"] = (rf_grid.best_estimator_, rf_grid.best_params_, rf_grid.best_score_)
362     log.info("    Best params: %s  |  CV F1: %.4f", rf_grid.best_params_, rf_grid.best_score_)  # logs th
363
```

Figure 107 Random Forest Tuning

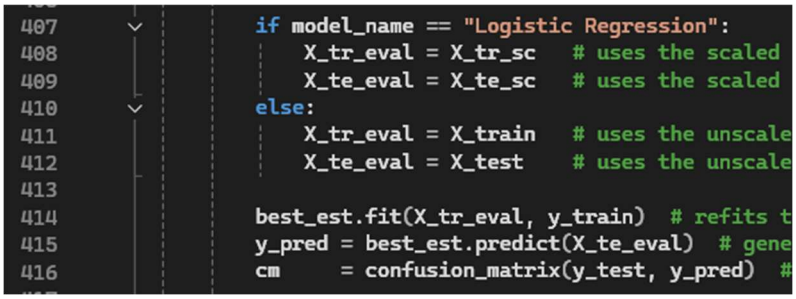
Runs GridSearchCV for Random Forest using the reduced, unscaled feature set and searches over estimator count and tree depth.



```
369     dt_grid = GridSearchCV(
370         DecisionTreeClassifier(random_state=42, class_weight="balanced"),  # instantiates the base Decision Tree est
371         PARAM_GRIDS["Decision Tree"],  # passes the max_depth and min_samples_split parameter grid to be exhaustivel
372         cv=cv,          # uses the same 5-fold stratified cross-validation splitter
373         scoring="f1",  # optimizes for F1-score
374         n_jobs=-1,      # uses all available CPU cores for parallel grid search
375         refit=True,     # automatically refits the best estimator on the full training set after grid search
376     )
377     dt_grid.fit(X_train, y_train)  # runs the full grid search on the unscaled training features across all paramete
378     best_models["Decision Tree (depth=10)"] = (dt_grid.best_estimator_, dt_grid.best_params_, dt_grid.best_score_)
379     log.info("    Best params: %s  |  CV F1: %.4f", dt_grid.best_params_, dt_grid.best_score_)  # logs the best para
380
381     return best_models  # returns the dictionary mapping each model name to its best estimator, best parameters, and
```

Figure 108 Decision Tree Tuning

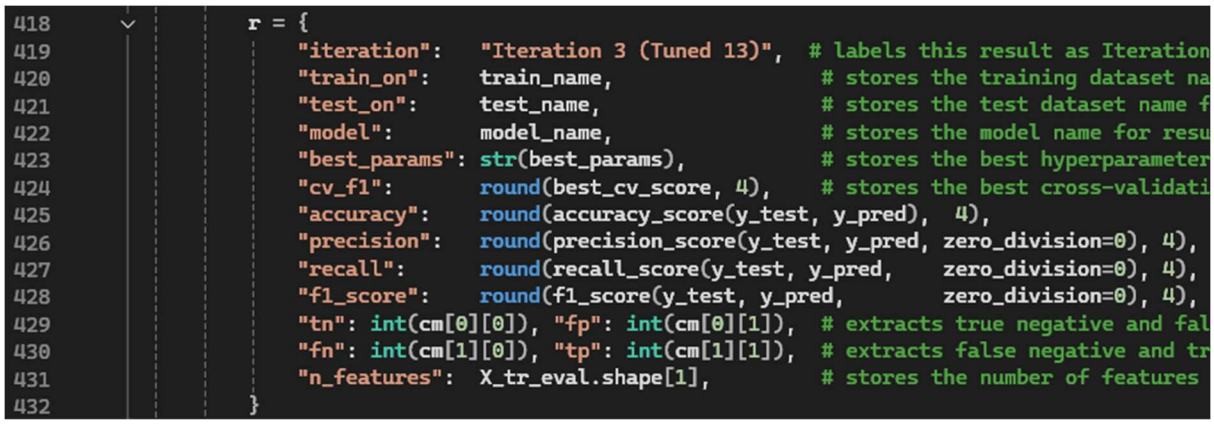
Runs GridSearchCV for Decision Tree using the reduced, unscaled feature set and searches over maximum depth and minimum samples required for a split.



```
407         if model_name == "Logistic Regression":
408             X_tr_eval = X_tr_sc   # uses the scaled
409             X_te_eval = X_te_sc   # uses the scaled
410         else:
411             X_tr_eval = X_train   # uses the unscale
412             X_te_eval = X_test    # uses the unscale
413
414         best_est.fit(X_tr_eval, y_train)  # refits t
415         y_pred = best_est.predict(X_te_eval)  # gene
416         cm     = confusion_matrix(y_test, y_pred)  #
```

Figure 109 Tuned Model Evaluation

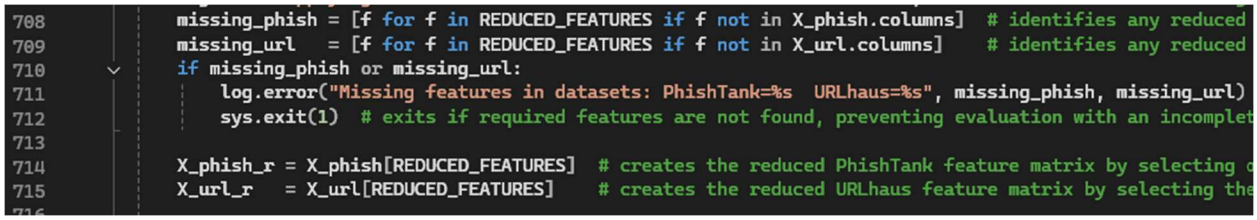
Applies the appropriate scaled or unscaled feature matrix for each tuned model, refits the best estimator on the training data, predicts on the test data, and computes the confusion matrix.



```
418         r = {
419             "iteration":   "Iteration 3 (Tuned 13)",  # labels this result as Iteration
420             "train_on":    train_name,                # stores the training dataset na
421             "test_on":     test_name,                 # stores the test dataset name f
422             "model":       model_name,                # stores the model name for resu
423             "best_params": str(best_params),          # stores the best hyperparameter
424             "cv_f1":       round(best_cv_score, 4),   # stores the best cross-validati
425             "accuracy":    round(accuracy_score(y_test, y_pred),  4),
426             "precision":   round(precision_score(y_test, y_pred, zero_division=0), 4),
427             "recall":      round(recall_score(y_test, y_pred,    zero_division=0), 4),
428             "f1_score":    round(f1_score(y_test, y_pred,        zero_division=0), 4),
429             "tn": int(cm[0][0]), "fp": int(cm[0][1]),  # extracts true negative and fal
430             "fn": int(cm[1][0]), "tp": int(cm[1][1]),  # extracts false negative and tr
431             "n_features":  X_tr_eval.shape[1],         # stores the number of features
432         }
```

Figure 110 Tuned Result Record

Stores Iteration 3 tuned model results, including best parameters, cross-validation F1-score, final evaluation metrics, confusion matrix values, and feature count.



```
708     missing_phish = [f for f in REDUCED_FEATURES if f not in X_phish.columns]  # identifies any reduced
709     missing_url   = [f for f in REDUCED_FEATURES if f not in X_url.columns]    # identifies any reduced
710     if missing_phish or missing_url:
711         log.error("Missing features in datasets: PhishTank=%s  URLhaus=%s", missing_phish, missing_url)
712         sys.exit(1)  # exits if required features are not found, preventing evaluation with an incomplet
713
714     X_phish_r = X_phish[REDUCED_FEATURES]  # creates the reduced PhishTank feature matrix by selecting
715     X_url_r   = X_url[REDUCED_FEATURES]    # creates the reduced URLhaus feature matrix by selecting the
```

Figure 111 Reduced Feature Validation

Checks that all 13 reduced features exist in both feature datasets before tuning begins.

```
733         X_tr, X_te, y_tr, y_te = train_test_split(
734             X_full_r, y_full,
735             test_size=test_size, random_state=42, stratify=y_full
736         )  # splits the reduced feature dataset into 80/20 training and test sets
737
738         scaler = StandardScaler()       # instantiates a new StandardScaler for t
739         scaler.fit(X_tr)                # fits the scaler exclusively on the trai
740
741         best_models = run_grid_search(X_tr, y_tr, ds_label, scaler)  # runs grid
742
743         for model_name, (best_est, best_params, best_cv) in best_models.items():
744             best_params_log.append({
745                 "dataset":    ds_label,       # records which dataset this best
746                 "model":      model_name,     # records the model name
747                 "best_params": str(best_params),  # records the best parameter di
748                 "cv_f1":      round(best_cv, 4),  # records the best cross-valida
749             })  # appends a record of the best parameters for this model and data
750
751         log.info("\nEvaluating tuned models: %s within-dataset...", ds_label)  #
752         print(f"\n{'='*60}")
753         print(f"WITHIN-DATASET EVALUATION — {ds_label} — Iteration 3 (Tuned)")
754         print(f"{'='*60}")
755
756         i3_results = evaluate_tuned_models(
757             X_tr, y_tr, X_te, y_te,
758             best_models, ds_label, ds_label,
759             all_results, scaler
760         )  # evaluates all three tuned models on the within-dataset test set and
761         within_i3[ds_name] = i3_results  # stores the Iteration 3 within-dataset
```

Figure 112 Within-Dataset Tuning Workflow

Splits each reduced-feature dataset into stratified training and test sets, fits the scaler on training data only, runs grid search, records best parameters, and evaluates tuned models within the same dataset.

```
783         scaler_cross = StandardScaler()    # instantiates a new StandardScaler for this cross
784         scaler_cross.fit(X_tr_full)        # fits the scaler exclusively on the training data
785
786         log.info("  Running grid search on training data (%s)...", train_name)  # logs that g
787         best_models_cross = run_grid_search(X_tr_full, y_tr, train_name, scaler_cross)  # run
788
789         print(f"\n{'='*60}")
790         print(f"CROSS-DATASET: Train {train_name} → Test {test_name} — Iteration 3 (Tuned)")
791         print(f"{'='*60}")
792
793         i3_cross_results = evaluate_tuned_models(
794             X_tr_full, y_tr, X_te_full, y_te,
795             best_models_cross, train_name, test_name,
796             all_results, scaler_cross
797         )  # evaluates all three tuned models on the cross-dataset test set and appends resul
798         cross_i3[key] = i3_cross_results  # stores the Iteration 3 cross-dataset results for
```

Figure 113 Cross-Dataset Tuning Workflow

Performs cross-dataset tuning by fitting the scaler and running grid search only on the training dataset, then evaluating tuned models on the separate test dataset.

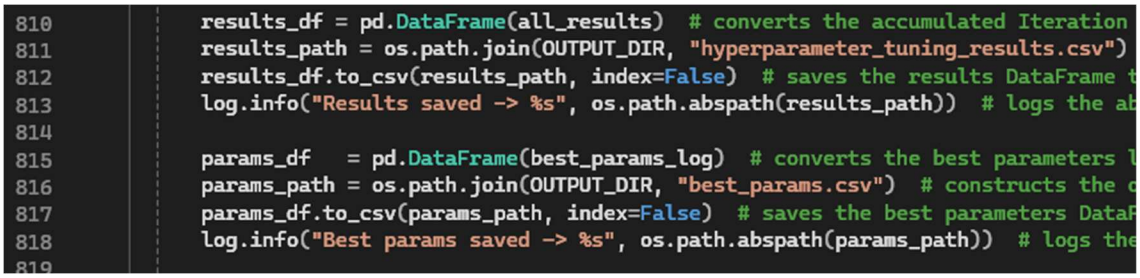


Figure 114 Results and Parameters Export
Saves Iteration 3 evaluation and best hyperparameter configurations to CSV files.

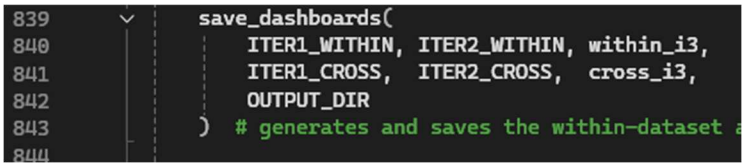


Figure 115 Dashboard Generation
Generates and saves the Iteration 3 within-dataset and cross-dataset comparison dashboards.

A.11 Graphical User Interface Module (phishing_gui.py)

Section Description:

This module provides a tkinter-based graphical interface for classifying URLs without using the command line. It wraps the same feature extraction and inference logic used in predict.py, loads the trained model artifacts, displays individual model predictions, and presents the final majority-vote result for demonstration and usability purposes.

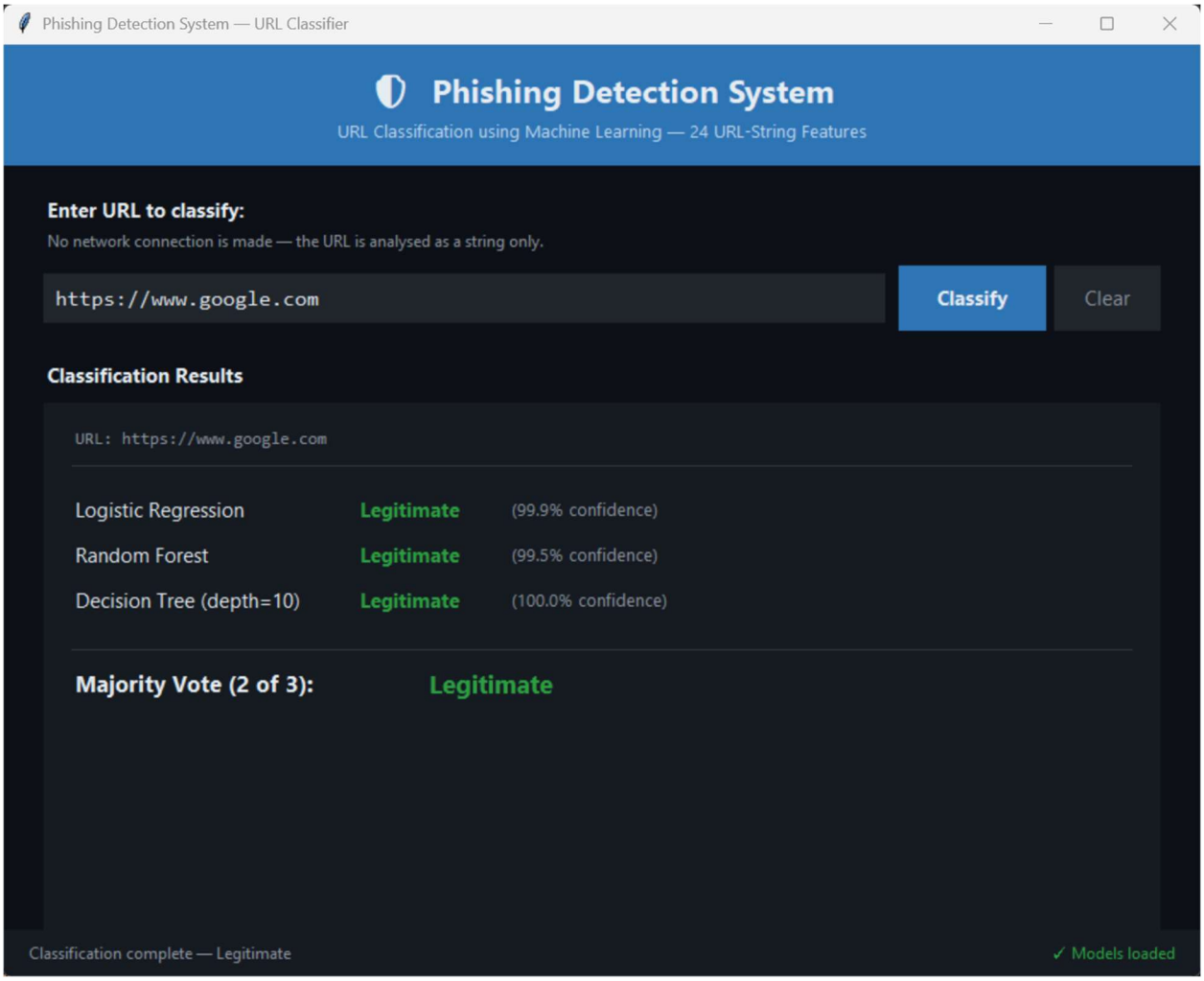


Figure 116 GUI Interface
This screenshot shows the implemented phishing detection GUI, including the URL input field, Classify button, Clear button, model result panel, majority-vote output area, and model-loading status indicator.

```
43          OUTPUT_DIR  = "outputs"
44          MODEL_LR    = os.path.join(OUTPUT_DIR, "model_lr.pkl")
45          MODEL_RF    = os.path.join(OUTPUT_DIR, "model_rf.pkl")
46          MODEL_DT    = os.path.join(OUTPUT_DIR, "model_dt.pkl")
47          SCALER_PATH = os.path.join(OUTPUT_DIR, "scaler.pkl")
```

Figure 117 Model Artifact Paths
Defines the saved model and scaler file paths required for GUI-based inference.

```
53        BG_DARK      = "#0D1117"
54        BG_PANEL     = "#161B22"
55        BG_INPUT     = "#21262D"
56        ACCENT_BLUE  = "#2E75B6"
57        TEXT_WHITE   = "#E6EDF3"
58        TEXT_GREY    = "#8B949E"
59        GREEN        = "#2EA043"
60        RED          = "#DA3633"
61        YELLOW       = "#D29922"
62        BORDER       = "#30363D"
63
```

Figure 118 GUI Color Scheme
Defines the dark visual theme used by the GUI, including background, panel, input, status, and verdict colors.

```
87        url    = str(url).strip()
88        parsed = urlparse(url if "://" in url else "http://" + url)
89        hostname = parsed.hostname or ""
90        path     = parsed.path     or ""
91        query    = parsed.query    or ""
92
93        url_length      = len(url)
94        hostname_length = len(hostname)
95        path_length     = len(path)
96        query_length    = len(query)
97        num_dots        = url.count(".")
98        num_hyphens     = url.count("-")
99        num_underscores = url.count("_")
100       num_slashes     = url.count("/")
101       num_qmarks      = url.count("?")
102       num_equals      = url.count("=")
103       num_ampersands  = url.count("&")
104       num_at          = url.count("@")
105       num_percent     = url.count("%")
106       num_digits      = sum(c.isdigit() for c in url)
107       has_https       = int(parsed.scheme == "https")
108       has_ip          = int(bool(
109           hostname and
110           all(part.isdigit() and 0 <= int(part) <= 255
111               for part in hostname.split("."))
112           and hostname.count(".") == 3
113       ))
114       has_port        = int(bool(parsed.port))
115       has_www         = int(hostname.startswith("www."))
116       has_at_in_url   = int("@" in url)
117       subdomain_depth = max(0, hostname.count(".") - 1) if hostname else 0
118       url_entropy      = _shannon_entropy(url)
119       hostname_entropy = _shannon_entropy(hostname)
120       url_len_safe     = url_length if url_length > 0 else np.nan
121       digit_ratio      = num_digits / url_len_safe
122       special_char_ratio = (
123           (num_dots + num_hyphens + num_underscores + num_at +
124            num_percent + num_qmarks + num_equals + num_ampersands)
125           / url_len_safe
126       )
```

Figure 119 GUI Feature Extraction
Extracts the same URL-string components used during command-line inference, including hostname, path, query, length, character-count, structural, entropy, and ratio features.

```
    features = {
        "url_length": url_length, "hostname_length": hostname_length,
        "path_length": path_length, "query_length": query_length,
        "num_dots": num_dots, "num_hyphens": num_hyphens,
        "num_underscores": num_underscores, "num_slashes": num_slashes,
        "num_qmarks": num_qmarks, "num_equals": num_equals,
        "num_ampersands": num_ampersands, "num_at": num_at,
        "num_percent": num_percent, "num_digits": num_digits,
        "has_https": has_https, "has_ip": has_ip,
        "has_port": has_port, "has_www": has_www,
        "has_at_in_url": has_at_in_url, "subdomain_depth": subdomain_depth,
        "url_entropy": url_entropy, "hostname_entropy": hostname_entropy,
        "digit_ratio": digit_ratio, "special_char_ratio": special_char_ratio,
    }
    df = pd.DataFrame([features])
    df = df.fillna(0)
    return df
```

Figure 120 Feature DataFrame Assembly
Assembles the extracted URL-string values into a single-row DataFrame matching the trained model feature schema.

```
    required = {                                # map display names to fil
        "Logistic Regression": MODEL_LR,
        "Random Forest":       MODEL_RF,
        "Decision Tree":       MODEL_DT,
        "StandardScaler":      SCALER_PATH,
    }
    missing = [name for name, path in required.items()
               if not os.path.exists(path)]  # collect names of any miss

    if missing:                                 # if any files are missing,
        raise FileNotFoundError(
            f"Missing model files: {', '.join(missing)}\n\n"
            "Please run train_models.py first to generate these files."
        )

    with open(MODEL_LR,    "rb") as f: lr     = pickle.load(f)  # load L
    with open(MODEL_RF,    "rb") as f: rf     = pickle.load(f)  # load R
    with open(MODEL_DT,    "rb") as f: dt     = pickle.load(f)  # load D
    with open(SCALER_PATH, "rb") as f: scaler = pickle.load(f)  # load S

    return lr, rf, dt, scaler
```

Figure 121 Model Loading

Checks that all required trained model artifacts exist and loads the Logistic Regression, Random Forest, Decision Tree, and StandardScaler objects for GUI inference.

```
193             self.root = root                        # store reference to the
194             self.root.title("Phishing Detection System – URL Classifier")
195             self.root.configure(bg=BG_DARK)         # set the main window bac
196             self.root.resizable(True, True)         # allow the window to be
197
198             # Set a minimum window size to prevent widgets from being hidd
199             self.root.minsize(750, 600)             # minimum width 750px, mi
200
201             # Try to set the window to open at a reasonable size
202             self.root.geometry("900x700")           # initial window size: 90
203
204             # Load model artifacts at startup – show error and exit if mis
205             try:
206                 self.lr, self.rf, self.dt, self.scaler = load_models()  #
207                 self.models_loaded = True           # flag that models loaded
208             except FileNotFoundError as e:
209                 self.models_loaded = False          # flag that models failed
210                 messagebox.showerror(               # show a popup error dial
211                     "Model Files Not Found", str(e)
212                 )
213
214             # Build all the GUI widgets
215             self._build_header()                    # title bar at the top of
216             self._build_input_section()             # URL input field and Cla
217             self._build_results_section()           # results panel showing p
218             self._build_footer()                    # status bar at the botto
219
220             # Set initial focus to the URL input field so user can type im
221             self.url_entry.focus_set()
222
223             # Bind the Enter key to trigger classification – more convenie
224             self.root.bind("<Return>", lambda event: self._classify())
```

Figure 122 GUI Window Initialization

Creates the main application window, applies the dark background theme, sets window sizing behavior, loads model artifacts at startup, builds the GUI sections, and binds the Enter key to classification.

```
278            self.url_var = tk.StringVar()        # StringVar to hol
279            self.url_entry = tk.Entry(           # create the text
280                entry_row,
281                textvariable=self.url_var,       # bind the entry t
282                font=("Consolas", 12),           # monospace font a
283                bg=BG_INPUT,                     # dark input backg
284                fg=TEXT_WHITE,                   # white text
285                insertbackground=TEXT_WHITE,     # white cursor in
286                relief=tk.FLAT,                  # flat border styl
287                bd=8,                            # border/padding w
288            )
289            self.url_entry.pack(                 # place the entry
290                side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 10)
291            )
292
293            # Classify button
294            self.classify_btn = tk.Button(       # create the class
295                entry_row,
296                text="  Classify  ",             # button label wit
297                font=("Segoe UI", 11, "bold"),
298                bg=ACCENT_BLUE,                  # blue button matc
299                fg=TEXT_WHITE,                   # white text
300                activebackground="#1A5490",      # slightly darker
301                activeforeground=TEXT_WHITE,
302                relief=tk.FLAT,                  # flat style - mod
303                padx=15, pady=8,                 # internal padding
304                cursor="hand2",                  # change cursor to
305                command=self._classify,          # call _classify()
306            )
307            self.classify_btn.pack(side=tk.LEFT) # place the button
```

Figure 123 Input Controls
Creates the URL entry field, Classify button, and Clear button used in the GUI.

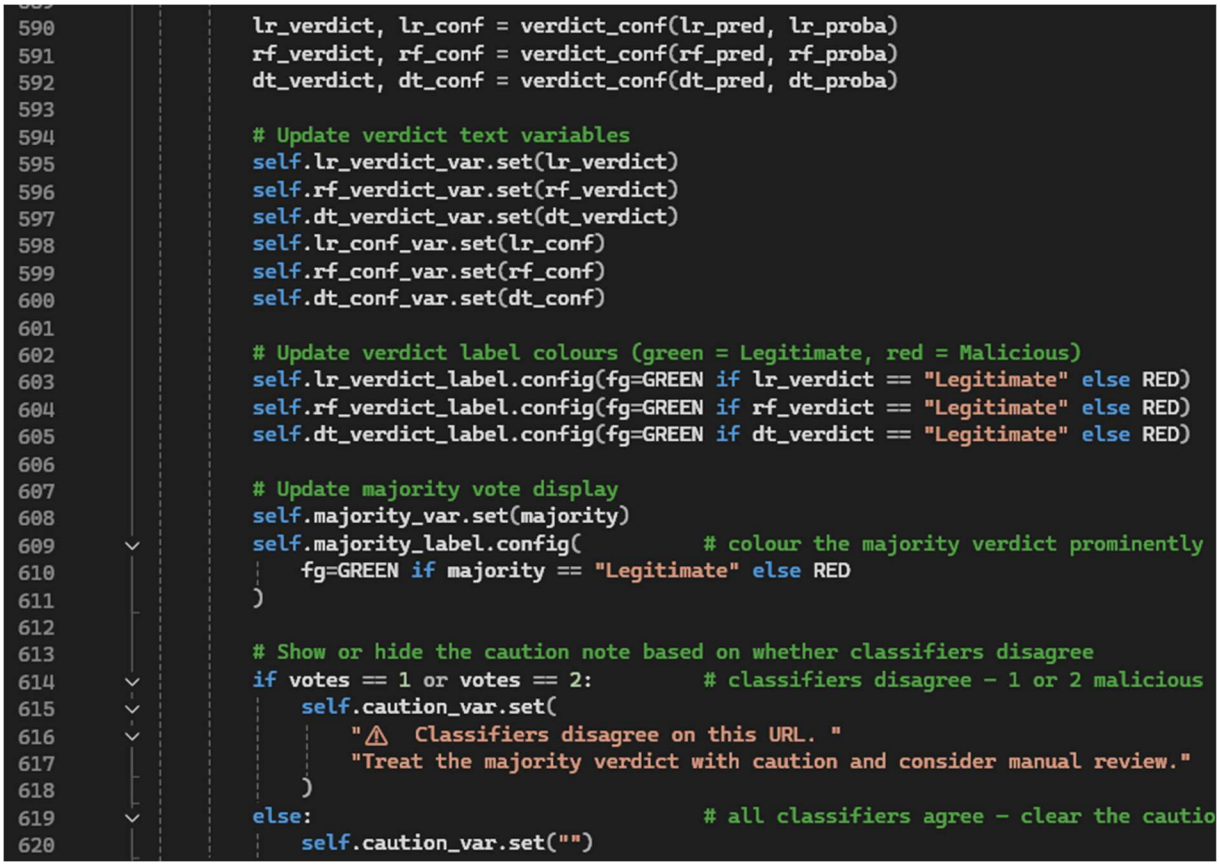
```
383            self.lr_verdict_var    = tk.StringVar()  # Logistic Regression verdict text
384            self.rf_verdict_var    = tk.StringVar()  # Random Forest verdict text
385            self.dt_verdict_var    = tk.StringVar()  # Decision Tree verdict text
386            self.lr_conf_var       = tk.StringVar()  # LR confidence text
387            self.rf_conf_var       = tk.StringVar()  # RF confidence text
388            self.dt_conf_var       = tk.StringVar()  # DT confidence text
389            self.lr_verdict_label  = None            # will hold the Label widget for colour
390            self.rf_verdict_label  = None
391            self.dt_verdict_label  = None
392
393            # Build one row per model
394            model_rows = [
395                ("Logistic Regression",     self.lr_verdict_var, self.lr_conf_var, "lr"),
396                ("Random Forest",           self.rf_verdict_var, self.rf_conf_var, "rf"),
397                ("Decision Tree (depth=10)", self.dt_verdict_var, self.dt_conf_var, "dt"),
398            ]
```

Figure 124 Results Panel
Builds the results panel where the GUI displays the classified URL, individual model verdicts, confidence scores, majority-vote result, and disagreement warning.

```
519             url = self.url_var.get().strip()     # get the URL text from the input field
520
521             if not url:                          # if the input field is empty, show a w
522                 messagebox.showwarning("No URL", "Please enter a URL to classify.")
523                 return
524
525             # Update status bar to show classification is in progress
526             self.status_var.set("Classifying...")
527             self.root.update_idletasks()         # force the GUI to refresh so the statu
528
529             try:
530                 # Extract 24 URL-string features from the input URL
531                 features_df     = extract_features_single(url)
532
533                 # Apply the StandardScaler (transform only - no re-fitting) for Logistic
534                 features_scaled = self.scaler.transform(features_df)
535
536                 # Run all three classifiers
537                 lr_pred  = self.lr.predict(features_scaled)[0]           # LR prediction
538                 lr_proba = self.lr.predict_proba(features_scaled)[0]     # LR class prob
539
540                 rf_pred  = self.rf.predict(features_df)[0]               # RF prediction
541                 rf_proba = self.rf.predict_proba(features_df)[0]         # RF class prob
542
543                 dt_pred  = self.dt.predict(features_df)[0]               # DT prediction
544                 dt_proba = self.dt.predict_proba(features_df)[0]         # DT class prob
545
546                 # Compute majority vote
547                 votes    = lr_pred + rf_pred + dt_pred                   # sum of binary
548                 majority = "Malicious" if votes >= 2 else "Legitimate"   # majority verd
549
```

Figure 125 GUI Classification Logic

Retrieves the input URL, extracts features, scales only the Logistic Regression input, runs all three classifiers, computes the majority vote, and updates the GUI output.



```
590            lr_verdict, lr_conf = verdict_conf(lr_pred, lr_proba)
591            rf_verdict, rf_conf = verdict_conf(rf_pred, rf_proba)
592            dt_verdict, dt_conf = verdict_conf(dt_pred, dt_proba)
593
594            # Update verdict text variables
595            self.lr_verdict_var.set(lr_verdict)
596            self.rf_verdict_var.set(rf_verdict)
597            self.dt_verdict_var.set(dt_verdict)
598            self.lr_conf_var.set(lr_conf)
599            self.rf_conf_var.set(rf_conf)
600            self.dt_conf_var.set(dt_conf)
601
602            # Update verdict label colours (green = Legitimate, red = Malicious)
603            self.lr_verdict_label.config(fg=GREEN if lr_verdict == "Legitimate" else RED)
604            self.rf_verdict_label.config(fg=GREEN if rf_verdict == "Legitimate" else RED)
605            self.dt_verdict_label.config(fg=GREEN if dt_verdict == "Legitimate" else RED)
606
607            # Update majority vote display
608            self.majority_var.set(majority)
609            self.majority_label.config(          # colour the majority verdict prominently
610                fg=GREEN if majority == "Legitimate" else RED
611            )
612
613            # Show or hide the caution note based on whether classifiers disagree
614            if votes == 1 or votes == 2:         # classifiers disagree - 1 or 2 malicious
615                self.caution_var.set(
616                    "⚠  Classifiers disagree on this URL. "
617                    "Treat the majority verdict with caution and consider manual review."
618                )
619            else:                                # all classifiers agree - clear the cautio
620                self.caution_var.set("")
```

Figure 126 Result Update Logic

Updates the GUI with each model's verdict and confidence score, colors verdict labels based on classification, displays the majority-vote result, and shows a caution messages when models disagree.

```
647     def main():
648         """
649         Create the tkinter root window and launch the application.
650         Must be run from the project root directory so that the
651         outputs/ path resolves correctly to find the .pkl files.
652         """
653         root = tk.Tk()                          # create the main t
654
655         # Set the window icon if the icon file exists (optional - wo
656         try:
657             root.iconbitmap("icon.ico")         # set a custom wind
658         except Exception:
659             pass                                # silently skip if
660
661         app = PhishingDetectorApp(root)         # create the applic
662         root.mainloop()                         # start the tkinter
663
664
665     if __name__ == "__main__":
666         main()                                  # run the GUI when
667
```

Figure 127 GUI Entry Point

Creates the tkinter root window, initializes the phishing detection application, and starts the GUI event loop.

APPENDIX III: System Architecture and Design Diagrams

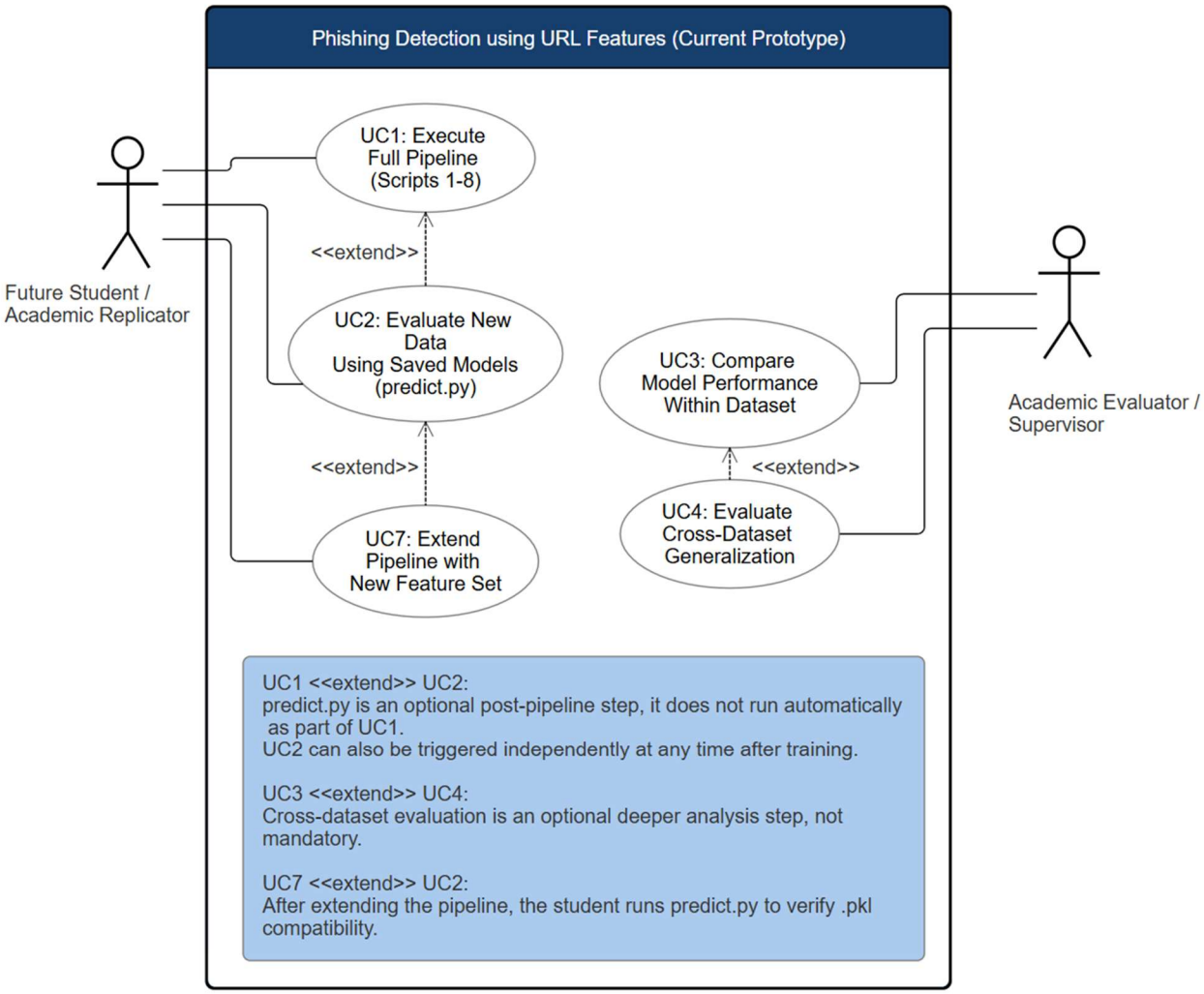


Figure 128 UML Use Case Diagram- Current Prototype
Use case model illustrating the interactions between the Future Student/Academic Replicator and Academic Evaluator/Supervisor actors and the four primary use cases supported by the current prototype, including full pipeline execution, saved model inference, within-dataset performance comparison, and cross-dataset generalization evaluation.

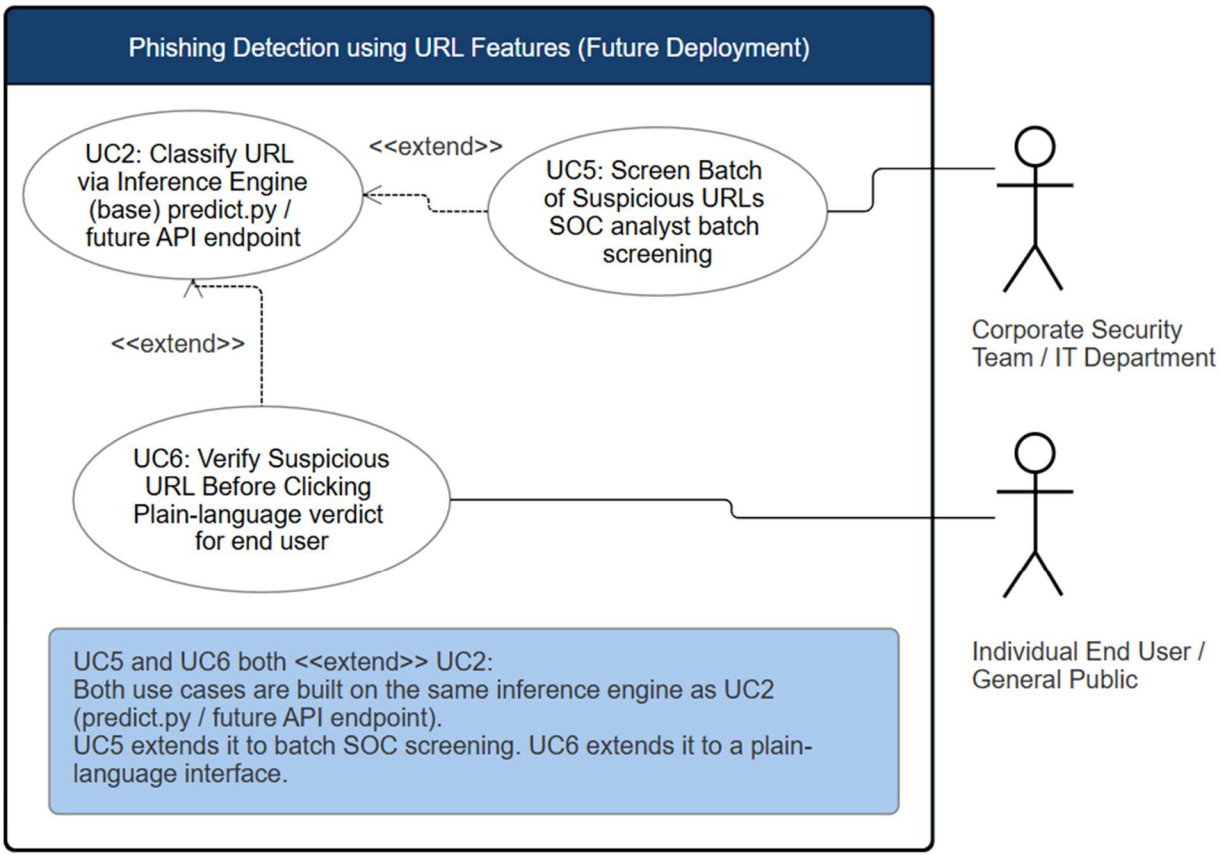


Figure 129 UML Use Case Diagram- Future Deployment
Use case model illustrating two forward-looking deployment scenarios extending the inference engine to support corporate security team batch URL screening through a future API endpoint or SIEM integration. Additionally, a standalone downloadable application or browser extension enabling individual end users to verify suspicious URLs through a plain-language safety verdict interface, without requiring command-line interaction or technical knowledge. Both use cases build upon the same underlying inference engine established in the current prototype.

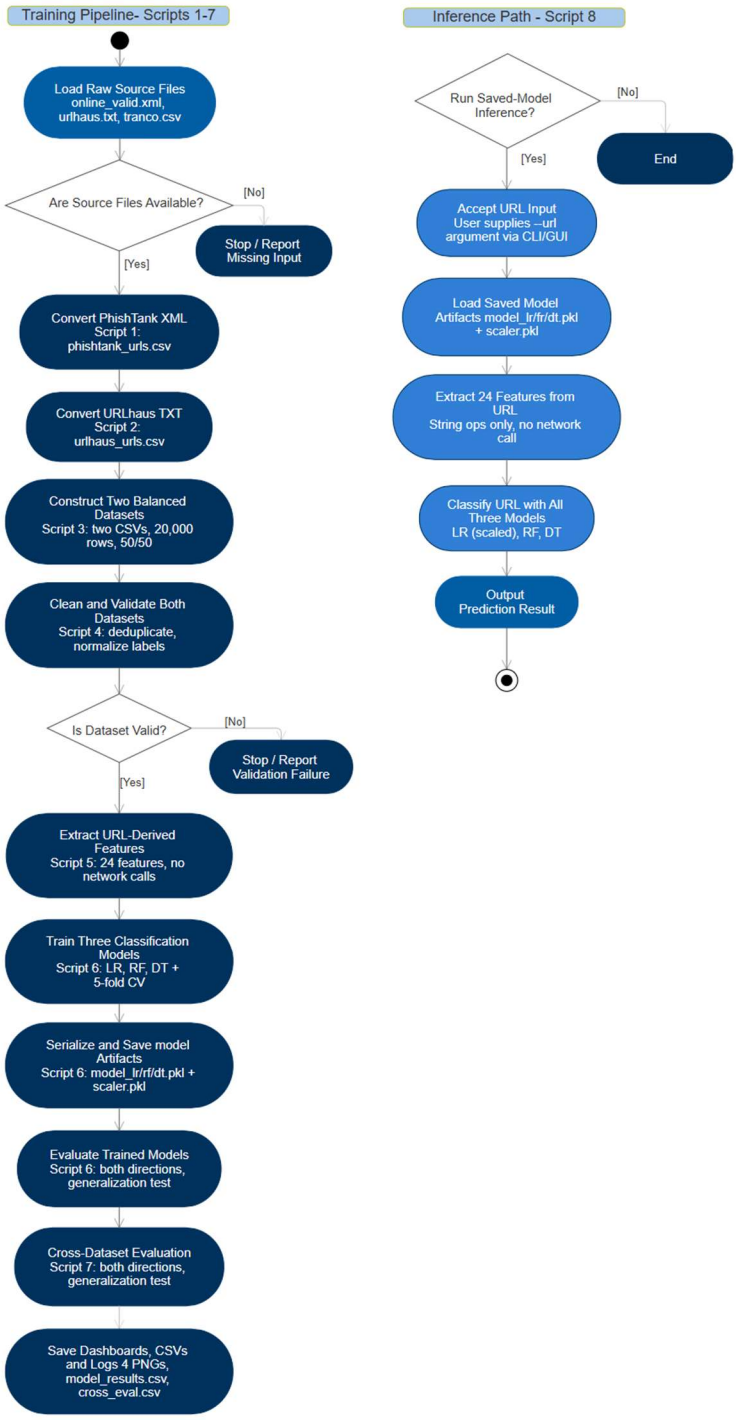


Figure 130 UML Activity Diagram- Training Pipeline and Inference Path
Activity diagram modeling the sequential control flow of the seven-script training pipeline from raw source file loading through dashboard and artifact generation, alongside the independent on-demand inference path, including fail-fast error branches for missing input files, validation failures, and schema mismatches.

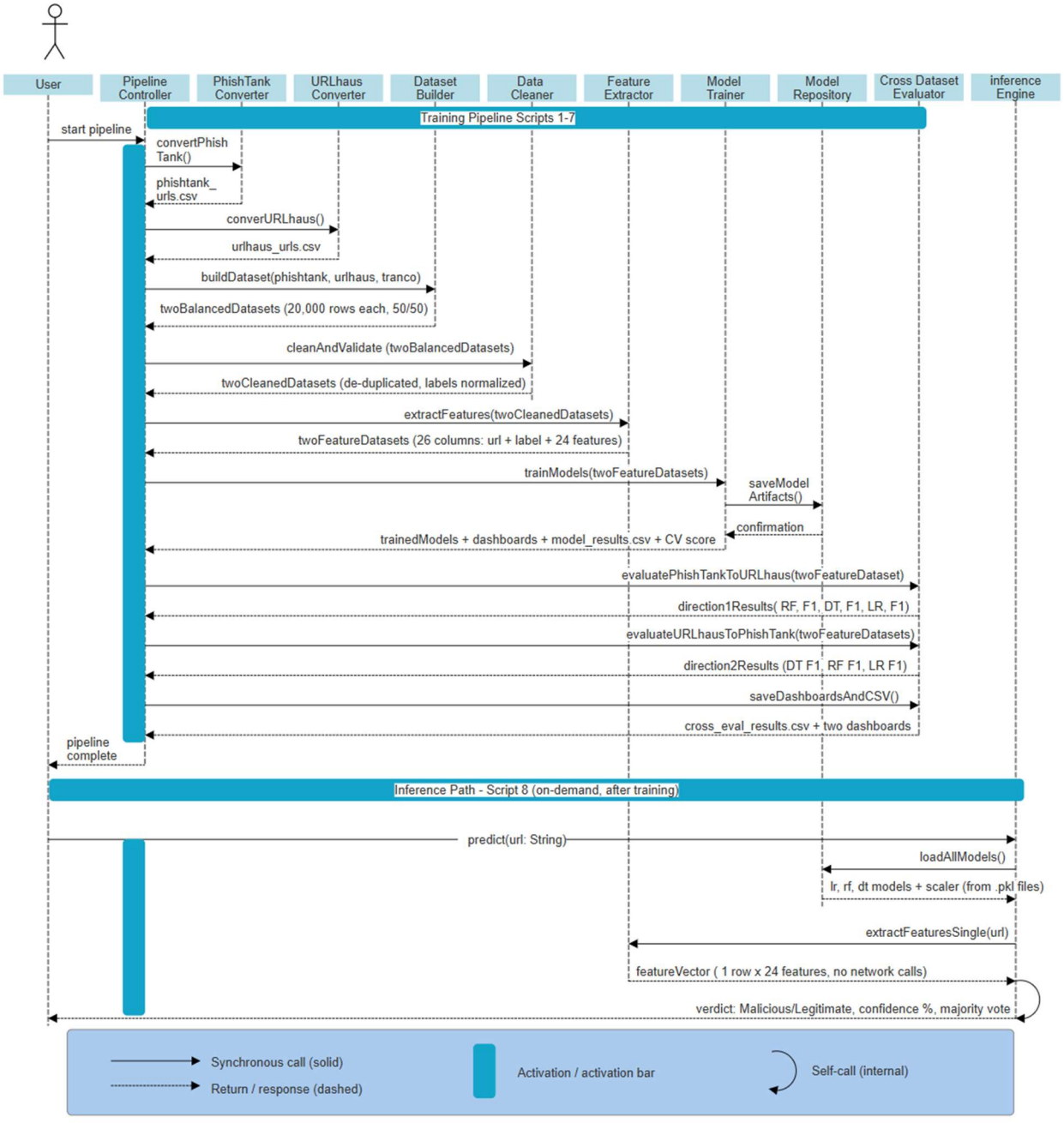


Figure 131 UML Sequence Diagram- End-to-End Pipeline Interaction
Sequence diagram illustrating the interactions between system components across the training pipeline and inference path, showing the synchronous call and return flow from pipeline initiation through cross-dataset evaluation completion and the independent inference sequence from URL input through priority-vote verdict output.

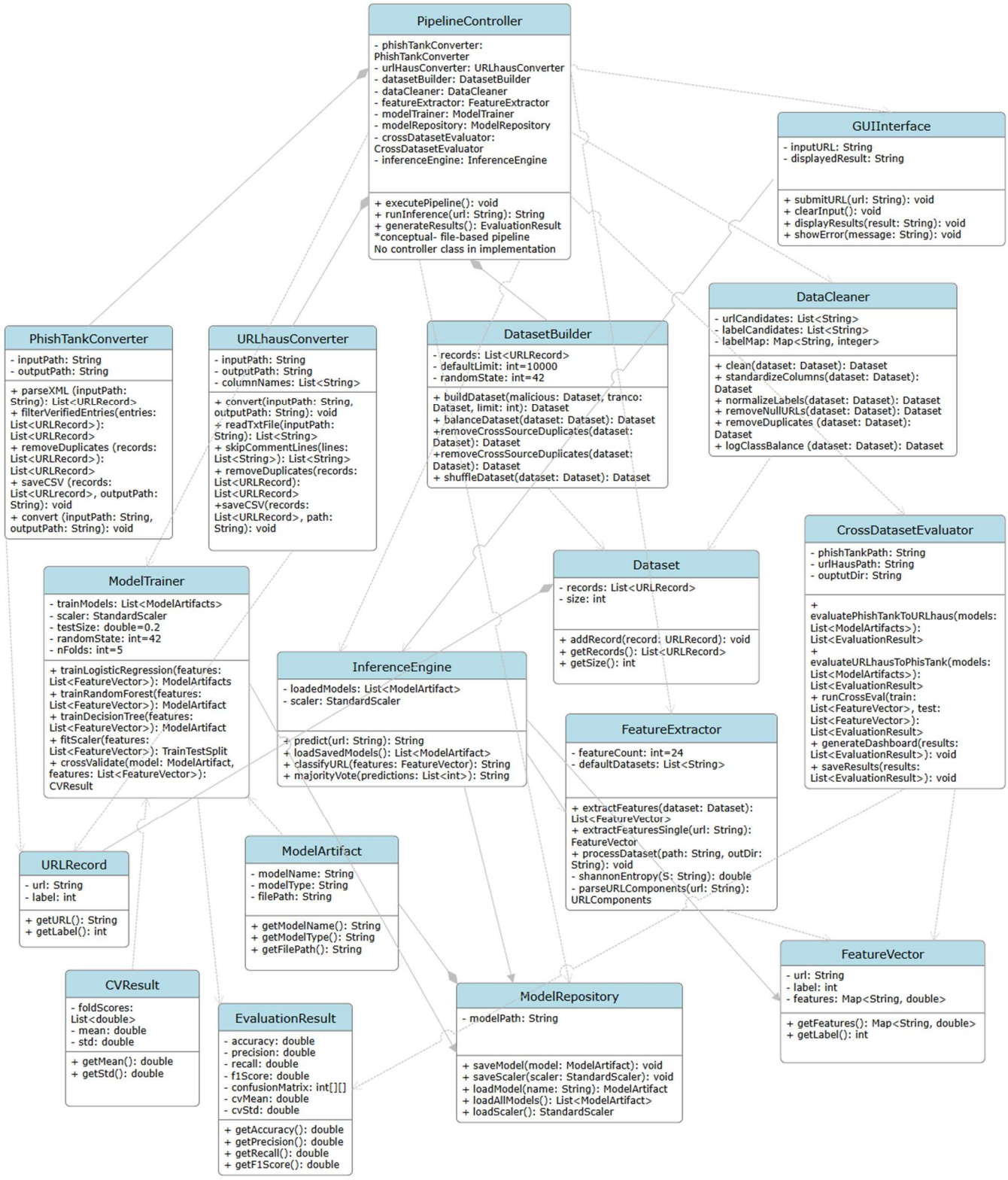

PipelineController
- phishTankConverter: PhishTankConverter
- urlHausConverter: URLhausConverter
- datasetBuilder: DatasetBuilder
- dataCleaner: DataCleaner
- featureExtractor: FeatureExtractor
- modelTrainer: ModelTrainer
- modelRepository: ModelRepository
- crossDatasetEvaluator: CrossDatasetEvaluator
- inferenceEngine: InferenceEngine
+ executePipeline(): void
+ runInference(url: String): String
+ generateResults(): EvaluationResult
*conceptual- file-based pipeline
No controller class in implementation
GUIInterface
- inputURL: String
- displayedResult: String
+ submitURL(url: String): void
+ clearInput(): void
+ displayResults(result: String): void
+ showError(message: String): void
DataCleaner
- urlCandidates: List<String>
- labelCandidates: List<String>
- labelMap: Map<String, integer>
+ clean(dataset: Dataset): Dataset
+ standardizeColumns(dataset: Dataset): Dataset
+ normalizeLabels(dataset: Dataset): Dataset
+ removeNullURLs(dataset: Dataset): Dataset
+ removeDuplicates (dataset: Dataset): Dataset
+ logClassBalance (dataset: Dataset): Dataset
PhishTankConverter
- inputPath: String
- outputPath: String
+ parseXML (inputPath: String): List<URLRecord>
+ filterVerifiedEntries(entries: List<URLRecord>): List<URLRecord>
+ removeDuplicates (records: List<URLRecord>): List<URLRecord>
+ saveCSV (records: List<URLrecord>, outputPath: String): void
+ convert (inputPath: String, outputPath: String): void
URLhausConverter
- inputPath: String
- outputPath: String
- columnNames: List<String>
+ convert(inputPath: String, outputPath: String): void
+ readTxtFile(inputPath: String): List<String>
+ skipCommentLines(lines: List<String>): List<String>
+ removeDuplicates(records: List<URLRecord): List<URLRecord>
+saveCSV(records: List<URLRecord>, path: String): void
DatasetBuilder
- records: List<URLRecord>
- defaultLimit: int=10000
- randomState: int=42
+ buildDataset(malicious: Dataset, tranco: Dataset, limit: int): Dataset
+ balanceDataset(dataset: Dataset): Dataset
+removeCrossSourceDuplicates(dataset: Dataset): Dataset
+removeCrossSourceDuplicates(dataset: Dataset): Dataset
+ shuffleDataset(dataset: Dataset): Dataset
CrossDatasetEvaluator
- phishTankPath: String
- urlHausPath: String
- ouptutDir: String
+ evaluatePhishTankToURLhaus(models: List<ModelArtifacts>): List<EvaluationResult>
+ evaluateURLhausToPhisTank(models: List<ModelArtifacts>): List<EvaluationResult>
+ runCrossEval(train: List<FeatureVector>, test: List<FeatureVector>): List<EvaluationResult>
+ generateDashboard(results: List<EvaluationResult>): void
+ saveResults(results: List<EvaluationResult>): void
Dataset
- records: List<URLRecord>
- size: int
+ addRecord(record: URLRecord): void
+ getRecords(): List<URLRecord>
+ getSize(): int
ModelTrainer
- trainModels: List<ModelArtifacts>
- scaler: StandardScaler
- testSize: double=0.2
- randomState: int=42
- nFolds: int=5
+ trainLogisticRegression(features: List<FeatureVector>): ModelArtifacts
+ trainRandomForest(features: List<FeatureVector>): ModelArtifact
+ trainDecisionTree(features: List<FeatureVector>): ModelArtifact
+ fitScaler(features: List<FeatureVector>): TrainTestSplit
+ crossValidate(model: ModelArtifact, features: List<FeatureVector>): CVResult
InferenceEngine
- loadedModels: List<ModelArtifact>
- scaler: StandardScaler
+ predict(url: String): String
+ loadSavedModels(): List<ModelArtifact>
+ classifyURL(features: FeatureVector): String
+ majorityVote(predictions: List<int>): String
FeatureExtractor
- featureCount: int=24
- defaultDatasets: List<String>
+ extractFeatures(dataset: Dataset): List<FeatureVector>
+ extractFeaturesSingle(url: String): FeatureVector
+ processDataset(path: String, outDir: String): void
- shannonEntropy(S: String): double
- parseURLComponents(url: String): URLComponents
URLRecord
- url: String
- label: int
+ getURL(): String
+ getLabel(): int
ModelArtifact
- modelName: String
- modelType: String
- filePath: String
+ getModelName(): String
+ getModelType(): String
+ getFilePath(): String
FeatureVector
- url: String
- label: int
- features: Map<String, double>
+ getFeatures(): Map<String, double>
+ getLabel(): int
CVResult
- foldScores: List<double>
- mean: double
- std: double
+ getMean(): double
+ getStd(): double
EvaluationResult
- accuracy: double
- precision: double
- recall: double
- f1Score: double
- confusionMatrix: int[][]
- cvMean: double
- cvStd: double
+ getAccuracy(): double
+ getPrecision(): double
+ getRecall(): double
+ getF1Score(): double
ModelRepository
- modelPath: String
+ saveModel(model: ModelArtifact): void
+ saveScaler(scaler: StandardScaler): void
+ loadModel(name: String): ModelArtifact
+ loadAllModels(): List<ModelArtifact>
+ loadScaler(): StandardScaler

Figure 132 UML Class Diagram- System Architecture
Class diagram representing the system's modular architecture through primary module classes corresponding to the eight pipeline scripts, supporting data classes, and their associations, illustrating the loosely coupled file-based integration model where all inter-component dependencies are mediated through persistent CSV artifacts rather than direct method invocation.

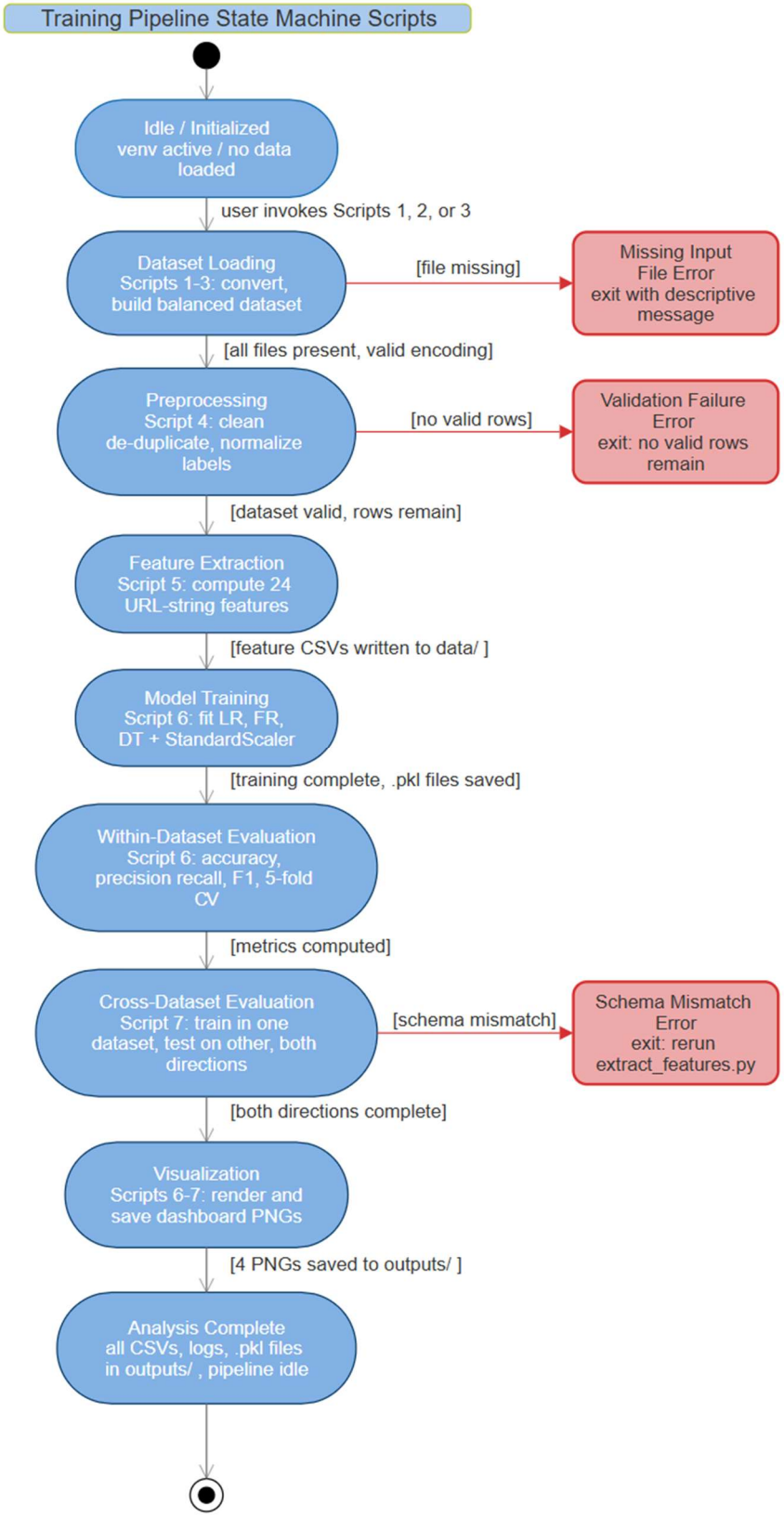

Training Pipeline State Machine Scripts
Idle / Initialized
venv active / no data
loaded
user invokes Scripts 1, 2, or 3
Dataset Loading
Scripts 1-3: convert,
build balanced dataset
[file missing]
Missing Input
File Error
exit with descriptive
message
[all files present, valid encoding]
Preprocessing
Script 4: clean
de-duplicate, normalize
labels
[no valid rows]
Validation Failure
Error
exit: no valid rows
remain
[dataset valid, rows remain]
Feature Extraction
Script 5: compute 24
URL-string features
[feature CSVs written to data/]
Model Training
Script 6: fit LR, FR,
DT + StandardScaler
[training complete, .pkl files saved]
Within-Dataset Evaluation
Script 6: accuracy,
precision recall, F1, 5-fold
CV
[metrics computed]
Cross-Dataset Evaluation
Script 7: train in one
dataset, test on other, both
directions
[schema mismatch]
Schema Mismatch
Error
exit: rerun
extract_features.py
[both directions complete]
Visualization
Scripts 6-7: render and
save dashboard PNGs
[4 PNGs saved to outputs/]
Analysis Complete
all CSVs, logs, .pkl files
in outputs/ , pipeline idle

Figure 133 UML State Diagram- Training Pipeline
State machine diagram modeling the training pipeline execution states form idle initialization through dataset loading, preprocessing, feature extraction, model training, within-dataset evaluation, cross-dataset evaluation, and visualization, including terminal error states from missing input files, validation failures, and schema mismatches.

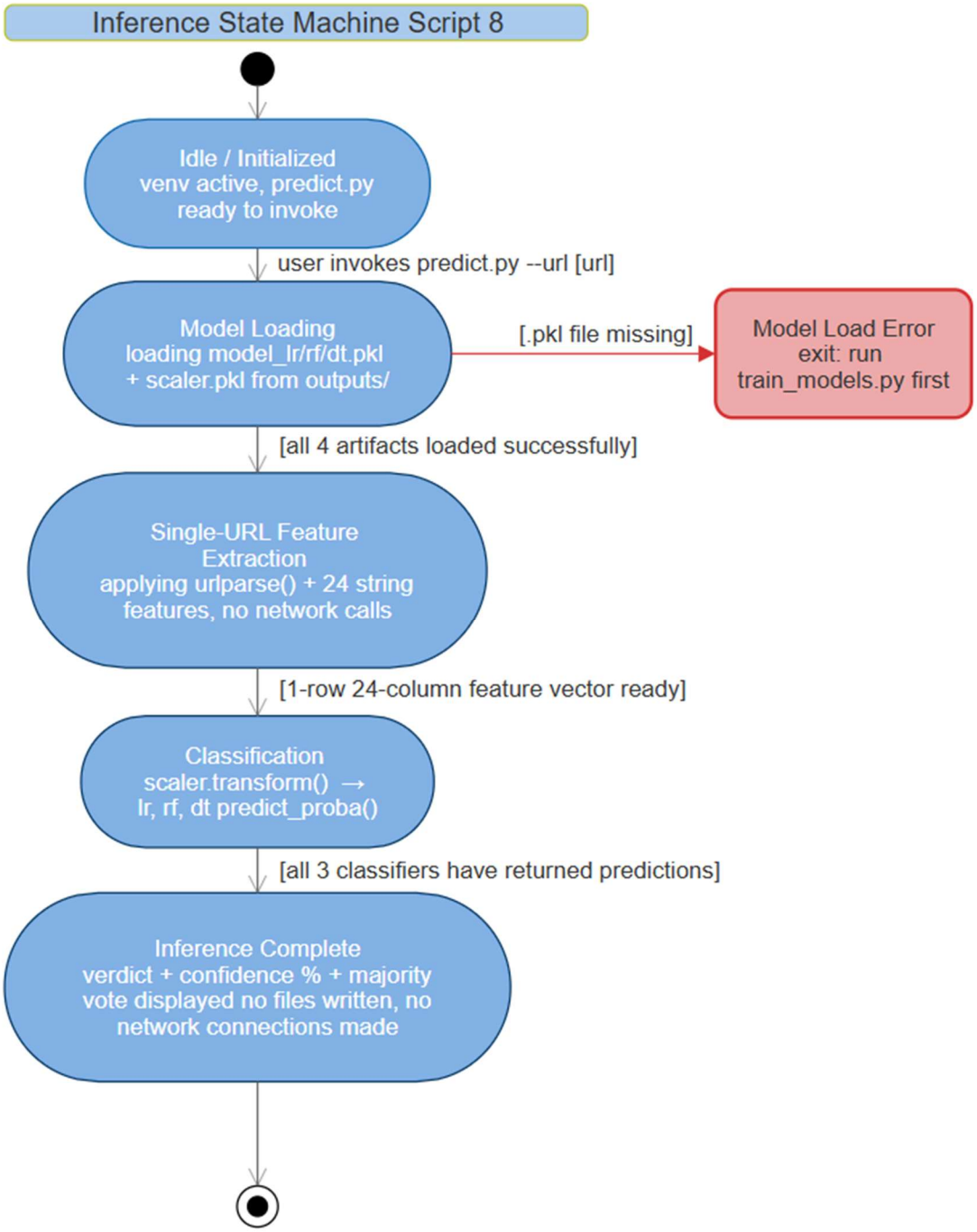


Figure 134 UML State Diagram- Inference Path
State machine diagram modeling the inference path execution states from idle initialization through model artifact loading, single-URL feature extraction, classification, and inference completion, including a terminal error state triggered when required pickle model files are absent from the outputs/ directory.

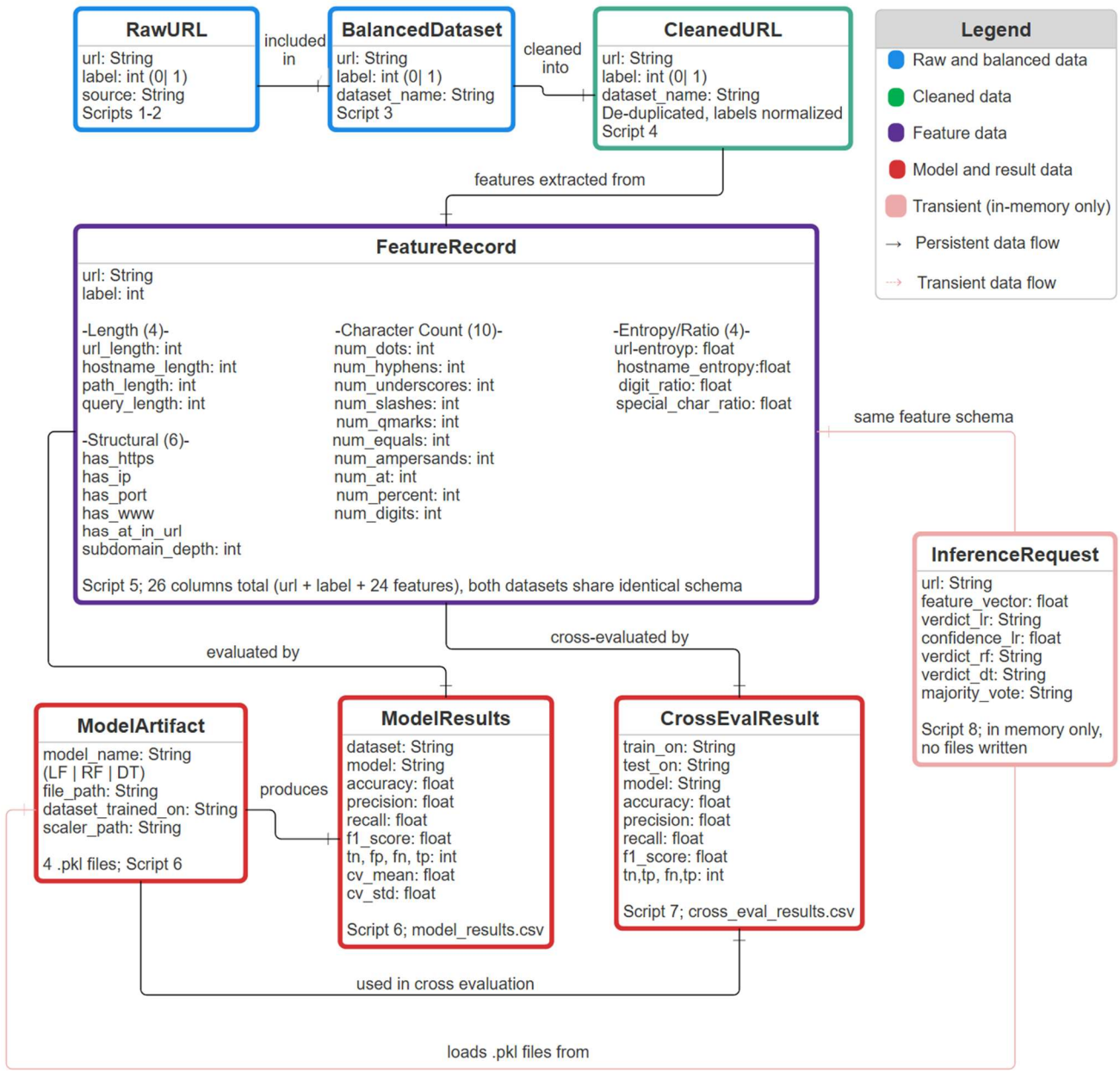


Figure 135 Entity-Relationship Diagram- Pipeline Data Flow
Logical data relationship diagram illustrating the sequential transformation of data from raw URL ingestion through balanced dataset construction, cleaning, feature engineering, model training, and evaluation, with the transient in-memory inference request entity representing the independent inference path that maintains a read-only dependency on trained model artifacts.